

Predictive analysis of naval incidents in the USA, 2002 - 2015:

Annex 5.2. Data Model MergedActivity

Author: [Oscar Anton](#)

Date: 2024

License: [CC BY-NC-ND 4.0 DEED](#)

Version: 0.9

0. Loadings

Libraries

```
In [1]: # System environment
import os

# Data general management
import numpy as np
import pandas as pd

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Sample balance
import cube # https://github.com/acdmammoths/paralle
from imblearn.over_sampling import SMOTENC # https://github.com/scikit-learn-contrib

# NaN imputation
from sklearn.experimental import enable_iterative_imputer # Needed for IterativeImputer
from sklearn.impute import IterativeImputer

# Data scaling
from sklearn.preprocessing import StandardScaler, LabelEncoder, label_binarize

# Data splitting
from sklearn.model_selection import train_test_split, KFold, cross_val_score

# Model training
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier

from keras.models import Sequential, Model, load_model
from keras.layers import Input, Dense, Dropout, concatenate
from keras.utils import plot_model
from keras.callbacks import EarlyStopping

# h2o framework
import h2o
from h2o.automl import H2OAutoML

# Model metrics
```

```

from sklearn.metrics import accuracy_score, matthews_corrcoef, f1_score, roc_auc_score,
from sklearn.inspection import permutation_importance

# Model Export
import joblib

# Model explainers
import dalex as dx

# Ignore warnings
import warnings
warnings.filterwarnings("ignore")

```

General Variables

```

In [2]: # Main data folders
merged_activity_folder = '../3.DataPreprocess/DataMergedActivity'
datasets_folder = 'Datasets'
models_folder = 'Models'

# Toggle for export data to external file
file_export_enabled = False
# Toggle for train model
train_model_enabled = False

# Available CPU cores for multiprocessing (training models)
n_jobs = os.cpu_count() - 1

# Random seed for reproducibility
seed = 42

```

Base dataframe

```

In [3]: # Load dataframe
merged_activity = pd.read_feather(merged_activity_folder + '/' + 'merged_activity.feather')

# Check dataframe
merged_activity.head(5)

```

```

Out[3]:

```

	activity_id	date	hour	region	latitude	longitude	watertype	event_type	dam
0	1475897	2002-01-01	03:45	Mississippi	37.017330	-88.274720	river	Grounding	L
1	1475897	2002-01-01	03:45	Mississippi	37.017330	-88.274720	river	Grounding	L
2	1477008	2002-01-01	13:53	East Coast	39.322020	-76.363650	ocean	Damage to the Environment	L
3	1477373	2002-01-01	18:10	Mississippi	31.525833	-87.971667	river	Material Failure (Vessels)	
4	1477402	2002-01-01	10:00	Gulf of Mexico	30.641667	-88.034167	ocean	Grounding	L

5 rows × 28 columns

1. Dataframe creation

1.1. Variable transformation

```
In [4]: # Dropping unnecessary variables for the models
columns_to_drop = ['vessel_id', 'imo_number', 'vessel_name', 'event_type',
                   'build_year', 'wave_hgt', 'visibility', 'casualty',
                   'pollution', 'flag_abbr', 'classification_society', 'solas_desc']

merged_activity = merged_activity.drop(columns=columns_to_drop)

# Renaming columns
merged_activity = merged_activity.rename(columns={'length': 'vessel_length', 'event_class': 'vessel_class'})

# Date & Time variables to continuous data
merged_activity['date'] = merged_activity['date'].dt.dayofyear

merged_activity['hour'] = (pd.to_numeric(merged_activity['hour'].str.split(':').str[0]) * 60 +
                          pd.to_numeric(merged_activity['hour'].str.split(':').str[1])/60)

# Function: group minority values
def lump_factorials(column, prop=0.008, other_level="other value"):
    counts = column.value_counts(normalize=True)
    mask = column.isin(counts[counts < prop].index)
    column[mask] = other_level
    return column

# Apply function for reducing variability in vessel_class
merged_activity[['vessel_class']] = merged_activity[['vessel_class']].apply(lump_factorials)

# Converting some columns to factors (categorical)
columns_to_factorize = ['region', 'watertype', 'damage_status', 'vessel_class', 'y']
merged_activity[columns_to_factorize] = merged_activity[columns_to_factorize].astype('category')

# Check structure
merged_activity.dtypes
```

```
Out[4]: activity_id      int32
date      int32
hour      float64
region    category
latitude  float64
longitude float64
watertype category
damage_status category
vessel_class category
age       int32
gross_ton int32
vessel_length float64
air_temp   float64
wind_speed float64
damage_assessment float64
y          category
dtype: object
```

```
In [5]: # Check target variable frequency
merged_activity['y'].value_counts()
```

```
Out[5]: y
Maritime Accidents      18467
Material Issues          17158
Critical Events          16938
Third-party Damages      8807
Onboard Emergencies     6630
Name: count, dtype: int64
```

1.2. Target variable balance

1.2.1. Subsampling: Cube Method

```
In [6]: def cube_subsampling(data, target_size):
        # Drop all incomplete cases if not lower than target size (keeping best information)
        if len(data.dropna()) >= target_size:
            data = data.dropna()

        # Constant column of "1s"
        ONE = np.full(len(data), 1)

        # Select numerical variables
        X1 = data.select_dtypes(include=['number']).reset_index(drop=True)

        # Categorical variables to numerical (one hot encoding)
        X2 = pd.get_dummies(data.drop(columns='y').select_dtypes(exclude=['number']).reset_index(drop=True))

        # Join data
        X = np.column_stack((ONE, X1, X2))

        # Inclusion probability (constant according to target size)
        init_probs = np.full(len(X), target_size / len(X))

        # Inclusion index (algorithm by Alexander Lee, Stefan Walzer-Goldfeld, Shukry Zablah)
        sample_indexes = cube.sample_cube_parallel(X, init_probs, n_jobs, is_pop_size_fixed=True)

        # Return selected rows according sample_indexes
        return data.loc[sample_indexes == 1, :]
```

Critical events: Subsampling to 9000 rows

```
In [7]: # Call cube_subsampling for y = Critical Events
CriticalEvents_cube = cube_subsampling(merged_activity[merged_activity['y'] == 'Critical Events'])

# Check sampled Length
print(f"Critical Events subsample dimensions: {CriticalEvents_cube.shape}")
```

Critical Events subsample dimensions: (9000, 16)

Maritime Accidents: Subsampling to 9000 rows

```
In [8]: # Call cube_subsampling for y = Maritime Accidents
MaritimeAccidents_cube = cube_subsampling(merged_activity[merged_activity['y'] == 'Maritime Accidents'])

# Check sampled Length
print(f"Maritime Accidents subsample dimensions: {MaritimeAccidents_cube.shape}")
```

Maritime Accidents subsample dimensions: (9000, 16)

Material Issues: Subsampling to 9000 rows

```
In [9]: # Call cube_subsampling for y = Critical Events
MaterialIssues_cube = cube_subsampling(merged_activity[merged_activity['y'] == 'Material

# Check sampled Length
print(f"Material Issues subsample dimensions: {MaterialIssues_cube.shape}")
```

Material Issues subsample dimensions: (9000, 16)

1.2.2. NaN imputation

Note: In this case NaN imputation is before oversampling because used algorithm needs complete cases to work properly

```
In [10]: # Identify columns with NaN
nan_columns = ['air_temp', 'wind_speed', 'damage_assessment']

# MICE (Multiple Imputation by Chained Equations ) imputation
imputer = IterativeImputer(random_state=seed)
imputed_values = imputer.fit_transform(merged_activity[nan_columns])

# Imputation apply
imputed_data = merged_activity.copy()
imputed_data[nan_columns] = imputed_values

# Check imputation
imputed_data.isna().sum()
```

```
Out[10]: activity_id      0
date      0
hour      0
region    0
latitude  0
longitude 0
watertype 0
damage_status 0
vessel_class 0
age      0
gross_ton 0
vessel_length 0
air_temp  0
wind_speed 0
damage_assessment 0
y         0
dtype: int64
```

Imputation quality check

```
In [11]: # Calculate means for before and after imputed variables
mean_before_impute = merged_activity[nan_columns].mean().to_frame(name='Mean before impute')
mean_after_impute = imputed_data[nan_columns].mean().to_frame(name='Mean after impute')

# Join both dataframes, transposed
nan_means = pd.concat([mean_before_impute, mean_after_impute], axis=1).T

# Calculate difference porcentaje
difference_percentage = 100 * (nan_means.loc['Mean after impute'] - nan_means.loc['Mean
# Add values to the table with % format
nan_means.loc['Diference for Mean (%)'] = difference_percentage.apply(lambda x: f"{x:.4f

# Show joined data
nan_means
```

Out[11]:

	air_temp	wind_speed	damage_assessment
Mean before impute	149.763586	50.583439	122959.803912
Mean after impute	149.75645	50.760162	122959.803926
Diference for Mean (%)	-0.0048%	0.3494%	0.0000%

In [12]:

```
# Calculate standard deviation for before and after imputed variables
stds_before_impute = merged_activity[nan_columns].std().to_frame(name='Std before impute')
stds_after_impute = imputed_data[nan_columns].std().to_frame(name='Std after impute')

# Join both dataframes, transposed
nan_std = pd.concat([stds_before_impute, stds_after_impute], axis=1).T

# Calculate diference percentaje
difference_percentage = 100 * (nan_std.loc['Std after impute'] - nan_std.loc['Std before impute'])
# Add values to the table with % format
nan_std.loc['Diference for Std (%)'] = difference_percentage.apply(lambda x: f"{x:.4f}%")

# Show joined data
nan_std
```

Out[12]:

	air_temp	wind_speed	damage_assessment
Std before impute	94.999003	30.646614	3887042.365253
Std after impute	91.242978	24.480288	3885441.465189
Diference for Std (%)	-3.9538%	-20.1207%	-0.0412%

1.2.3. Oversampling: SMOTE (Synthetic Minority Over-sampling Technique)

In [13]:

```
# Split target variable
y = imputed_data['y']
X = imputed_data.drop(columns=['y'])

# Set target size for variables to be oversampled
target_size = 9000

# Create SMOTE-NC (Synthetic Minority Over-sampling Technique for Nominal and Continuous)
smote = SMOTENC(sampling_strategy={'Onboard Emergencies': target_size, 'Third-party Damages': target_size},
categorical_features=[3, 6, 7, 8],
random_state=seed)

# Apply SMOTE-NC fit
X_resampled, y_resampled = smote.fit_resample(X, y)

# Back to join data
resampled_data = pd.concat([X_resampled, y_resampled], axis=1)

# Check balance
resampled_data['y'].value_counts()

# Extract resampled 'Onboard Emergencies' class
onboard_emergencies_smote = resampled_data[resampled_data['y'] == 'Onboard Emergencies']
# Check sampled length
print(f"Onboard Emergencies oversample dimensions: {onboard_emergencies_smote.shape}")

# Extract resampled 'Third-party Damages' class
```

```
thirdparty_damages_smote = resampled_data[resampled_data['y'] == 'Third-party Damages']
# Check sampled length
print(f"Third-party Damages oversample dimensions: {thirdparty_damages_smote.shape}")
```

Onboard Emergencies oversample dimensions: (9000, 16)

Third-party Damages oversample dimensions: (9000, 16)

1.2.4. All events data join

```
In [14]: # Sampled pieces join
merged_activity_general = pd.concat([CriticalEvents_cube, MaritimeAccidents_cube, Materi
                                     onboard_emergencies_smote, thirdparty_damages_s

# Data Length check
print(f"Joined dataframe dimensions: {merged_activity_general.shape}")
```

Joined dataframe dimensions: (45000, 16)

1.3. Data consolidation

```
In [15]: # Drop statistically irrelevant variables, according to EDA
merged_activity_general = merged_activity_general.drop(columns=['date', 'latitude', 'dam

# Export to external file
if file_export_enabled :
    (merged_activity_general
     .reset_index(drop=True)
     .to_feather(datasets_folder + '/' + 'merged_activity_general.feather'))
else:
    merged_activity_general = pd.read_feather(datasets_folder + '/' + 'merged_activity_g

# Data values check
print(f"Consolidated dataframe dimensions: {merged_activity_general.shape}")
merged_activity_general.head(5)
```

Consolidated dataframe dimensions: (45000, 13)

```
Out[15]:
```

	activity_id	hour	region	longitude	watertype	damage_status	vessel_class	age	gross_ton
0	1475186	22.00	Gulf of Mexico	-82.770848	ocean	Damaged	Recreational	39	13
1	1484669	13.92	West Coast	-122.222294	ocean	Damaged	Passenger Ship	1	9
2	1482821	18.50	West Coast	-124.128000	ocean	Damaged	Fishing Vessel	36	49
3	1479220	11.83	Gulf of Mexico	-82.478333	ocean	Damaged	Passenger Ship	58	1
4	1730323	10.00	Gulf of Mexico	-93.825000	ocean	Undamaged	Towing Vessel	30	16

1.3.1. Explanatory variables: Scale numerics & OHE not numerics

```
In [16]: # Scale numeric variables
data_scaled = pd.DataFrame(StandardScaler()
                             .fit_transform(merged_activity_general
                                             .select_dtypes(include=['number'])))

# Rename numeric column names
```

```

data_scaled.columns = merged_activity_general.select_dtypes(include=['number']).columns

# One hot encoding for not numeric variables
data_ohe = (pd.get_dummies(merged_activity_general
                          .drop(columns=['y'])
                          .select_dtypes(exclude=['number']))
            .astype(int))

# X: Scaled and ohe join
X = pd.concat([data_ohe, data_scaled], axis=1)

# Verify variables
for column in X.columns:
    print(f"Variable name: {column} | Type: {X[column].dtype} | Levels: {X[column].nunique}")

```

```

Variable name: region_Alaska | Type: int32 | Levels: 2
Variable name: region_Canada | Type: int32 | Levels: 1
Variable name: region_East Coast | Type: int32 | Levels: 2
Variable name: region_Gulf of Mexico | Type: int32 | Levels: 2
Variable name: region_Mississippi | Type: int32 | Levels: 2
Variable name: region_West Coast | Type: int32 | Levels: 2
Variable name: watertype_ocean | Type: int32 | Levels: 2
Variable name: watertype_river | Type: int32 | Levels: 2
Variable name: damage_status_Actual Total Loss | Type: int32 | Levels: 2
Variable name: damage_status_Damaged | Type: int32 | Levels: 2
Variable name: damage_status_Total Constructive Loss: Salvaged | Type: int32 | Levels: 2
Variable name: damage_status_Total Constructive Loss: Unsalvaged | Type: int32 | Levels: 2
Variable name: damage_status_Undamaged | Type: int32 | Levels: 2
Variable name: vessel_class_Barge | Type: int32 | Levels: 2
Variable name: vessel_class_Bulk Carrier | Type: int32 | Levels: 2
Variable name: vessel_class_Fishing Vessel | Type: int32 | Levels: 2
Variable name: vessel_class_General Dry Cargo Ship | Type: int32 | Levels: 2
Variable name: vessel_class_Miscellaneous Vessel | Type: int32 | Levels: 2
Variable name: vessel_class_Offshore | Type: int32 | Levels: 2
Variable name: vessel_class_Passenger Ship | Type: int32 | Levels: 2
Variable name: vessel_class_Recreational | Type: int32 | Levels: 2
Variable name: vessel_class_Tank Ship | Type: int32 | Levels: 2
Variable name: vessel_class_Towing Vessel | Type: int32 | Levels: 2
Variable name: vessel_class_other value | Type: int32 | Levels: 2
Variable name: activity_id | Type: float64 | Levels: 29520
Variable name: hour | Type: float64 | Levels: 3938
Variable name: longitude | Type: float64 | Levels: 14647
Variable name: age | Type: float64 | Levels: 123
Variable name: gross_ton | Type: float64 | Levels: 4227
Variable name: vessel_length | Type: float64 | Levels: 5743
Variable name: air_temp | Type: float64 | Levels: 14304
Variable name: wind_speed | Type: float64 | Levels: 11708

```

1.3.2. Target variable: Numeric Labeling

```

In [17]: # Label encoding
label_encoder = LabelEncoder()
y = pd.Series(label_encoder.fit_transform(merged_activity_general['y']))

# Verify values
np.unique(y)

```

```
Out[17]: array([0, 1, 2, 3, 4])
```

1.4. Train / Test Split

```
In [18]: # Data splitting
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=se
```

```
In [19]: # Save / Load dataframes in one file (h5 format for multiple data)
if file_export_enabled :
    dfs = {'X_train':X_train, 'X_test':X_test, 'y_train':y_train, 'y_test':y_test}
    for key, df in dfs.items():
        df.to_hdf(datasets_folder + '/' + 'datasets_MA_splited.h5', key=key, format='tab
        print(f'{key} {df.shape} saved')
else:
    dfs = ['X_train', 'X_test', 'y_train', 'y_test']
    for df in dfs:
        globals()[df] = pd.read_hdf(datasets_folder + '/' + 'datasets_MA_splited.h5', ke
        print(f'{df} {eval(df).shape} loaded')
```

X_train (36000, 32) loaded

X_test (9000, 32) loaded

y_train (36000,) loaded

y_test (9000,) loaded

2. Model Training

Performance functions (sklearn)

Parameters

```
In [20]: # Function: Show Parameters of trained model
def show_parameters(model):
    # Get model parameters and print as a one row list
    print(f'Parameters:')
    params = model.get_params()
    for param_name, param_value in params.items():
        print(f" {param_name}: {param_value}")
```

Metrics

```
In [21]: # Function: Table with main metrics data
def ma_model_metrics(model, X, y, styled=False):
    # Predictions
    predict = pd.Series(model.predict(X))
    predict_prob = pd.DataFrame(model.predict_proba(X))

    # Target variable
    y_ohe = pd.DataFrame(label_binarize(y, classes=np.unique(y)))

    # Calculate main metrics
    roc_auc = round(roc_auc_score(y_ohe, predict_prob, multi_class='ovr'), 4)
    accuracy = round(accuracy_score(y, predict), 4)
    loss = round(log_loss(y_ohe, predict_prob), 4)
    kappa = round(cohen_kappa_score(y, predict), 4)
    mcc = round(matthews_corrcoef(y, predict), 4)
    f1 = round(f1_score(y, predict, average='macro'), 4)

    # Build dataframe metric - value
    df = pd.DataFrame([['ROC AUC:', roc_auc], ['Accuracy:', accuracy], ['Log loss:', los
```

```

        ['Kappa:', kappa], ['MCC:', mcc], ['F1:', f1]],
        columns=('metric', 'value'))

# Add title
if styled:
    title = f'{model.__class__.__name__} Training'
    df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])
    return df.style.format(precision=4).hide()
else:
    return df

```

Recall & Precision

```

In [22]: # Function: Table for recall & precision, sensitivity & specificity
def sens_spec(model, X, y, styled=False):
    # Predictions (absolute)
    y_pred = model.predict(X)

    # Recall & Precision values
    recall = round(recall_score(y, y_pred, average='macro'), 4)
    precision = round(precision_score(y, y_pred, average='macro'), 4)

    # Confusion matrix
    conf_matrix = confusion_matrix(y, y_pred)

    # List compression for sens & spec values calculation
    sensitivity, specificity = zip(*[(round(recall_score(y, y_pred, labels=[i], average=
        round(conf_matrix[i, i] / sum(conf_matrix[:, i]), 4)
        for i in range(len(conf_matrix)))])

    # Labels and indexes
    column_labels = label_encoder.inverse_transform(model.classes_)
    index_1 = ['Recall:', 'Precision:']
    index_2 = [recall, precision]
    index_ = [' - ', ' - ']
    index_3 = ['Sensitivity:', 'Specificity:']

    # Build multiindex table
    df = pd.DataFrame([sensitivity, specificity])
    df.columns = column_labels
    df.index = [index_1, index_2, index_, index_3]

    # Dataframe style
    if styled:
        title = f'{model.__class__.__name__} Model'
        df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])
        return df.style.format(precision=4).set_table_styles([{'selector': 'th.col_headi
    else:
        return df

```

Confusion Matrix

```

In [23]: # Function: Table with Confusion Matrix
def ma_confusion_matrix_table(model, X, y):
    # Predictions (absolute)
    y_pred = model.predict(X)

    # Get labels decoding target variable
    labels = label_encoder.inverse_transform(model.classes_)

    # Build table
    df = pd.DataFrame(confusion_matrix(y, y_pred),

```

```

        columns=pd.MultiIndex.from_product([[f'{model.__class__.__name__}_{class_name}'],
        index=labels)

# Dataframe style
styled_df = df.style.set_table_styles([
    {'selector': 'th.col_heading', 'props': 'text-align: center;'},
    {'selector': 'td', 'props': 'text-align: center;'},
], overwrite=False)

return styled_df

```

Multiclass ROC Curves

```

In [24]: # Function: Plot Multiclass ROC Curve
def roc_curve_plot(model, X, y):
    # Predictions
    y_pred_proba = model.predict_proba(X)

    # Data binarize for auc calculation
    y_bin = label_binarize(y, classes=model.classes_)

    # Compute ROC curve and ROC area for each class
    fpr = dict()
    tpr = dict()
    roc_auc = dict()
    for i in model.classes_:
        fpr[i], tpr[i], _ = roc_curve(y_bin[:, i], y_pred_proba[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Compute micro-average ROC curve and ROC area
    fpr["micro"], tpr["micro"], _ = roc_curve(y_bin.ravel(), y_pred_proba.ravel())
    roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

    # Plot ROC curve for each class
    plt.figure()
    colors = sns.color_palette("hls", 5)
    for i, color in zip(model.classes_, colors):
        plt.plot(fpr[i], tpr[i], color=color, lw=2,
                 label='ROC of class {0} (AUC = {1:0.2f})'.format(i, roc_auc[i]))

    # Plot micro-average ROC curve
    plt.plot(fpr["micro"], tpr["micro"], color='grey', lw=1, linestyle='dashed',
             label='micro-average ROC (AUC = {0:0.2f})'.format(roc_auc["micro"]))

    plt.plot([0, 1], [0, 1], color='black', lw=1, linestyle='dotted')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.01])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC (Multiclass) for {model.__class__.__name__}')
    plt.legend(loc="lower right", fontsize="8")
    plt.show()

```

Feature importances

```

In [25]: # Function: Built a dataframe for feature importances (for sklearn regular models)
def sklearn_importances(model):
    # Dataframe creation for importance values in sklearn, sorted by importance_value
    importances = (pd.DataFrame({'variable_name': model.feature_names_in_,
                                'importance_value': model.feature_importances_})
                    .sort_values(by='importance_value', ascending=False)
                    .reset_index(drop=True))

```

```

# Add custom attributes for model name and calculation method
importances.model_name = model.__class__.__name__
importances.importance_method = 'feature_importances_ (sklearn)'

return importances

```

```

In [26]: # Function: Built a dataframe for feature importances, according to feature permutation
def permutation_importances(model, X, y, repeats=3):
    # Feature permutation importance calculation (from sklearn.inspection)
    importance_values = permutation_importance(model, X, y, n_repeats=repeats, random_state=None)

    # Importance absolute values and normalization
    importance_values = np.array(abs(importance_values))
    importance_values = importance_values / importance_values.sum()

    # Built importances dataframe
    importances = (pd.DataFrame({'variable_name': X.columns,
                                'importance_value': importance_values})
                    .sort_values(by='importance_value', ascending=False)
                    .reset_index(drop=True))

    # Add custom attribute for model name and calculation method
    importances.model_name = model.__class__.__name__
    importances.importance_method = 'feature permutation'

    return importances

```

```

In [27]: # Function: Horizontal bar plot for importance values
def plot_importances(importances_df, top=25):
    plt.figure()
    bars = plt.barh(importances_df['variable_name'][:top], importances_df['importance_value'][:top])
    plt.title(f'Top {top} Feature importances of {importances_df.model_name} model')
    plt.xlabel(f'Relative Feature importance, according to {importances_df.importance_method}')
    plt.gca().invert_yaxis()

    # Add value labels on the bars
    for i, bar in enumerate(bars):
        plt.text(0, bar.get_y() + bar.get_height() / 2, f'{importances_df["importance_value"].values[i]:.2f}')

    plt.show()

```

2.1. Bayesian networks

2.1.1. Naïve Bayes -Gaussian- (NB)

Model Training

```

In [28]: if train_model_enabled :
    # Define model parameters
    params = {} # No parameters for this model

    # Create and train model
    nb_MA_train = GaussianNB(**params).fit(X_train, y_train)

    # Save to external file
    joblib.dump(nb_MA_train, models_folder + '/' + 'nb_MA_train.pkl')
    print(f'{nb_MA_train.__class__.__name__} model exported to {models_folder} folder')
else:
    # Load model from external file

```

```

nb_MA_train = joblib.load(models_folder + '/' + 'nb_MA_train.pkl')
print(f'{nb_MA_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(nb_MA_train)

```

GaussianNB model loaded from Models folder

Parameters:

priors: None

var_smoothing: 1e-09

Model Metrics

```

In [29]: # Model performance main metrics
ma_model_metrics(nb_MA_train, X_test, y_test, styled=True)

```

Out[29]: **GaussianNB Training**

metric	value
ROC AUC:	0.6579
Accuracy:	0.3252
Log loss:	4.3607
Kappa:	0.1592
MCC:	0.1760
F1:	0.2853

```

In [30]: # recall & precision, sensitivity & specificity
sens_spec(nb_MA_train, X_test, y_test, styled=True)

```

Out[30]:

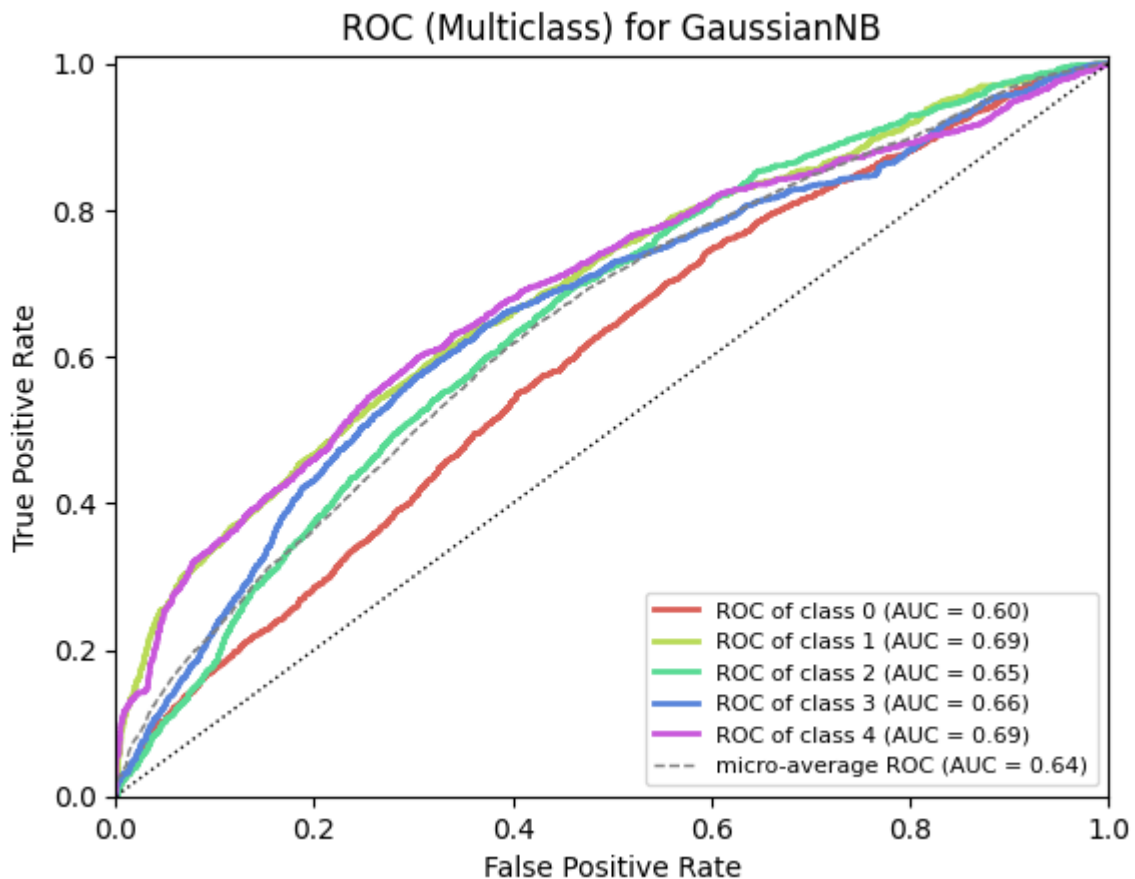
GaussianNB Model								
			Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third- party Damages	
Recall:	0.328700	- Sensitivity:	0.0776	0.6571	0.5286	0.0902	0.2897	
Precision:	0.379200	- Specificity:	0.3956	0.2827	0.2993	0.3980	0.5202	

```
In [31]: # Confusion matrix
ma_confusion_matrix_table(nb_MA_train, X_test, y_test)
```

Out[31]:

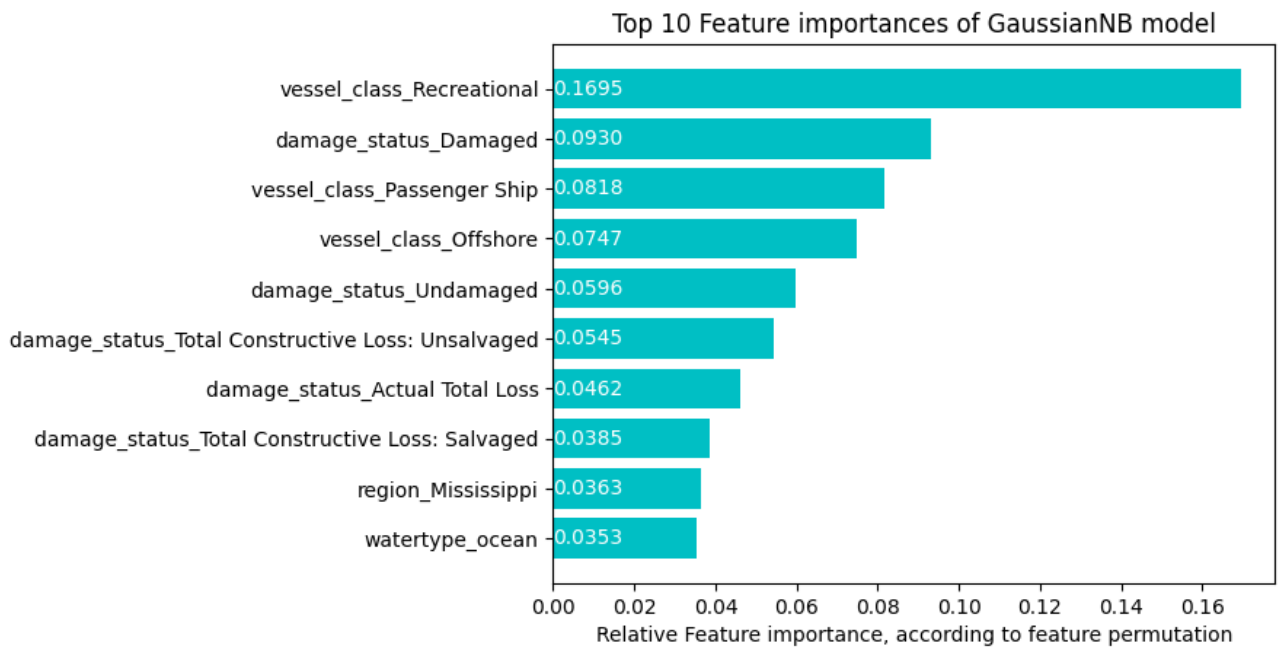
GaussianNB: Confusion Matrix					
	Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events	144	677	812	90	133
Maritime Accidents	19	1125	438	26	104
Material Issues	29	666	969	39	130
Onboard Emergencies	121	811	560	160	121
Third-party Damages	51	700	459	87	529

```
In [32]: # Plot Receiver Operating Characteristic (ROC) curves for each class
roc_curve_plot(nb_MA_train, X_test, y_test)
```



Feature importances

```
In [33]: # Call previous functions for plotting top 10 importances according to theta values
plot_importances(permutation_importances(nb_MA_train, X_train, y_train), top=10)
```



2.2. Gradient Boosting Models

2.2.1. Gradient Boosting Machine (GBM)

Model Training

```
In [34]: if train_model_enabled :
# Define model parameters
params = {
    'n_estimators': 500,           # Number of trees
    'learning_rate': 0.1,         # Contribution of each tree to the model
    'max_depth': 3,              # Maximum levels of each tree
    'random_state': seed,        # Random seed for reproducibility
}

# Create and train model
GBM_MA_train = GradientBoostingClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(GBM_MA_train, models_folder + '/' + 'GBM_MA_train.pkl')
print(f'{GBM_MA_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
GBM_MA_train = joblib.load(models_folder + '/' + 'GBM_MA_train.pkl')
print(f'{GBM_MA_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(GBM_MA_train)
```

GradientBoostingClassifier model loaded from Models folder

Parameters:

```
ccp_alpha: 0.0
criterion: friedman_mse
init: None
learning_rate: 0.1
loss: log_loss
max_depth: 3
max_features: None
max_leaf_nodes: None
min_impurity_decrease: 0.0
min_samples_leaf: 1
min_samples_split: 2
min_weight_fraction_leaf: 0.0
n_estimators: 500
n_iter_no_change: None
random_state: 42
subsample: 1.0
tol: 0.0001
validation_fraction: 0.1
verbose: 0
warm_start: False
```

Model Metrics

```
In [35]: # Model performance main metrics
ma_model_metrics(GBM_MA_train, X_test, y_test, styled=True)
```

Out[35]: **GradientBoostingClassifier Training**

metric	value
ROC AUC:	0.7914
Accuracy:	0.4832
Log loss:	1.2390
Kappa:	0.3541
MCC:	0.3549
F1:	0.4823

```
In [36]: # recall & precision, sensitivity & specificity
sens_spec(GBM_MA_train, X_test, y_test, styled=True)
```

Out[36]: **GradientBoostingClassifier Model**

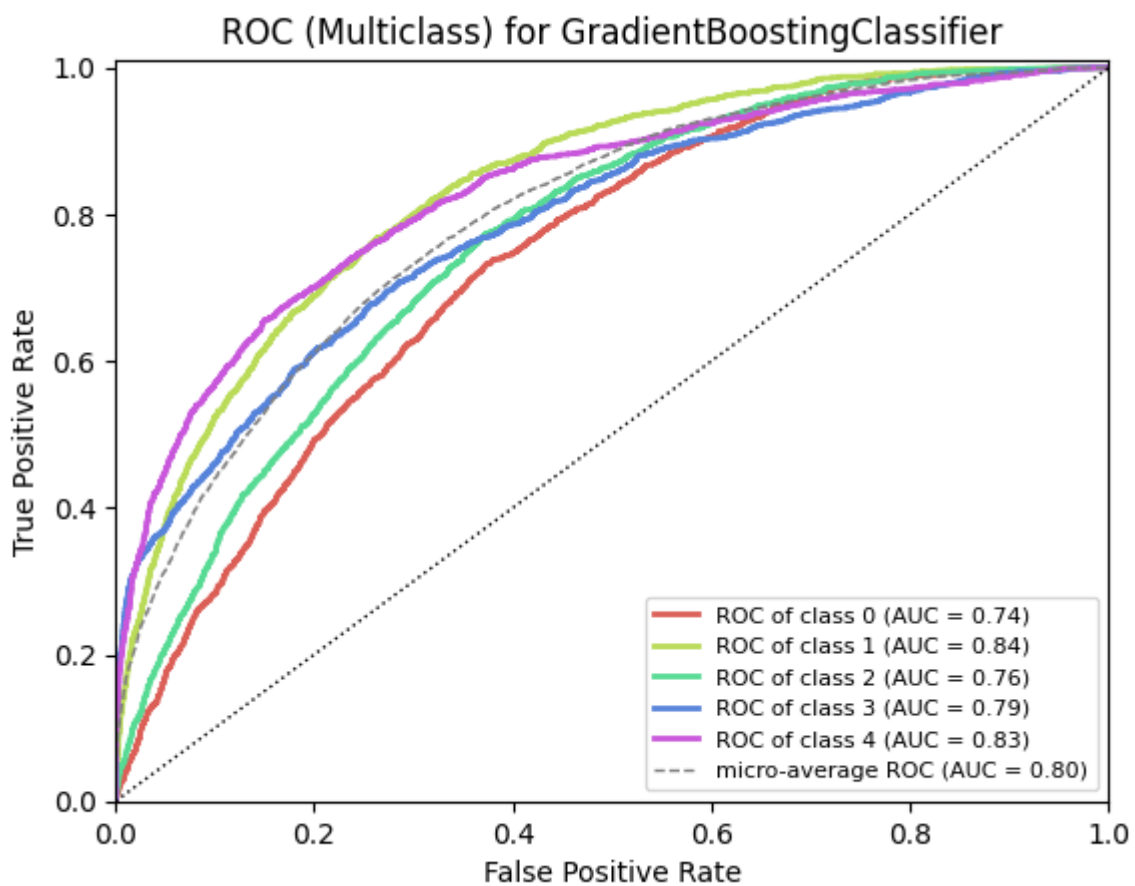
			Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third- party Damages
Recall:	0.484900	- Sensitivity:	0.3297	0.5894	0.4735	0.4597	0.5723
Precision:	0.483500	- Specificity:	0.3859	0.5086	0.4189	0.5330	0.5714

```
In [37]: # Confusion matrix
ma_confusion_matrix_table(GBM_MA_train, X_test, y_test)
```

Out[37]:

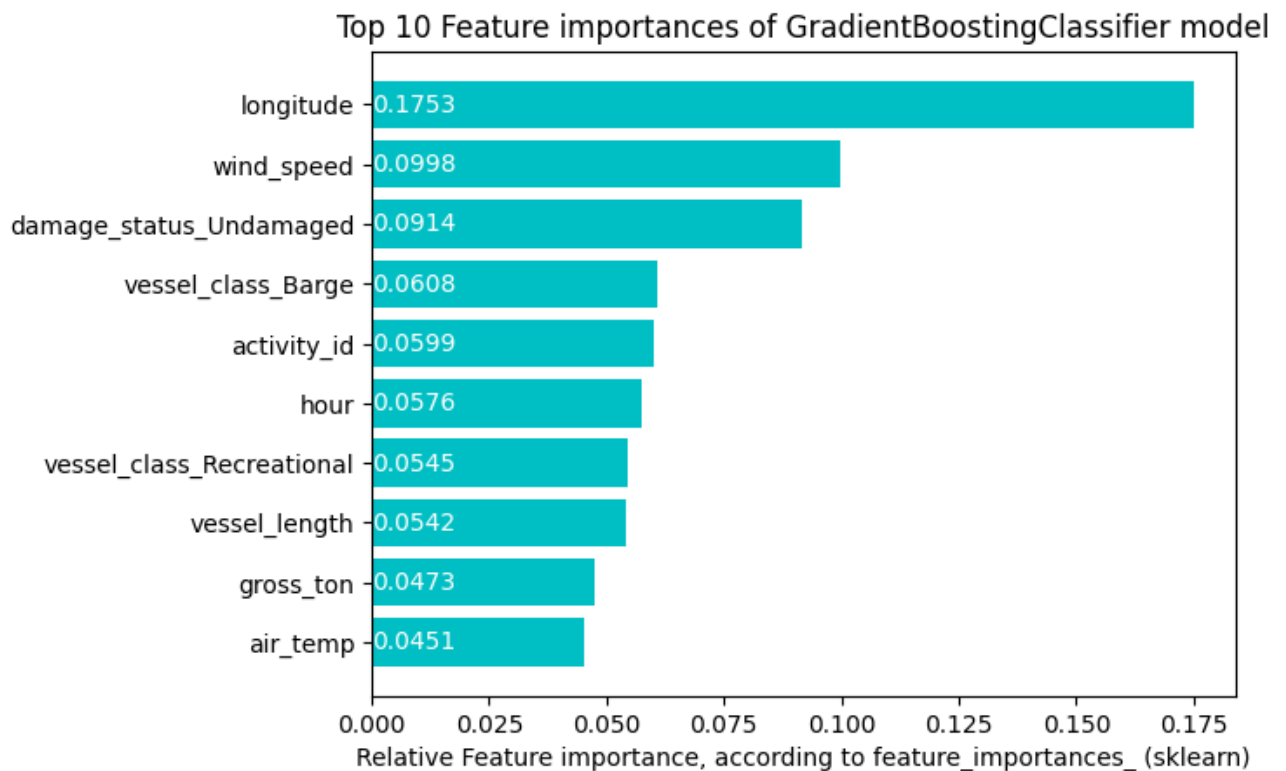
GradientBoostingClassifier: Confusion Matrix					
	Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events	612	295	538	199	212
Maritime Accidents	169	1009	235	123	176
Material Issues	356	281	868	159	169
Onboard Emergencies	269	193	269	815	227
Third-party Damages	180	206	162	233	1045

In [38]: *# Plot Receiver Operating Characteristic (ROC) curves for each class*
 roc_curve_plot(GBM_MA_train, X_test, y_test)



Feature importances

In [39]: *# Call previous functions for plotting top 10 importances according to feature_importance*
 plot_importances(sklearn_importances(GBM_MA_train), top=10)



2.3. More Models

2.3.1. Random Forest (RF)

Model Training

```
In [40]: if train_model_enabled :
# Define model parameters
params = {
    'n_estimators': 100,           # Number of trees in the forest
    'max_depth': None,            # Maximum depth of the trees (no restrictions)
    'random_state': seed         # Random seed for reproducibility
}

# Create and train model
rf_MA_train = RandomForestClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(rf_MA_train, models_folder + '/' + 'rf_MA_train.pkl')
print(f'{rf_MA_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
rf_MA_train = joblib.load(models_folder + '/' + 'rf_MA_train.pkl')
print(f'{rf_MA_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(rf_MA_train)
```

RandomForestClassifier model loaded from Models folder

Parameters:

```
bootstrap: True
ccp_alpha: 0.0
class_weight: None
criterion: gini
max_depth: None
max_features: sqrt
max_leaf_nodes: None
max_samples: None
min_impurity_decrease: 0.0
min_samples_leaf: 1
min_samples_split: 2
min_weight_fraction_leaf: 0.0
monotonic_cst: None
n_estimators: 100
n_jobs: None
oob_score: False
random_state: 42
verbose: 0
warm_start: False
```

Model Metrics

```
In [41]: # Model performance main metrics
ma_model_metrics(rf_MA_train, X_test, y_test, styled=True)
```

Out[41]: **RandomForestClassifier Training**

metric	value
ROC AUC:	0.8587
Accuracy:	0.6427
Log loss:	1.1947
Kappa:	0.5532
MCC:	0.5561
F1:	0.6376

```
In [42]: # recall & precision, sensitivity & specificity
sens_spec(rf_MA_train, X_test, y_test, styled=True)
```

Out[42]: **RandomForestClassifier Model**

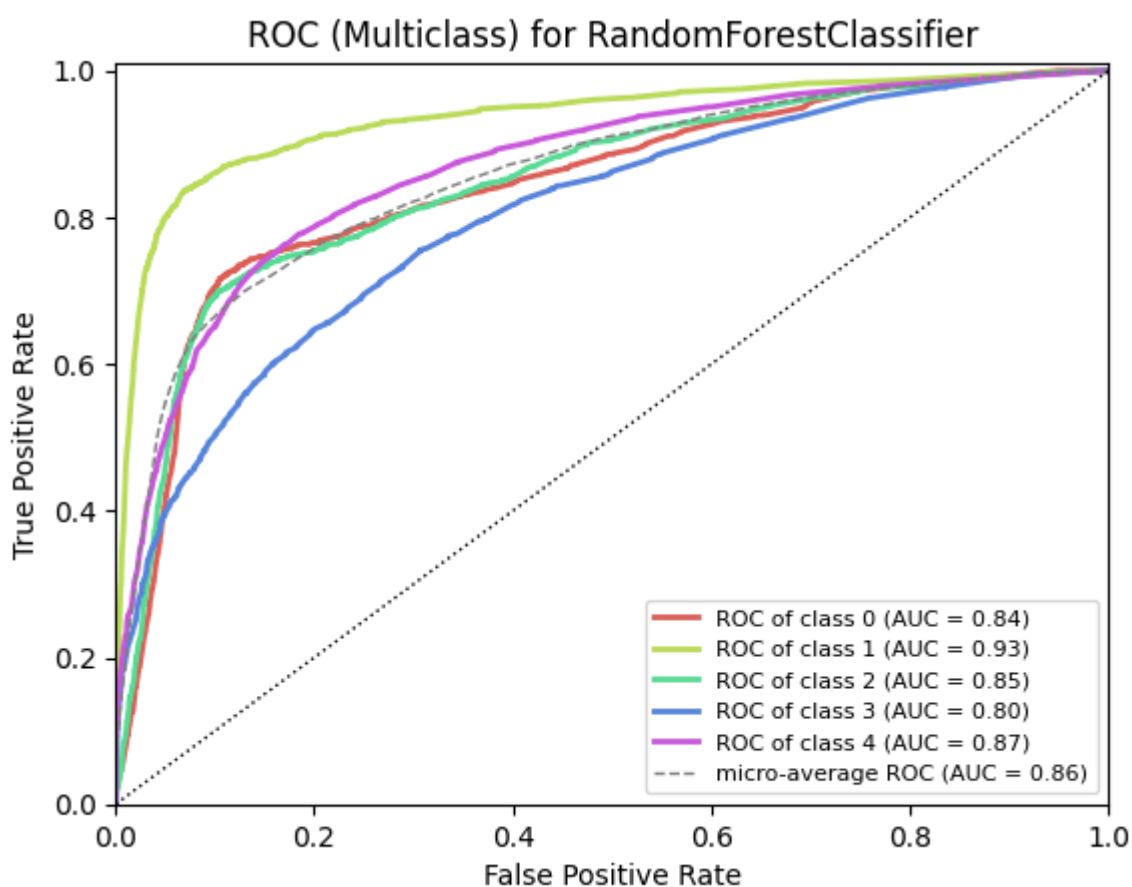
			Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third- party Damages
Recall:	0.643800	- Sensitivity:	0.7026	0.8394	0.6809	0.4535	0.5427
Precision:	0.645800	- Specificity:	0.6037	0.7079	0.5920	0.6218	0.7033

```
In [43]: # Confusion matrix
ma_confusion_matrix_table(rf_MA_train, X_test, y_test)
```

Out[43]:

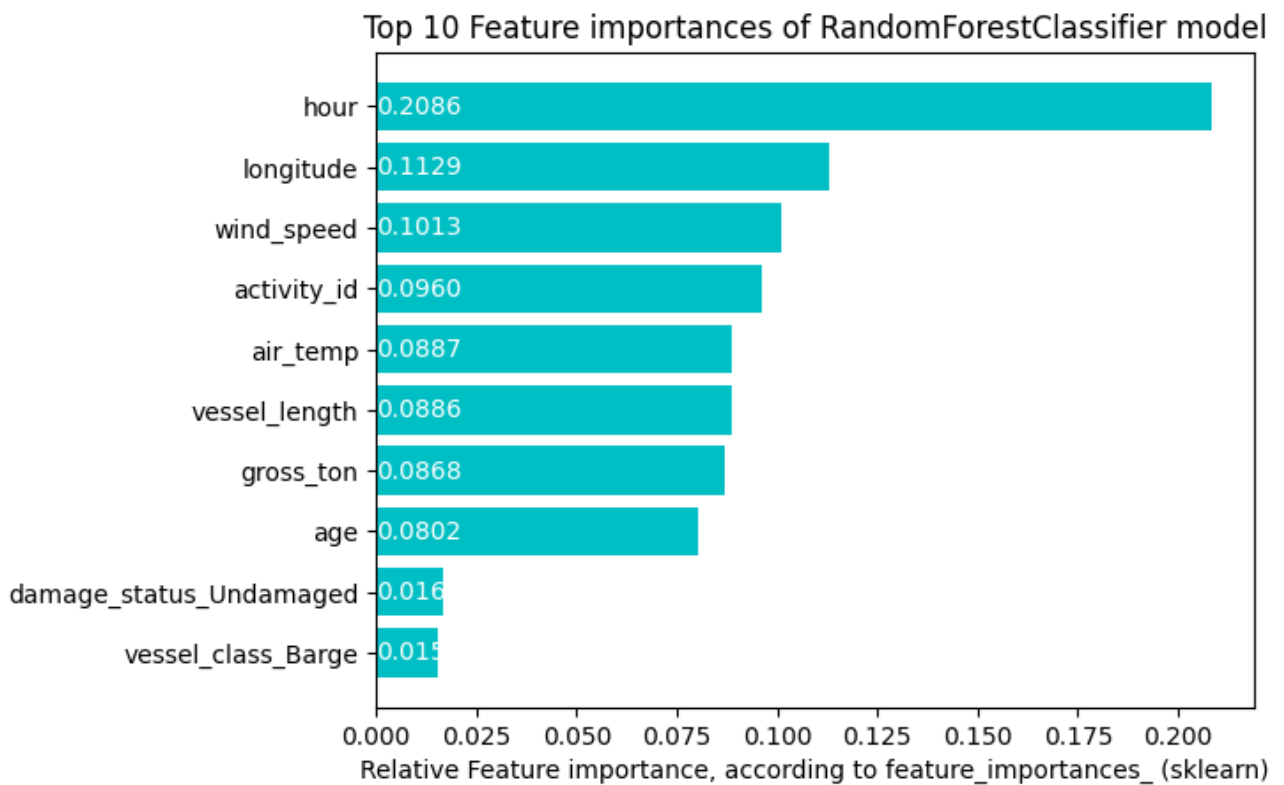
RandomForestClassifier: Confusion Matrix					
	Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events	1304	86	286	107	73
Maritime Accidents	67	1437	102	51	55
Material Issues	318	119	1248	106	42
Onboard Emergencies	261	156	304	804	248
Third-party Damages	210	232	168	225	991

In [44]: *# Plot Receiver Operating Characteristic (ROC) curves for each class*
 roc_curve_plot(rf_MA_train, X_test, y_test)



Feature importances

In [45]: *# Call previous functions for plotting top 10 importances according to feature_importance*
 plot_importances(sklearn_importances(rf_MA_train), top=10)



2.3.2. Multilayer Perceptron (MLP)

Model Training

```
In [46]: if train_model_enabled :
# Define model parameters
params = {
    'hidden_layer_sizes': (100, 50),    # Two hidden layers with 100 and 50 neurons
    'activation': 'relu',               # Activation function for the hidden layers:
    'solver': 'adam',                  # Optimization algorithm
    'random_state': seed                # Random seed for reproducibility
}

# Create and train model
nnet_MA_train = MLPClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(nnet_MA_train, models_folder + '/' + 'nnet_MA_train.pkl')
print(f'{nnet_MA_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
nnet_MA_train = joblib.load(models_folder + '/' + 'nnet_MA_train.pkl')
print(f'{nnet_MA_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(nnet_MA_train)
```

MLPClassifier model loaded from Models folder

Parameters:

```
activation: relu
alpha: 0.0001
batch_size: auto
beta_1: 0.9
beta_2: 0.999
early_stopping: False
epsilon: 1e-08
hidden_layer_sizes: (100, 50)
learning_rate: constant
learning_rate_init: 0.001
max_fun: 15000
max_iter: 200
momentum: 0.9
n_iter_no_change: 10
nesterovs_momentum: True
power_t: 0.5
random_state: 42
shuffle: True
solver: adam
tol: 0.0001
validation_fraction: 0.1
verbose: False
warm_start: False
```

Model Metrics

```
In [47]: # Model performance main metrics
ma_model_metrics(nnet_MA_train, X_test, y_test, styled=True)
```

Out[47]: **MLPClassifier Training**

metric	value
ROC AUC:	0.7754
Accuracy:	0.4518
Log loss:	1.2952
Kappa:	0.3155
MCC:	0.3187
F1:	0.4455

```
In [48]: # recall & precision, sensitivity & specificity
sens_spec(nnet_MA_train, X_test, y_test, styled=True)
```

Out[48]:

MLPClassifier Model							
			Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third- party Damages
Recall:	0.454800	-	Sensitivity:	0.2139	0.6384	0.4664	0.4467
Precision:	0.452800	-	Specificity:	0.3513	0.4558	0.3714	0.5035
							0.5817

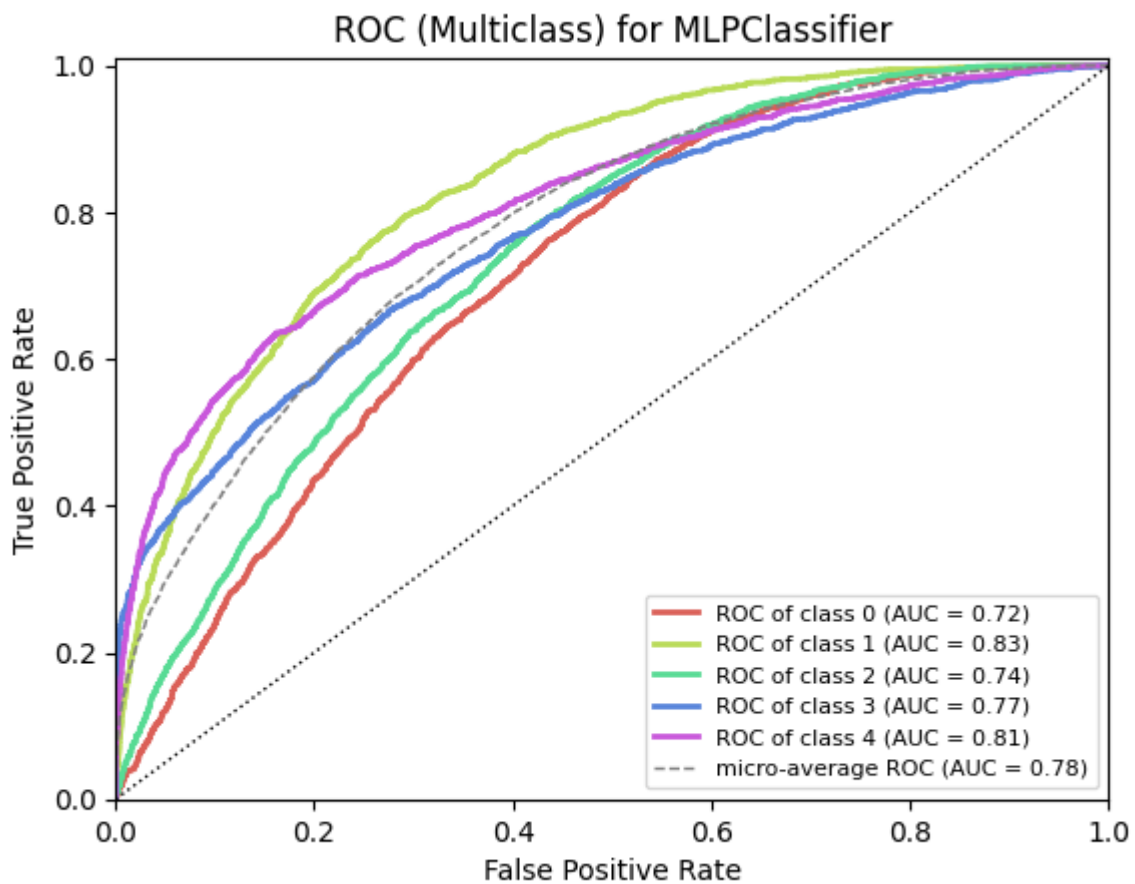
```
In [49]: # Confusion matrix
ma_confusion_matrix_table(nnet_MA_train, X_test, y_test)
```

Out[49]:

MLPClassifier: Confusion Matrix

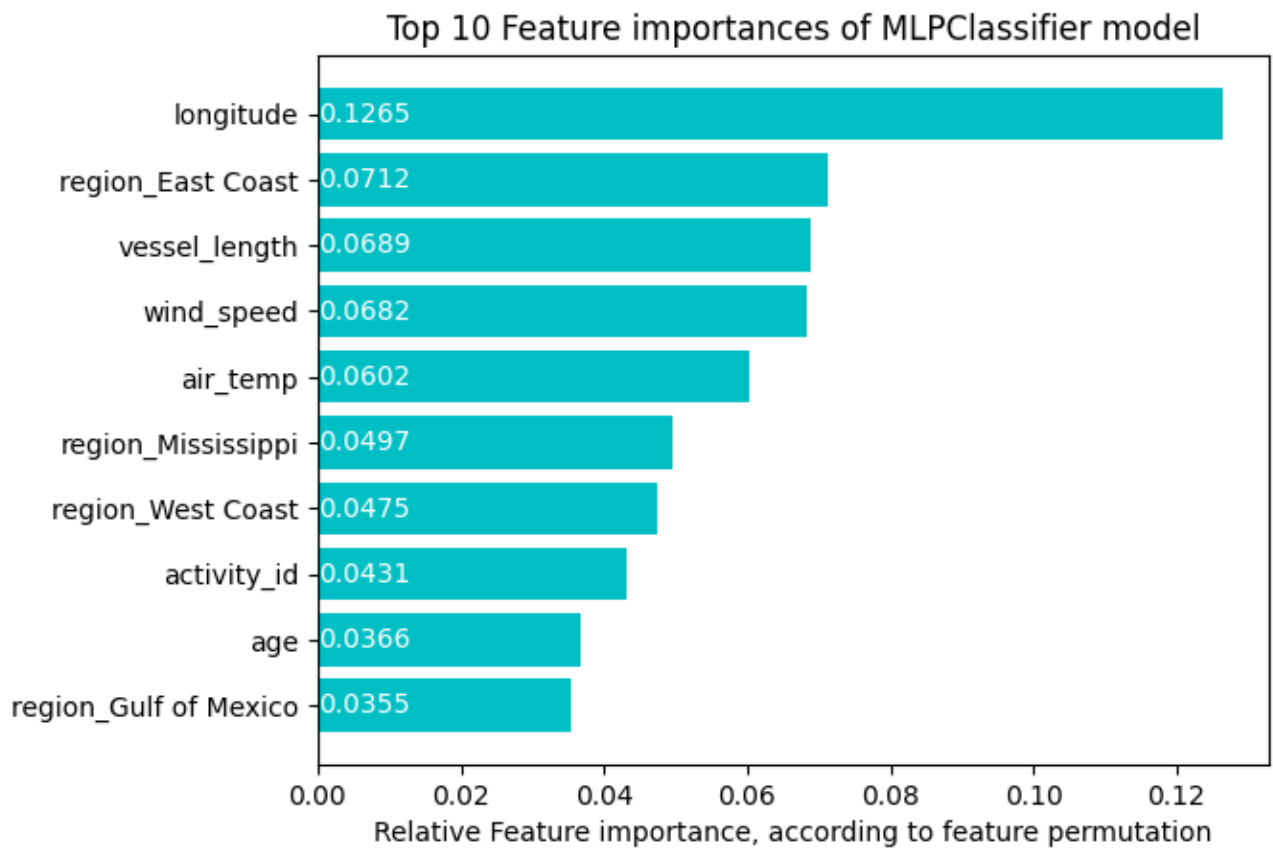
	Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events	397	410	677	214	158
Maritime Accidents	133	1093	261	107	118
Material Issues	259	376	855	200	143
Onboard Emergencies	200	223	309	792	249
Third-party Damages	141	296	200	260	929

```
In [50]: # Plot Receiver Operating Characteristic (ROC) curves for each class
roc_curve_plot(nnet_MA_train, X_test, y_test)
```



Feature importances

```
In [51]: # Call previous functions for plotting top 10 importances according to feature permutation
plot_importances(permutation_importances(nnet_MA_train, X_train, y_train), top=10)
```



2.3.3. Classification and Regression Trees (CART)

Model Training

```
In [52]: if train_model_enabled :
# Define model parameters
params = {
    'criterion': 'gini',                # Criteria
    'random_state': seed                # Random seed for reproducibility
}

# Create and train model
cart_MA_train = DecisionTreeClassifier(**params).fit(X_train, y_train)

# Save to external file
joblib.dump(cart_MA_train, models_folder + '/' + 'cart_MA_train.pkl')
print(f'{cart_MA_train.__class__.__name__} model exported to {models_folder} folder')
else:
# Load model from external file
cart_MA_train = joblib.load(models_folder + '/' + 'cart_MA_train.pkl')
print(f'{cart_MA_train.__class__.__name__} model loaded from {models_folder} folder')

# Call previous function to show used parameters
show_parameters(cart_MA_train)
```

DecisionTreeClassifier model loaded from Models folder

Parameters:

```
ccp_alpha: 0.0
class_weight: None
criterion: gini
max_depth: None
max_features: None
max_leaf_nodes: None
min_impurity_decrease: 0.0
min_samples_leaf: 1
min_samples_split: 2
min_weight_fraction_leaf: 0.0
monotonic_cst: None
random_state: 42
splitter: best
```

Model Metrics

```
In [53]: # Model performance main metrics
ma_model_metrics(cart_MA_train, X_test, y_test, styled=True)
```

Out[53]: **DecisionTreeClassifier Training**

metric	value
ROC AUC:	0.7504
Accuracy:	0.5904
Log loss:	14.2234
Kappa:	0.4879
MCC:	0.4910
F1:	0.5832

```
In [54]: # recall & precision, sensitivity & specificity
sens_spec(cart_MA_train, X_test, y_test, styled=True)
```

Out[54]:

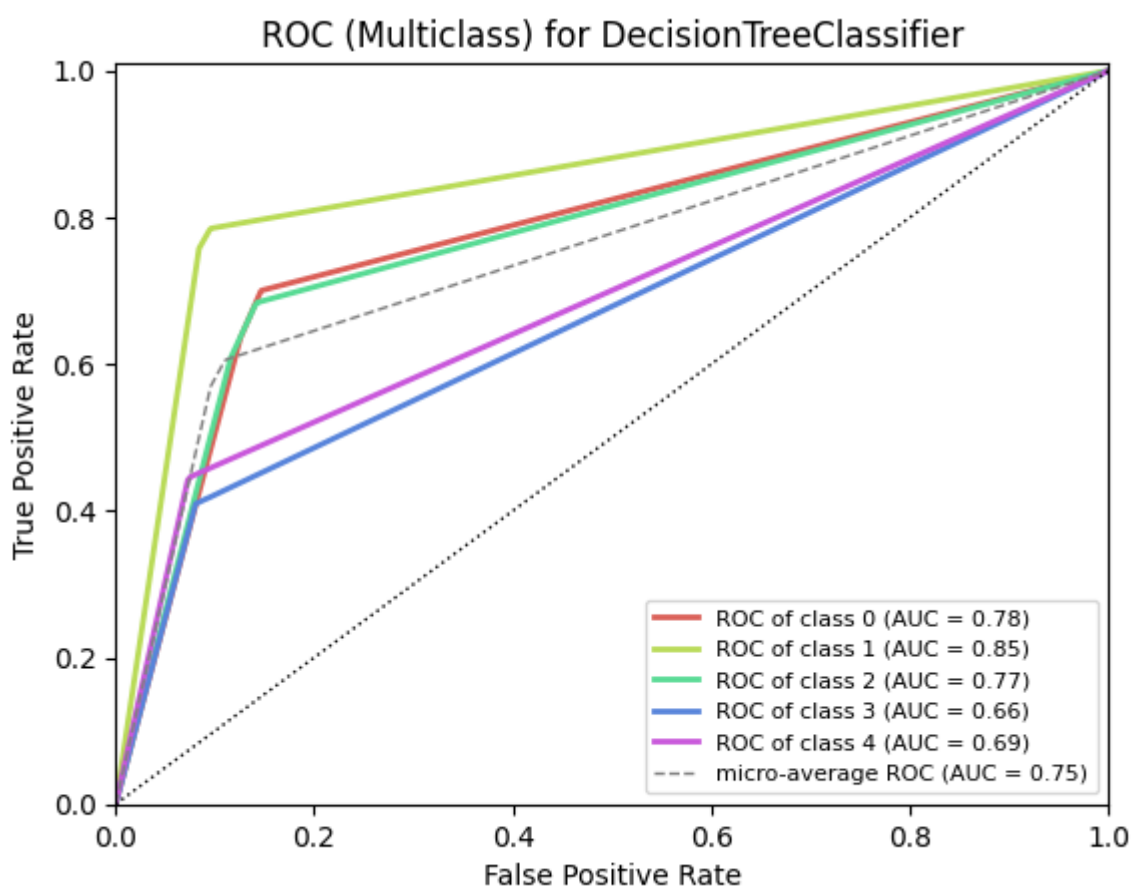
DecisionTreeClassifier Model							
		Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third- party Damages	
Recall:	0.591400	-	Sensitivity:	0.7004	0.7786	0.6258	0.4100
Precision:	0.590400	-	Specificity:	0.5532	0.6635	0.5726	0.5550
							0.4419
							0.6077

```
In [55]: # Confusion matrix
ma_confusion_matrix_table(cart_MA_train, X_test, y_test)
```

Out[55]:

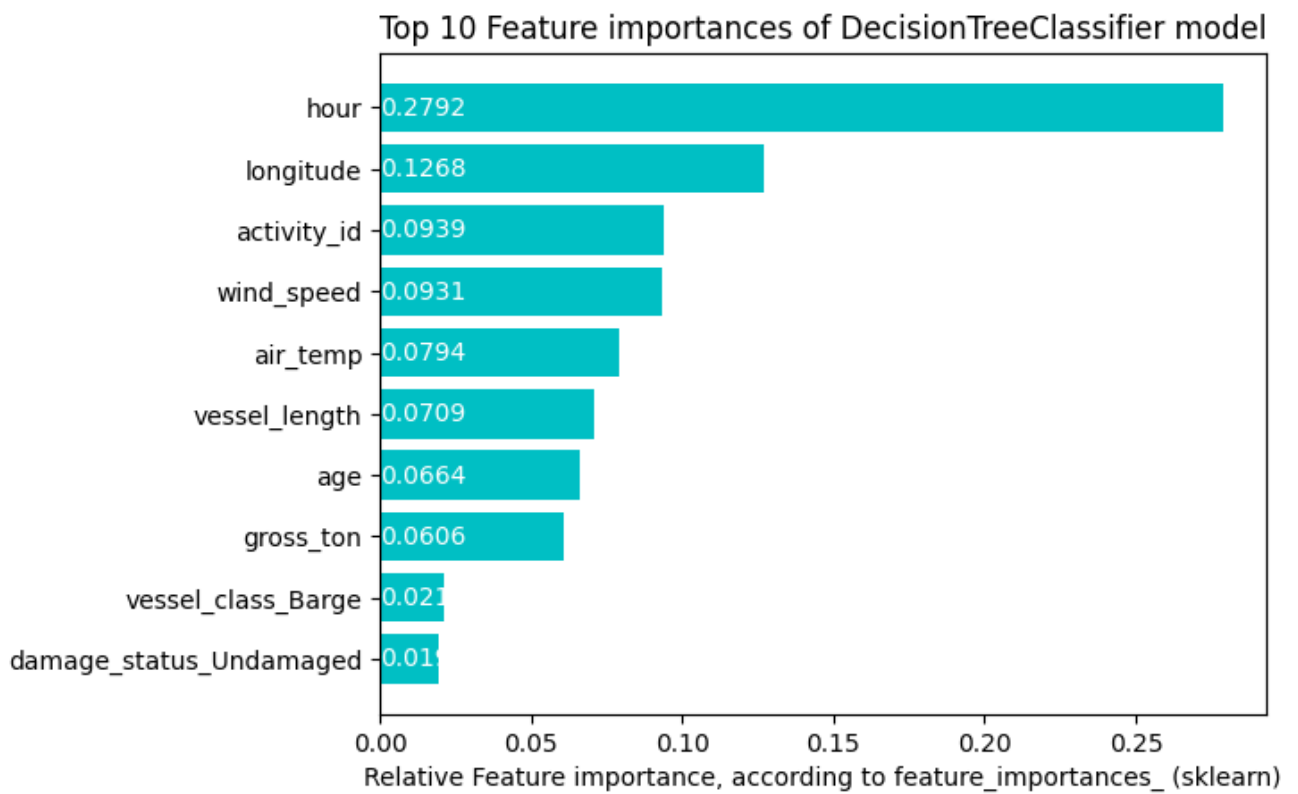
DecisionTreeClassifier: Confusion Matrix					
	Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events	1300	108	243	112	93
Maritime Accidents	127	1333	108	64	80
Material Issues	357	138	1147	112	79
Onboard Emergencies	298	185	294	727	269
Third-party Damages	268	245	211	295	807

In [56]: *# Plot Receiver Operating Characteristic (ROC) curves for each class*
 roc_curve_plot(cart_MA_train, X_test, y_test)



Feature importances

In [57]: *# Call previous functions for plotting top 10 importances according to feature_importance*
 plot_importances(sklearn_importances(cart_MA_train), top=10)



2.4. Keras API with TensorFlow

Performance functions (keras)

Loss / Accuracy train evolution

```
In [58]: # Function: Plot Loss / accuracy train evolution
def keras_train_plot(data):
    # Train process visualization
    df_train=pd.DataFrame(data)
    # df_train['epochs']=history.epoch
    df_train['epochs']=list(range(0, len(data['accuracy'])))

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6))

    fig.suptitle('Train process', fontsize=12)

    ax1.plot(df_train['epochs'], df_train['accuracy'], label='train_accuracy')
    ax1.plot(df_train['epochs'], df_train['val_accuracy'], label='val_accuracy')

    ax2.plot(df_train['epochs'], df_train['loss'], label='train_loss')
    ax2.plot(df_train['epochs'], df_train['val_loss'], label='val_loss')

    ax1.legend(loc='best')
    ax2.legend(loc='best')
    plt.show()
```

Metrics

```
In [59]: # Function: Table with main metrics data
def keras_model_metrics(model, X, y_ohe, styled=False):
    # Predictions
    predict_prob = pd.DataFrame(model.predict(X))
    predict = predict_prob.idxmax(axis=1)
```

```

# Target variable
y = pd.Series(label_encoder.fit_transform(y_ohe.idxmax(axis=1)))

# Calculate main metrics
roc_auc = round(roc_auc_score(y_ohe, predict_prob, multi_class='ovr'), 4)
accuracy = round(accuracy_score(y, predict), 4)
loss = round(log_loss(y_ohe, predict_prob), 4)
kappa = round(cohen_kappa_score(y, predict), 4)
mcc = round(matthews_corrcoef(y, predict), 4)
f1 = round(f1_score(y, predict, average='macro'), 4)

# Build multiindex table
df = pd.DataFrame([['ROC AUC:', roc_auc], ['Accuracy:', accuracy], ['Log loss:', loss],
                  ['Kappa:', kappa], ['MCC:', mcc], ['F1:', f1]],
                  columns=('metric', 'value'))

if styled:
    title = f'{model.__class__.__name__} Training'
    df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])
    return df.style.format(precision=4).hide()
else:
    return df

```

Recall / Precision

```
In [60]: # Function: Table for recall & precision, sensitivity & specificity
def keras_sens_spec(model, X, y, styled=False):
    # Predictions (absolute)
    y_pred_ohe = pd.DataFrame(model.predict(X))
    y_pred_series = y_pred_ohe.idxmax(axis=1)
    y_series = pd.Series(label_encoder.fit_transform(y.idxmax(axis=1)))

    # Recall & Precision values
    recall = round(recall_score(y_series, y_pred_series, average='macro'), 4)
    precision = round(precision_score(y_series, y_pred_series, average='macro'), 4)

    # Confusion matrix
    conf_matrix = confusion_matrix(y_series, y_pred_series)

    # List compression for sens & spec values calculation
    sensitivity, specificity = zip(*[(round(recall_score(y_series, y_pred_series, labels=[
        round(conf_matrix[i, i] / sum(conf_matrix[:, i]), 4)
        for i in range(len(conf_matrix))]))])

    # Labels and indexes
    column_labels = y.columns
    index_1 = ['Recall:', 'Precision:']
    index_2 = [recall, precision]
    index_ = [' - ', ' - ']
    index_3 = ['Sensitivity:', 'Specificity:']

    # Build multiindex table
    df = pd.DataFrame([sensitivity, specificity])
    df.columns = column_labels
    df.index = [index_1, index_2, index_, index_3]

    # Dataframe style
    if styled:
        title = f'{model.__class__.__name__} Model'
        df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])
        return df.style.format(precision=4).set_table_styles([{'selector': 'th.col_headi
    else:
        return df
```

Confusion Matrix

```
In [61]: # Function: Table with Confusion Matrix
def keras_confusion_matrix_table(model, X, y):
    # Predictions (max)
    y_pred_max = np.argmax(model.predict(X), axis=1)
    y_max = np.argmax(y, axis=1)

    # Build table
    df = pd.DataFrame(confusion_matrix(y_max, y_pred_max),
                      columns=pd.MultiIndex.from_product([f'{model.name}: Confusi
                      index=y.columns)

    # Dataframe style
    styled_df = df.style.set_table_styles([
        {'selector': 'th.col_heading', 'props': 'text-align: center;'},
        {'selector': 'td', 'props': 'text-align: center;'},
    ], overwrite=False)

    return styled_df
```

Multiclass ROC Curves

```
In [62]: # Function: Plot Multiclass ROC Curve
def keras_roc_curve_plot(model, X, y):
    # Predictions (max)
    y_pred = pd.DataFrame(model.predict(X))
    y_pred.columns = label_encoder.inverse_transform(y_pred.columns)

    # Compute ROC curve and ROC area for each class
    fpr, tpr, roc_auc = {}, {}, {}
    for i in y.columns:
        fpr[i], tpr[i], _ = roc_curve(y[i], y_pred[i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Compute micro-average ROC curve and ROC area
    fpr["micro"], tpr["micro"], _ = roc_curve(y.values.ravel(), y_pred.values.ravel())
    roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])

    # Plot ROC curve for each class
    plt.figure()
    colors = sns.color_palette("hls", 5)
    for i, color in zip(y.columns, colors):
        plt.plot(fpr[i], tpr[i], color=color, lw=2,
                 label='ROC of class {0} (AUC = {1:0.2f})'.format(i, roc_auc[i]))

    # Plot micro-average ROC curve
    plt.plot(fpr["micro"], tpr["micro"], color='grey', lw=1, linestyle='dashed',
             label='micro-average ROC (AUC = {0:0.2f})'.format(roc_auc["micro"]))

    plt.plot([0, 1], [0, 1], color='black', lw=1, linestyle='dotted')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.01])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'ROC (Multiclass) for {model.name}')
    plt.legend(loc="lower right", fontsize="8")
    plt.show()
```

Feature importances

```
In [63]: # Particular function for feature importances in keras sequential model, according to fe
def keras_sec_permutation_importances(model, X, y, repeats=3):
    # Calculate accuracy for original data
    original_accuracy = accuracy_score(pd.Series(label_encoder.fit_transform(y.idxmax(ax
                                                pd.DataFrame(model.predict(X)).idxmax(axis=1))

    importances = np.zeros(len(X.columns))

    for _ in range(repeats):
        for i, col in enumerate(X.columns):
            # Permute only this column
            X_permuted = X.copy()
            X_permuted[col] = np.random.permutation(X_permuted[col])

            # Calculate accuracy for data with this column permuted
            permuted_accuracy = accuracy_score(pd.Series(label_encoder.fit_transform(y.i
                                                pd.DataFrame(model.predict(X_permuted)).i

            # Accumulate the difference in accuracy
            importances[i] += (original_accuracy - permuted_accuracy)

    # Calculate the mean importance values
    importances /= repeats
```

```

# Normalize importance values
total_importance = np.sum(importances)
if total_importance != 0:
    importances /= total_importance

# Build a DataFrame for the results
importance_df = pd.DataFrame({
    'variable_name': X.columns,
    'importance_value': importances
}).sort_values(by='importance_value', ascending=False).reset_index(drop=True)

# Add custom attributes for model name and calculation method
importance_df.model_name = model.__class__.__name__
importance_df.importance_method = 'feature permutation performance'

return importance_df

```

```

In [64]: # Particular function for feature importances in keras functional model, according to feat
def keras_func_permutation_importances(model, X, y, repeats=3):
    # Calculate accuracy for original data
    original_accuracy = accuracy_score(pd.Series(label_encoder.fit_transform(y.idxmax(ax
                                                pd.DataFrame(model.predict(list(X.values())))).idx

    importances = np.zeros(len(X.keys()))

    for _ in range(repeats):
        for i, key in enumerate(X.keys()):
            # Permute only this column
            X_permuted = X.copy()
            X_permuted[key] = X_permuted[key].sample(frac=1).reset_index(drop=True)

            # Calculate accuracy for data with this column permuted
            permuted_accuracy = accuracy_score(pd.Series(label_encoder.fit_transform(y.i
                                                pd.DataFrame(model.predict(list(X_permute

            # Accumulate the difference in accuracy
            importances[i] += (original_accuracy - permuted_accuracy)

    # Calculate the mean importance values
    importances /= repeats

    # Normalize importance values
    total_importance = np.sum(importances)
    if total_importance != 0:
        importances /= total_importance

    # Build a DataFrame for the results
    importance_df = pd.DataFrame({
        'variable_name': X.keys(),
        'importance_value': importances
    }).sort_values(by='importance_value', ascending=False).reset_index(drop=True)

    # Add custom attributes for model name and calculation method
    importance_df.model_name = model.__class__.__name__
    importance_df.importance_method = 'feature permutation performance'

    return importance_df

```

Train / Test Split (OHE)

```

In [65]: # One-hot-encoding for target variable, train sample
y_train_ohe = pd.get_dummies(y_train).astype(int)

```

```
y_train_ohe.columns = label_encoder.inverse_transform(y_train_ohe.columns)
```

```
# One-hot-encoding for target variable, test sample
```

```
y_test_ohe = pd.get_dummies(y_test).astype(int)
```

```
y_test_ohe.columns = label_encoder.inverse_transform(y_test_ohe.columns)
```

2.4.1. Sequential: Densely-connected Model (Simple)

Model Design

```
In [66]: # Define the neural network model
keras_model_1 = Sequential()
keras_model_1.add(Input(shape=(X_test.shape[1],)))      # Input layer: shape = Feature d
keras_model_1.add(Dense(128, activation='relu'))        # Dense layer, relu activation
keras_model_1.add(Dropout(0.5))                        # Dropout
keras_model_1.add(Dense(64, activation='relu'))
keras_model_1.add(Dropout(0.5))
keras_model_1.add(Dense(y_test_ohe.shape[1], activation='softmax'))

# Check structure
keras_model_1.summary()
plot_model(keras_model_1, show_shapes=True, show_layer_names=True, dpi=70)
```

Model: "sequential"

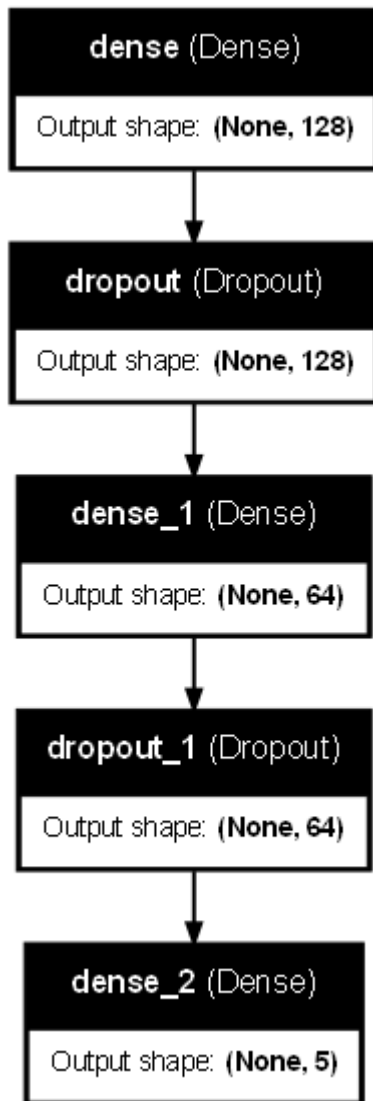
Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	4,224
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 5)	325

Total params: 12,805 (50.02 KB)

Trainable params: 12,805 (50.02 KB)

Non-trainable params: 0 (0.00 B)

Out[66]:



Model Training

```

In [67]: if train_model_enabled :
    # Compile model
    keras_model_1.compile(optimizer='adam',
                           loss='categorical_crossentropy',
                           metrics=['accuracy'])

    # Train model
    history = keras_model_1.fit(X_train, y_train_ohe,
                                epochs=80,
                                batch_size = 32,
                                callbacks = EarlyStopping(monitor = 'val_loss', patience = 10, restore_best_
                                validation_data=(X_test, y_test_ohe))

    # Save model and history to external files
    keras_model_1.save(models_folder + '/' + 'keras_model_1.keras')
    joblib.dump(history.history, models_folder + '/' + 'keras_model_1_evolution.pkl')
    print(f'{keras_model_1.name} model and train history exported to {models_folder} fol
else:
    # Load model and history from external files
    keras_model_1 = load_model(models_folder + '/' + 'keras_model_1.keras')
    class EmptyClass:
        def __init__(self, history):
            self.history = history
    history = EmptyClass('')
    history.history = joblib.load(models_folder + '/' + 'keras_model_1_evolution.pkl')
    print(f'{keras_model_1.name} model and train history loaded from {models_folder} fol

```

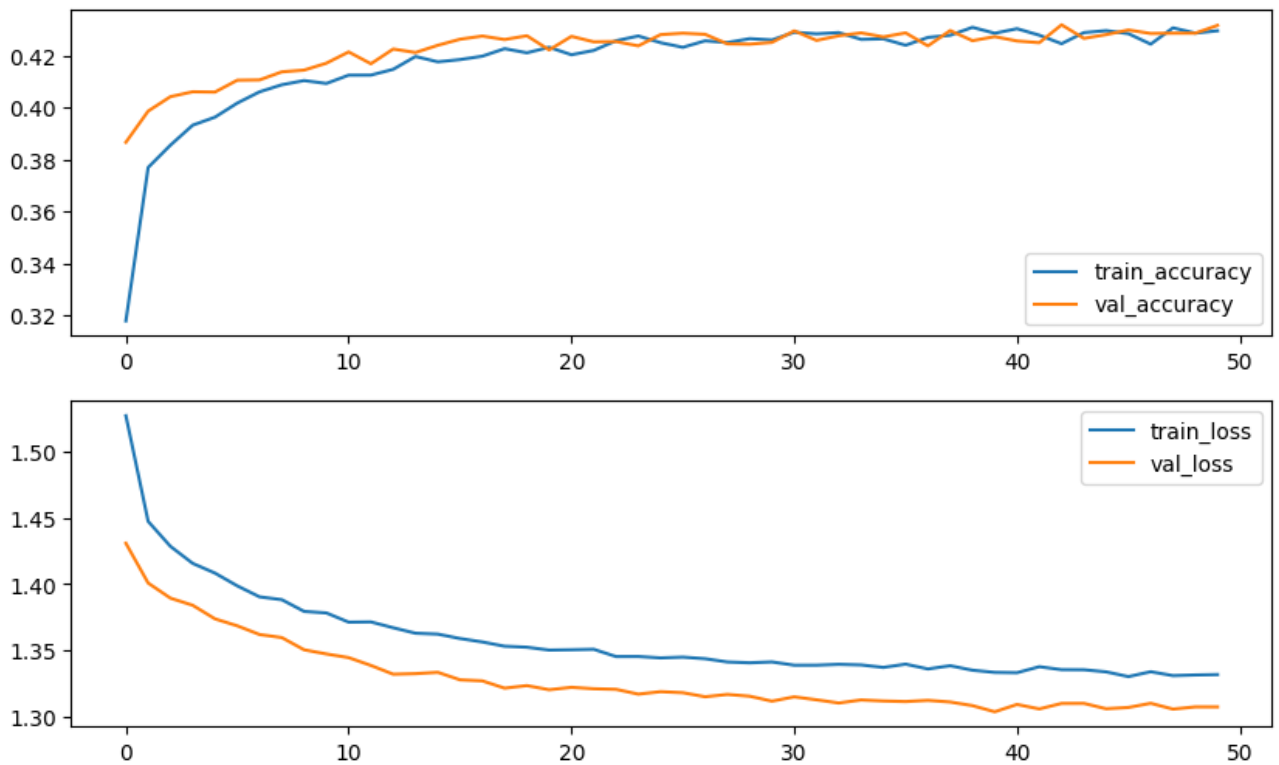
sequential_2 model and train history loaded from Models folder

Model Metrics

```
In [68]: # Accuracy / Loss Train evolution
keras_train_plot(history.history)

# Accuracy / Loss values
accuracy, loss = keras_model_1.evaluate(X_test, y_test_ohe)
print("accuracy = {:.4f}, loss = {:.4f}".format(accuracy, loss))
```

Train process



282/282 ————— 1s 416us/step - accuracy: 0.4246 - loss: 1.3249
accuracy = 1.3044, loss = 0.4326

```
In [69]: # Model performance main metrics
keras_model_metrics(keras_model_1, X_test, y_test_ohe, styled=True)
```

282/282 ————— 0s 461us/step

Out[69]: **Sequential Training**

metric	value
ROC AUC:	0.7566
Accuracy:	0.4326
Log loss:	1.3044
Kappa:	0.2909
MCC:	0.2953
F1:	0.4295

```
In [70]: # recall & precision, sensitivity & specificity
keras_sens_spec(keras_model_1, X_test, y_test_ohe, styled=True)
```

282/282 ————— 0s 426us/step

Out[70]:

				Sequential Model				
				Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Recall:	0.434300	-	Sensitivity:	0.2284	0.5549	0.5210	0.3604	0.5066
Precision:	0.450300	-	Specificity:	0.3318	0.4306	0.3463	0.5917	0.5513

```
In [71]: # Confusion matrix
keras_confusion_matrix_table(keras_model_1, X_test, y_test_oh)
```

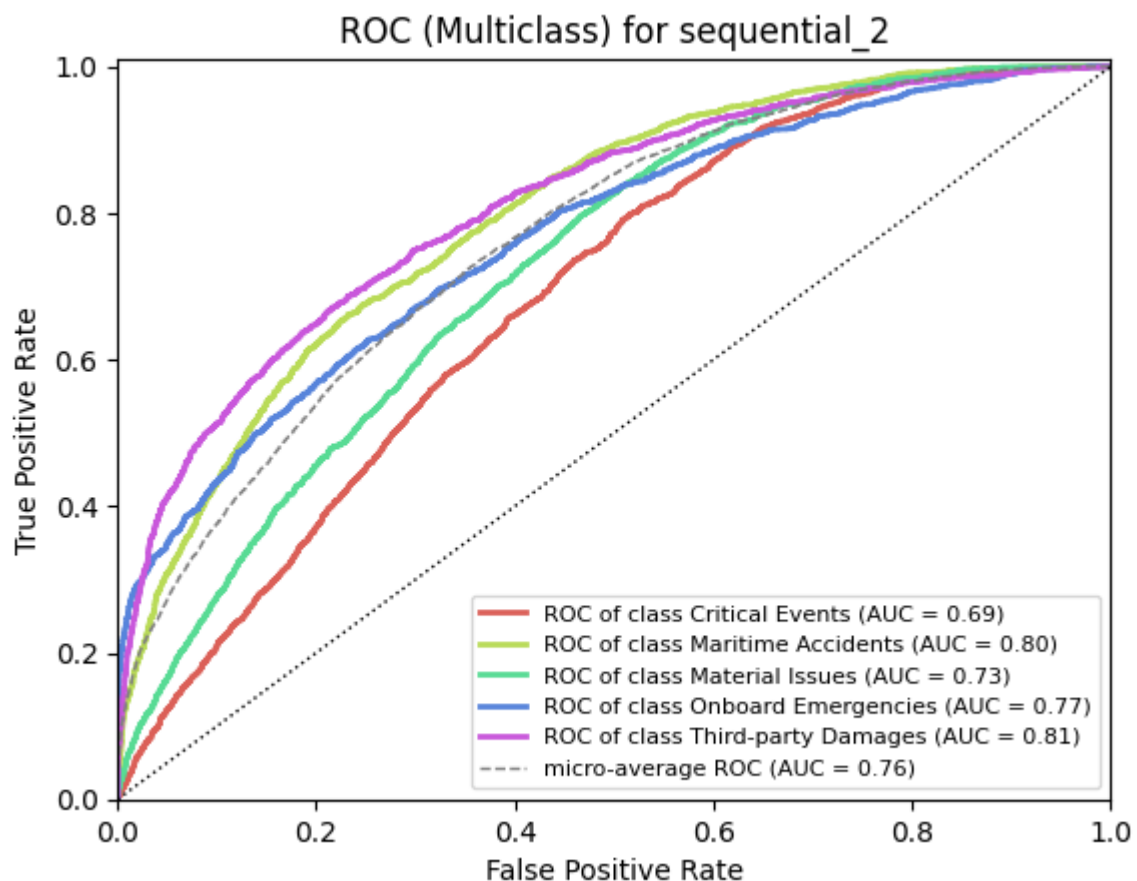
282/282 ————— 0s 426us/step

Out[71]:

		sequential_2: Confusion Matrix				
		Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events		424	371	757	115	189
Maritime Accidents		124	950	366	71	201
Material Issues		234	386	955	91	167
Onboard Emergencies		286	240	412	639	196
Third-party Damages		210	259	268	164	925

```
In [72]: # Plot Roc curves
keras_roc_curve_plot(keras_model_1, X_test, y_test_oh)
```

282/282 ————— 0s 439us/step



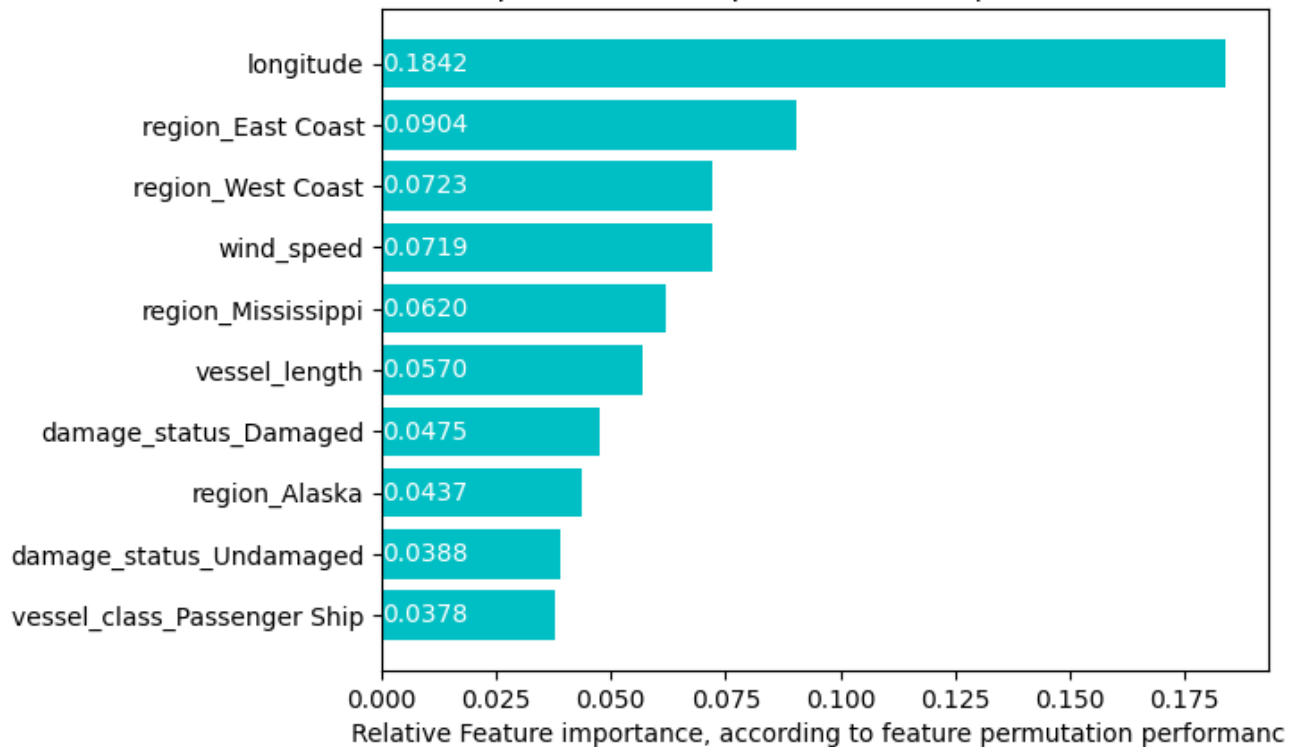
Feature importances

```
In [73]: # Call previous functions for plotting top 10 importances according to feature permutation  
plot_importances(keras_sec_permutation_importances(keras_model_1, X_train, y_train_ohe),
```

1125/1125	0s	413us/step
1125/1125	0s	416us/step
1125/1125	0s	423us/step
1125/1125	0s	414us/step
1125/1125	0s	421us/step
1125/1125	0s	413us/step
1125/1125	0s	409us/step
1125/1125	0s	422us/step
1125/1125	0s	425us/step
1125/1125	0s	414us/step
1125/1125	0s	426us/step
1125/1125	0s	425us/step
1125/1125	0s	420us/step
1125/1125	0s	423us/step
1125/1125	0s	406us/step
1125/1125	0s	420us/step
1125/1125	0s	427us/step
1125/1125	0s	409us/step
1125/1125	0s	420us/step
1125/1125	0s	422us/step
1125/1125	0s	410us/step
1125/1125	0s	425us/step
1125/1125	0s	423us/step
1125/1125	0s	409us/step
1125/1125	0s	427us/step
1125/1125	0s	422us/step
1125/1125	0s	420us/step
1125/1125	0s	429us/step
1125/1125	0s	414us/step
1125/1125	0s	427us/step
1125/1125	0s	420us/step
1125/1125	0s	416us/step
1125/1125	0s	421us/step
1125/1125	0s	431us/step
1125/1125	0s	420us/step
1125/1125	0s	424us/step
1125/1125	0s	432us/step
1125/1125	0s	423us/step
1125/1125	0s	433us/step
1125/1125	0s	430us/step
1125/1125	0s	416us/step
1125/1125	0s	437us/step
1125/1125	0s	430us/step
1125/1125	0s	422us/step
1125/1125	0s	427us/step
1125/1125	0s	430us/step
1125/1125	0s	416us/step
1125/1125	0s	428us/step
1125/1125	0s	423us/step
1125/1125	0s	421us/step
1125/1125	0s	434us/step
1125/1125	0s	416us/step
1125/1125	0s	424us/step
1125/1125	0s	430us/step
1125/1125	0s	424us/step
1125/1125	0s	425us/step
1125/1125	0s	430us/step
1125/1125	0s	417us/step
1125/1125	0s	431us/step
1125/1125	0s	428us/step
1125/1125	0s	420us/step
1125/1125	0s	428us/step
1125/1125	0s	424us/step

1125/1125	0s	423us/step
1125/1125	0s	425us/step
1125/1125	0s	415us/step
1125/1125	0s	419us/step
1125/1125	0s	427us/step
1125/1125	0s	415us/step
1125/1125	0s	427us/step
1125/1125	0s	433us/step
1125/1125	0s	415us/step
1125/1125	0s	432us/step
1125/1125	0s	430us/step
1125/1125	0s	416us/step
1125/1125	0s	427us/step
1125/1125	0s	420us/step
1125/1125	0s	409us/step
1125/1125	0s	420us/step
1125/1125	0s	427us/step
1125/1125	0s	421us/step
1125/1125	0s	423us/step
1125/1125	0s	412us/step
1125/1125	0s	427us/step
1125/1125	0s	423us/step
1125/1125	0s	416us/step
1125/1125	0s	424us/step
1125/1125	0s	423us/step
1125/1125	0s	420us/step
1125/1125	0s	427us/step
1125/1125	0s	420us/step
1125/1125	0s	424us/step
1125/1125	0s	425us/step
1125/1125	0s	420us/step
1125/1125	0s	427us/step
1125/1125	0s	427us/step
1125/1125	0s	425us/step

Top 10 Feature importances of Sequential model



2.4.2. Functional API Model (Multi-input)

This model will be trained (and tested) with a dict of insulated dataframes. Each of these

dataframes corresponds to a variable of the original data.

Data split per variable group

```
In [74]: def dataframe_var_split (df):
df = df.copy()
# Rename num variables with '_' adding extra '_' for keeping original name when all
df.rename(columns={'activity_id': 'activity_id_',
                  'air_temp': 'air_temp_',
                  'gross_ton': 'gross_ton_',
                  'vessel_length': 'vessel_length_',
                  'wind_speed': 'wind_speed_'}, inplace=True)
# Dispense last '_' part to group them easily
df = df.rename(columns=lambda x: x.rsplit('_', 1)[0])

# Iteration over each group of same variable names and create a DataFrame for each one
df_var_splited = {}
for name, group in df.groupby(df.columns, axis=1):
    df_var_splited[name] = group

return df_var_splited

# Apply to X_train
X_train_var_splited = dataframe_var_split(X_train)
# Verify splited variable names and Length
print(f'X_train_var_splited: {X_train_var_splited.keys()}')
print(f'Length: {len(X_train_var_splited[list(X_train_var_splited.keys())[0]])}')

# Apply to X_test
X_test_var_splited = dataframe_var_split(X_test)
# Verify splited variable name sand Length
print(f'X_test_var_splited: {X_test_var_splited.keys()}')
print(f'Length: {len(X_test_var_splited[list(X_test_var_splited.keys())[0]])}')
```

```
X_train_var_splited: dict_keys(['activity_id', 'age', 'air_temp', 'damage_status', 'gross_
ton', 'hour', 'longitude', 'region', 'vessel_class', 'vessel_length', 'watertype', 'wind
_speed'])
```

```
Length: 36000
```

```
X_test_var_splited: dict_keys(['activity_id', 'age', 'air_temp', 'damage_status', 'gross_
ton', 'hour', 'longitude', 'region', 'vessel_class', 'vessel_length', 'watertype', 'wind_
speed'])
```

```
Length: 9000
```

Model Design

```
In [75]: def functional_model():
# Numerical Input Layers: Dense + dropout Layers
input_act = Input(shape=(X_train_var_splited['activity_id'].shape[1],))
dense_act = Dense(128, activation='relu', name='activity_id')(input_act)
dropout_act = Dropout(0.1)(dense_act)

input_age = Input(shape=(X_train_var_splited['age'].shape[1],))
dense_age = Dense(128, activation='relu', name='age')(input_age)
dropout_age = Dropout(0.1)(dense_age)

input_air = Input(shape=(X_train_var_splited['air_temp'].shape[1],))
dense_air = Dense(128, activation='relu', name='air_temp')(input_air)
dropout_air = Dropout(0.1)(dense_air)

input_gro = Input(shape=(X_train_var_splited['gross_ton'].shape[1],))
dense_gro = Dense(128, activation='relu', name='gross_ton')(input_gro)
```

```

dropout_gro = Dropout(0.1)(dense_gro)

input_hou = Input(shape=(X_train_var_splited['hour'].shape[1],))
dense_hou = Dense(128, activation='relu', name='hour')(input_hou)
dropout_hou = Dropout(0.1)(dense_hou)

input_lon = Input(shape=(X_train_var_splited['longitude'].shape[1],))
dense_lon = Dense(128, activation='relu', name='longitude')(input_lon)
dropout_lon = Dropout(0.1)(dense_lon)

input_len = Input(shape=(X_train_var_splited['vessel_length'].shape[1],))
dense_len = Dense(128, activation='relu', name='vessel_length')(input_len)
dropout_len = Dropout(0.1)(dense_len)

input_win = Input(shape=(X_train_var_splited['wind_speed'].shape[1],))
dense_win = Dense(128, activation='relu', name='wind_speed')(input_win)
dropout_win = Dropout(0.1)(dense_win)

# Categorical Input Layers (one-hot): Dense Layers
input_dam = Input(shape=(X_train_var_splited['damage_status'].shape[1],))
dense_dam = Dense(128, activation='linear', name='damage_status')(input_dam)

input_reg = Input(shape=(X_train_var_splited['region'].shape[1],))
dense_reg = Dense(128, activation='linear', name='region')(input_reg)

input_cla = Input(shape=(X_train_var_splited['vessel_class'].shape[1],))
dense_cla = Dense(128, activation='linear', name='vessel_class')(input_cla)

input_wat = Input(shape=(X_train_var_splited['watertype'].shape[1],))
dense_wat = Dense(128, activation='linear', name='watertype')(input_wat)

# First Level joins
input_gro_len = concatenate([dropout_gro, dropout_len])
dense_gro_len = Dense(64, activation='relu')(input_gro_len)

input_lon_reg = concatenate([dropout_lon, dense_reg])
dense_lon_reg = Dense(64, activation='relu')(input_lon_reg)

input_air_win = concatenate([dropout_air, dropout_win])
dense_air_win = Dense(64, activation='relu')(input_air_win)

# Second Level joins
input_wat_lon_reg = concatenate([dense_lon_reg, dense_wat])
dense_wat_lon_reg = Dense(64, activation='relu')(input_wat_lon_reg)

# Third Level joins
input_gro_len_age_cla = concatenate([dense_gro_len, dropout_age, dense_cla])
dense_gro_len_age_cla = Dense(64, activation='relu')(input_gro_len_age_cla)

# Fourth Level joins
input_wat_lon_reg_air_win = concatenate([dense_wat_lon_reg, dense_air_win])
dense_wat_lon_reg_air_win = Dense(64, activation='relu')(input_wat_lon_reg_air_win)

# Fifth Level joins
input_gather = concatenate([dropout_act, dense_dam, dense_gro_len_age_cla, dropout_h
dense_gather = Dense(64, activation='relu')(input_gather)

# Output Layer
output = Dense(y_train_ohe.shape[1], activation='softmax')(dense_gather)

```

```
# Model definition. The input order will be followed for data feeding in the training
model = Model(inputs=[input_act, input_age, input_air, input_dam,
                      input_gro, input_hou, input_lon, input_reg,
                      input_cla, input_len, input_wat, input_win], outputs=output)

return model

keras_model_2 = functional_model()

# Check structure
keras_model_2.summary()
plot_model(keras_model_2, show_shapes=True, show_layer_names=True, dpi=80)
```

Model: "functional_11"

Layer (type)	Output Shape	Param #	Connected to
input_layer_6 (InputLayer)	(None , 1)	0	-
longitude (Dense)	(None , 128)	256	input_layer_6[0]...
input_layer_10 (InputLayer)	(None , 6)	0	-
input_layer_4 (InputLayer)	(None , 1)	0	-
input_layer_7 (InputLayer)	(None , 1)	0	-
dropout_7 (Dropout)	(None , 128)	0	longitude[0][0]
region (Dense)	(None , 128)	896	input_layer_10[0]...
input_layer_3 (InputLayer)	(None , 1)	0	-
input_layer_8 (InputLayer)	(None , 1)	0	-
gross_ton (Dense)	(None , 128)	256	input_layer_4[0]...
vessel_length (Dense)	(None , 128)	256	input_layer_7[0]...
concatenate_1 (Concatenate)	(None , 256)	0	dropout_7[0][0], region[0][0]
input_layer_12 (InputLayer)	(None , 2)	0	-
air_temp (Dense)	(None , 128)	256	input_layer_3[0]...
wind_speed (Dense)	(None , 128)	256	input_layer_8[0]...
dropout_5 (Dropout)	(None , 128)	0	gross_ton[0][0]
dropout_8 (Dropout)	(None , 128)	0	vessel_length[0]...
input_layer_2 (InputLayer)	(None , 1)	0	-
dense_4 (Dense)	(None , 64)	16,448	concatenate_1[0]...
watertype (Dense)	(None , 128)	384	input_layer_12[0]...
dropout_4 (Dropout)	(None , 128)	0	air_temp[0][0]
dropout_9 (Dropout)	(None , 128)	0	wind_speed[0][0]
concatenate (Concatenate)	(None , 256)	0	dropout_5[0][0], dropout_8[0][0]
age (Dense)	(None , 128)	256	input_layer_2[0]...
input_layer_11 (InputLayer)	(None , 11)	0	-
concatenate_3 (Concatenate)	(None , 192)	0	dense_4[0][0], watertype[0][0]

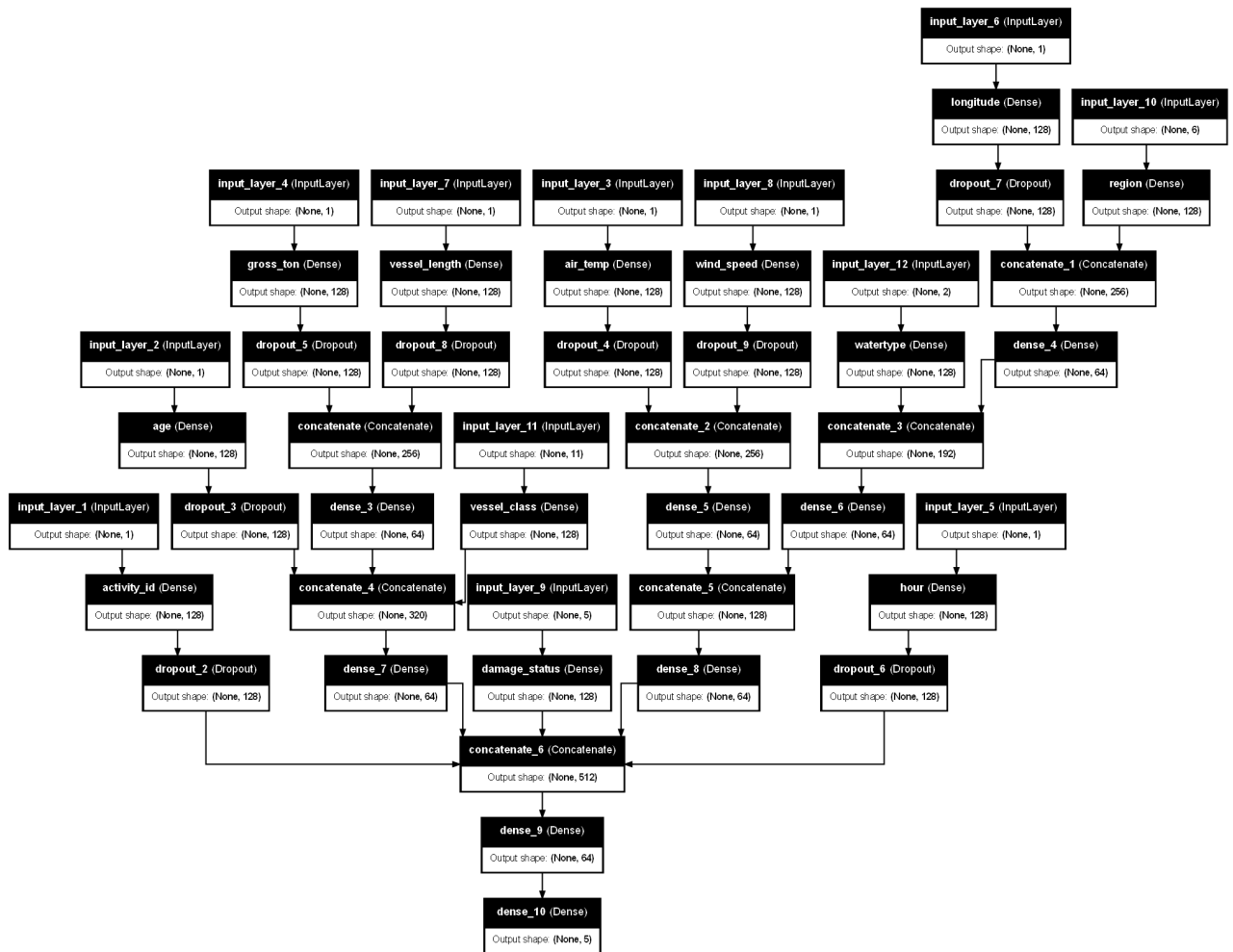
concatenate_2 (Concatenate)	(None, 256)	0	dropout_4[0][0], dropout_9[0][0]
input_layer_1 (InputLayer)	(None, 1)	0	-
dense_3 (Dense)	(None, 64)	16,448	concatenate[0][0]
dropout_3 (Dropout)	(None, 128)	0	age[0][0]
vessel_class (Dense)	(None, 128)	1,536	input_layer_11[0...
input_layer_5 (InputLayer)	(None, 1)	0	-
dense_6 (Dense)	(None, 64)	12,352	concatenate_3[0]...
dense_5 (Dense)	(None, 64)	16,448	concatenate_2[0]...
activity_id (Dense)	(None, 128)	256	input_layer_1[0]...
input_layer_9 (InputLayer)	(None, 5)	0	-
concatenate_4 (Concatenate)	(None, 320)	0	dense_3[0][0], dropout_3[0][0], vessel_class[0][...
hour (Dense)	(None, 128)	256	input_layer_5[0]...
concatenate_5 (Concatenate)	(None, 128)	0	dense_6[0][0], dense_5[0][0]
dropout_2 (Dropout)	(None, 128)	0	activity_id[0][0]
damage_status (Dense)	(None, 128)	768	input_layer_9[0]...
dense_7 (Dense)	(None, 64)	20,544	concatenate_4[0]...
dropout_6 (Dropout)	(None, 128)	0	hour[0][0]
dense_8 (Dense)	(None, 64)	8,256	concatenate_5[0]...
concatenate_6 (Concatenate)	(None, 512)	0	dropout_2[0][0], damage_status[0]... dense_7[0][0], dropout_6[0][0], dense_8[0][0]
dense_9 (Dense)	(None, 64)	32,832	concatenate_6[0]...
dense_10 (Dense)	(None, 5)	325	dense_9[0][0]

Total params: 129,285 (505.02 KB)

Trainable params: 129,285 (505.02 KB)

Non-trainable params: 0 (0.00 B)

Out[75]:



Model Training

```
In [76]: if train_model_enabled :
# Compile model
keras_model_2.compile(optimizer='adam',
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

# Train model. Data feeding order according to inputs definition
history = keras_model_2.fit([X_train_var_splited['activity_id'], X_train_var_splited
                             X_train_var_splited['gross_ton'], X_train_var_splited['
                             X_train_var_splited['vessel_class'], X_train_var_splite

epochs=50,
batch_size = 32,
callbacks = EarlyStopping(monitor = 'val_loss', patience = 15, restore_best_
validation_data=([X_test_var_splited['activity_id'], X_test_var_splited['age
X_test_var_splited['gross_ton'], X_test_var_splited['hour'
X_test_var_splited['vessel_class'], X_test_var_splited['ve

# Save model and history to external files
keras_model_2.save(models_folder + '/' + 'keras_model_2.keras')
joblib.dump(history.history, models_folder + '/' + 'keras_model_2_evolution.pkl')
print(f'{keras_model_2.name} model and train history exported to {models_folder} fol

else:
# Load model and history from external files
keras_model_2 = load_model(models_folder + '/' + 'keras_model_2.keras')
class EmptyClass:
    def __init__(self, history):
        self.history = history
history = EmptyClass('')
history.history = joblib.load(models_folder + '/' + 'keras_model_2_evolution.pkl')
print(f'{keras_model_2.name}> model and train history loaded from {models_folder} fo
```

functional_11> model and train history loaded from Models folder

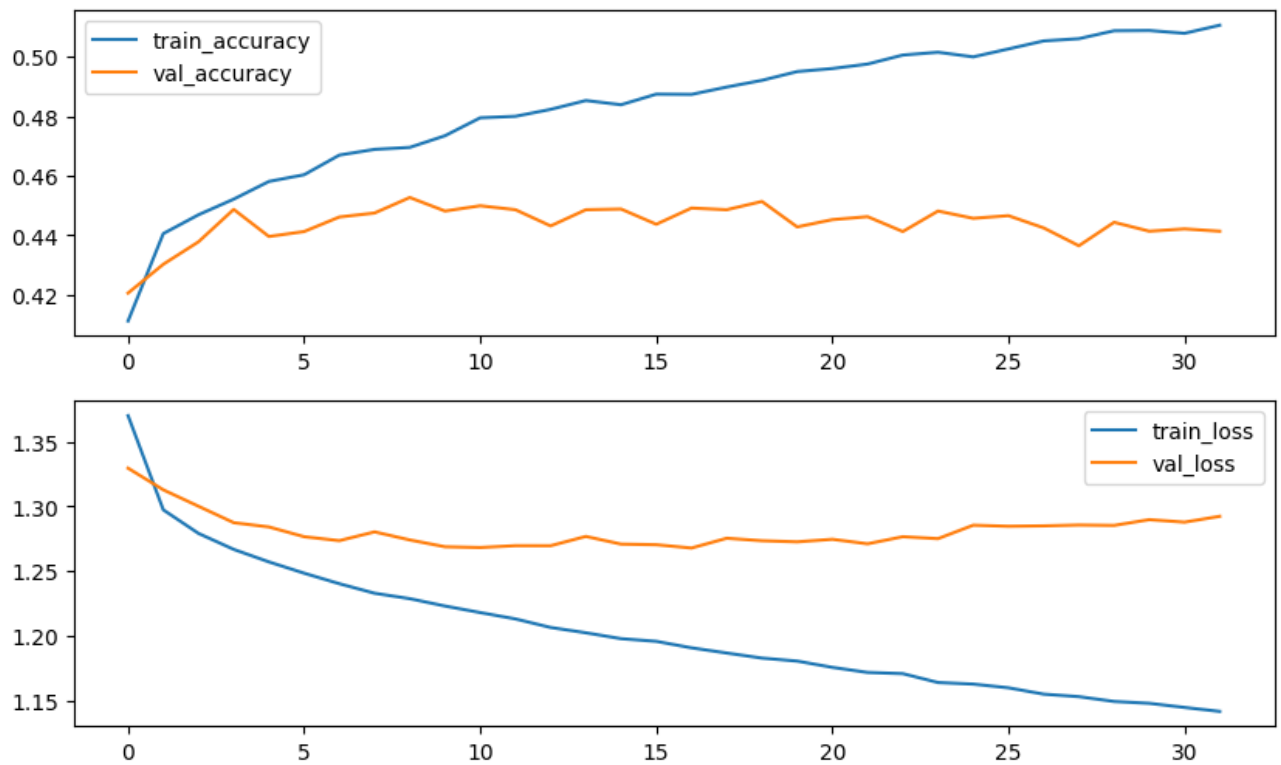
Model Metrics

```
In [77]: # Accuracy / Loss Train evolution
keras_train_plot(history.history)

# Accuracy / Loss values
accuracy, loss = keras_model_2.evaluate(list(X_train_var_splited.values()), y_train_ohe)

print("accuracy = {:.4f}, loss = {:.4f}".format(accuracy, loss))
```

Train process



```
1125/1125 ————— 1s 522us/step - accuracy: 0.5009 - loss: 1.1632
accuracy = 1.1674, loss = 0.5004
```

```
In [78]: # Model performance main metrics
keras_model_metrics(keras_model_2, list(X_test_var_splited.values()), y_test_ohe, styled
```

```
282/282 ————— 0s 1ms/step
```

Out[78]: **Functional Training**

metric	value
ROC AUC:	0.7735
Accuracy:	0.4491
Log loss:	1.2679
Kappa:	0.3115
MCC:	0.3129
F1:	0.4535

```
In [79]: # recall & precision, sensitivity & specificity
keras_sens_spec(keras_model_2, list(X_test_var_splited.values()), y_test_ohe, styled=True)
```

282/282 — 0s 958us/step

Out[79]:

Functional Model

				Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third- party Damages
Recall:	0.451200	-	Sensitivity:	0.3120	0.5718	0.4223	0.4602	0.4896
Precision:	0.464500	-	Specificity:	0.3164	0.4622	0.3568	0.5581	0.6291

In [80]: *# Confusion matrix*
 keras_confusion_matrix_table(keras_model_2, list(X_test_var_splited.values()), y_test_oh

282/282 — 0s 948us/step

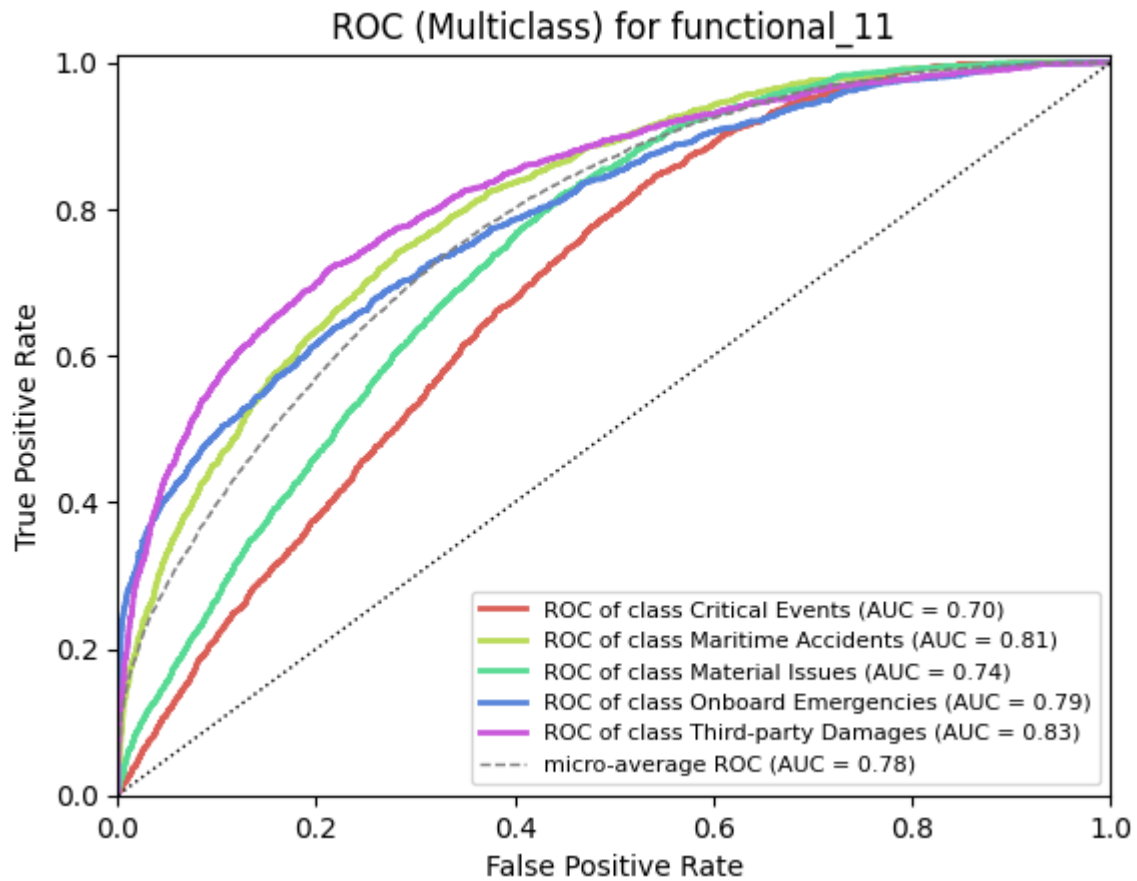
Out[80]:

functional_11: Confusion Matrix

	Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages
Critical Events	579	361	626	163	127
Maritime Accidents	240	979	292	99	102
Material Issues	463	340	774	143	113
Onboard Emergencies	287	190	295	816	185
Third-party Damages	261	248	182	241	894

```
In [81]: # Plot Roc curves
keras_roc_curve_plot(keras_model_2, list(X_test_var_splited.values()), y_test_ohe)
```

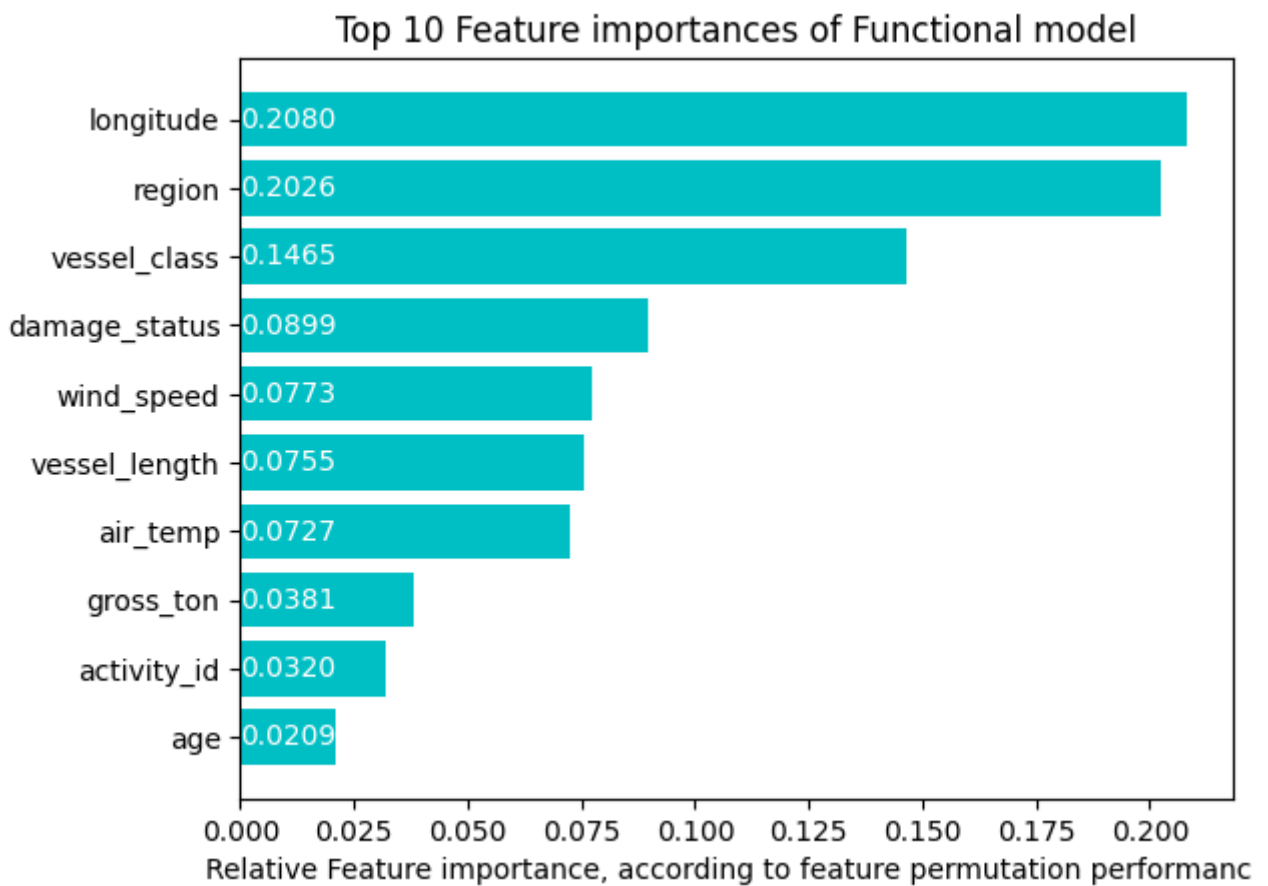
282/282 — 0s 940us/step



Feature importances

```
In [82]: # Call previous functions for plotting top 10 importances according to feature permutation
plot_importances(keras_func_permutation_importances(keras_model_2, X_train_var_splited,
```

1125/1125	1s	947us/step
1125/1125	1s	955us/step
1125/1125	1s	955us/step
1125/1125	1s	955us/step
1125/1125	1s	941us/step
1125/1125	1s	953us/step
1125/1125	1s	951us/step
1125/1125	1s	950us/step
1125/1125	1s	954us/step
1125/1125	1s	951us/step
1125/1125	1s	947us/step
1125/1125	1s	942us/step
1125/1125	1s	943us/step
1125/1125	1s	956us/step
1125/1125	1s	949us/step
1125/1125	1s	959us/step
1125/1125	1s	947us/step
1125/1125	1s	952us/step
1125/1125	1s	964us/step
1125/1125	1s	950us/step
1125/1125	1s	950us/step
1125/1125	1s	943us/step
1125/1125	1s	948us/step
1125/1125	1s	947us/step
1125/1125	1s	941us/step
1125/1125	1s	945us/step
1125/1125	1s	957us/step
1125/1125	1s	949us/step
1125/1125	1s	943us/step
1125/1125	1s	944us/step
1125/1125	1s	956us/step
1125/1125	1s	946us/step
1125/1125	1s	947us/step
1125/1125	1s	944us/step
1125/1125	1s	952us/step
1125/1125	1s	947us/step
1125/1125	1s	952us/step



2.5. H2o AutoML

h2o cluster starting

```
In [83]: # More than 1 attempt to init h2o cluster (to avoid problems)
def start_h2o_cluster():
    try:
        h2o.init() # Attempt to start the H2O cluster
        return True # Return True if the cluster starts successfully
    except Exception as e:
        return False # Return False if there is an error starting the cluster

attempts = 0
max_attempts = 3 # Maximum number of attempts

while attempts < max_attempts:
    if start_h2o_cluster():
        break # If the cluster starts successfully, exit the loop
    else:
        attempts += 1

if attempts == max_attempts:
    print("Failed to start H2O cluster after", max_attempts, "attempts.")
```

Checking whether there is an H2O instance running at http://localhost:54321..... not found.

Attempting to start a local H2O server...

; Java HotSpot(TM) 64-Bit Server VM (build 25.411-b09, mixed mode)

Starting server from C:\Users\0\anaconda3\Lib\site-packages\h2o\backend\bin\h2o.jar

Ice root: C:\Users\0\AppData\Local\Temp\tmpkuik5wif

JVM stdout: C:\Users\0\AppData\Local\Temp\tmpkuik5wif\h2o_0_started_from_python.out

JVM stderr: C:\Users\0\AppData\Local\Temp\tmpkuik5wif\h2o_0_started_from_python.err

Checking whether there is an H2O instance running at http://localhost:54321. connected.

Warning: Your H2O cluster version is (3 months and 27 days) old. There may be a newer version available.

Please download and install the latest version from: https://h2o-release.s3.amazonaws.com/h2o/latest_stable.html

H2O_cluster_uptime:	16 secs
H2O_cluster_timezone:	Europe/Paris
H2O_data_parsing_timezone:	UTC
H2O_cluster_version:	3.46.0.1
H2O_cluster_version_age:	3 months and 27 days
H2O_cluster_name:	H2O_from_python_0_50muma
H2O_cluster_total_nodes:	1
H2O_cluster_free_memory:	6.452 Gb
H2O_cluster_total_cores:	12
H2O_cluster_allowed_cores:	12
H2O_cluster_status:	locked, healthy
H2O_connection_url:	http://localhost:54321
H2O_connection_proxy:	{"http": null, "https": null}
H2O_internal_security:	False
Python_version:	3.11.5 final

Train / Test Split (h2o)

```
In [84]: # Convert dataframe to h2o format
h2o_data = h2o.H2OFrame(merged_activity_general)

# Train / Test split
h2o_train, h2o_test = h2o_data.split_frame(ratios=[0.85])
```

Parse progress:  (done)
100%

Model Training

```
In [85]: if train_model_enabled :
# AutoMachineLearning Model
aml = H2OAutoML(seed=seed, max_runtime_secs = 3000, keep_cross_validation_prediction

# Train
h2o_model = aml.train(y='y', training_frame=h2o_train)
```

Metrics (Calculated)

```
gbm prediction progress: |██████████████████████████████████████████| (done)
100%
gbm prediction progress: |██████████████████████████████████████████| (done)
100%
```

metric	value
ROC AUC:	0.8077
Accuracy:	0.4947
Log loss:	1.2072
Kappa:	0.3686
MCC:	0.3696
F1:	0.4944

```
gbm prediction progress: |████████████████████████████████████████| (done)
100%
```

GBM_grid_1_AutoML_1_20240628_231328_model_50
Model

Feature importances

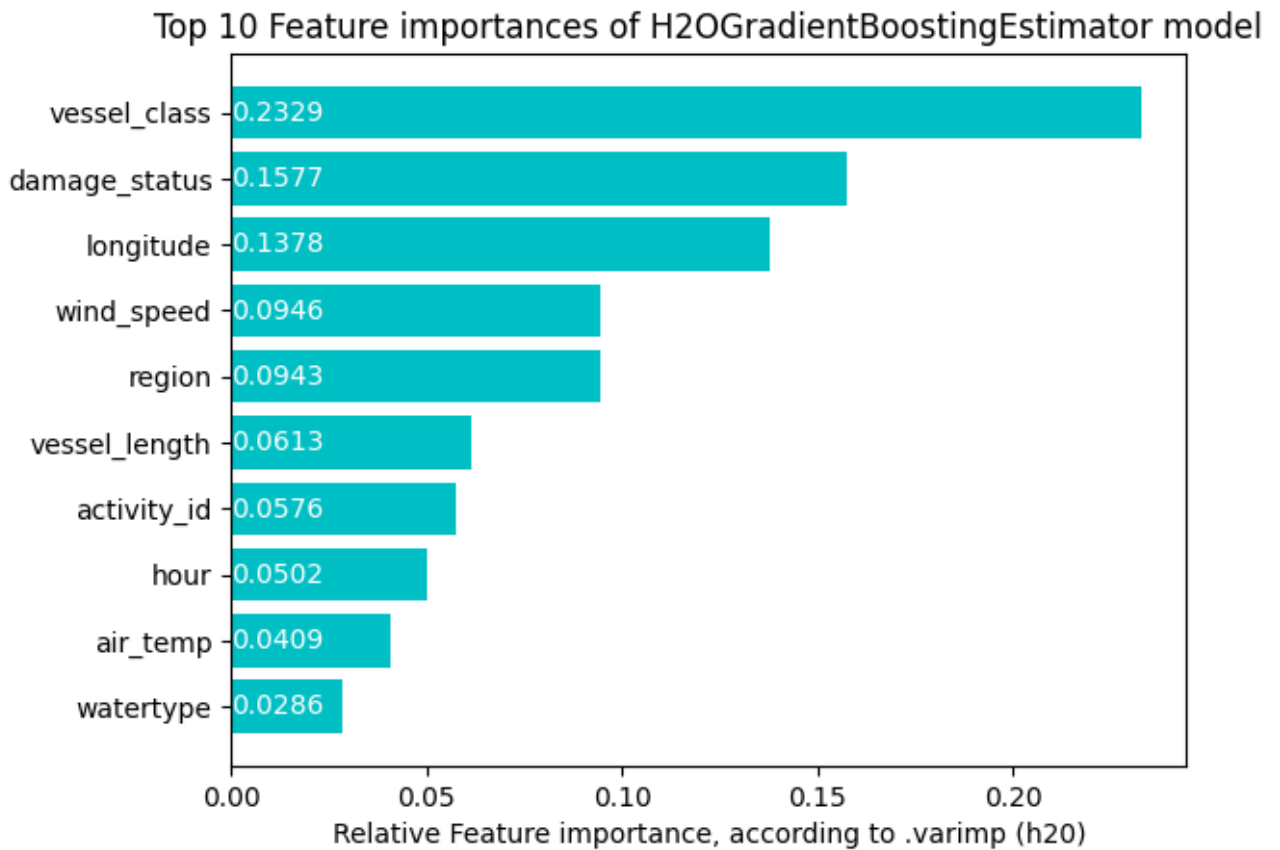
In [88]:

52.

```
importances.importance_method = '.varimp (h2o)'
```

```
return importances
```

```
In [89]: # Call previous functions for plotting top 10 importances according to .varimp (keras)
plot_importances(h2o_feature_importances(h2o_model), top=10)
```



Metrics (Auto-generated)

```
In [90]: # Best model details
h2o_model
```

Out[90]:

Model Details

=====

H2OGradientBoostingEstimator : Gradient Boosting Machine

Model Key: GBM_grid_1_AutoML_1_20240628_231328_model_50

Model Summary:

number_of_trees	number_of_internal_trees	model_size_in_bytes	min_depth	max_depth	me
104.0	520.0	177855.0	5.0	5.0	

ModelMetricsMultinomial: gbm

** Reported on train data. **

MSE: 0.4508532488959011

RMSE: 0.6714560662440254

LogLoss: 1.1900051851351936

Mean Per-Class Error: 0.4889595477595967

AUC table was not computed: it is either disabled (model parameter 'auc_type' was set to AUTO or NONE) or the domain size exceeds the limit (maximum is 50 domains).

AUCPR table was not computed: it is either disabled (model parameter 'auc_type' was set to AUTO or NONE) or the domain size exceeds the limit (maximum is 50 domains).

Confusion Matrix: Row labels: Actual class; Column labels: Predicted class

Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages	Error	Rate
2540.0	1180.0	2391.0	792.0	723.0	0.6669289	5.086 / 7.626
673.0	4515.0	1104.0	601.0	767.0	0.4105744	3.145 / 7.660
1335.0	1122.0	3831.0	726.0	682.0	0.5022089	3.865 / 7.696
924.0	734.0	1192.0	3922.0	884.0	0.4877220	3.734 / 7.656
666.0	708.0	716.0	804.0	4775.0	0.3773634	2.894 / 7.669
6138.0	8259.0	9234.0	6845.0	7831.0	0.4887879	18.724 / 38.307

Top-5 Hit Ratios:

k	hit_ratio
1	0.5112121
2	0.7448769
3	0.8787167

k	hit_ratio
---	-----------

4	0.9605817
---	-----------

5	1.0
---	-----

ModelMetricsMultinomial: gbm

** Reported on cross-validation data. **

MSE: 0.47996983364346496

RMSE: 0.6927985519929043

LogLoss: 1.2781845149562312

Mean Per-Class Error: 0.5525383124748917

AUC table was not computed: it is either disabled (model parameter 'auc_type' was set to AUTO or NONE) or the domain size exceeds the limit (maximum is 50 domains).

AUCPR table was not computed: it is either disabled (model parameter 'auc_type' was set to AUTO or NONE) or the domain size exceeds the limit (maximum is 50 domains).

Confusion Matrix: Row labels: Actual class; Column labels: Predicted class

Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages	Error	Rate
1899.0	1273.0	2752.0	914.0	788.0	0.7509835	5.727 / 7.626
767.0	4084.0	1222.0	742.0	845.0	0.4668407	3.576 / 7.660
1758.0	1252.0	3052.0	885.0	749.0	0.6034304	4.644 / 7.696
970.0	833.0	1286.0	3604.0	963.0	0.5292581	4.052 / 7.656
718.0	785.0	777.0	881.0	4508.0	0.4121789	3.161 / 7.669
6112.0	8227.0	9089.0	7026.0	7853.0	0.5523795	21.160 / 38.307

Top-5 Hit Ratios:

k	hit_ratio
---	-----------

1	0.4476205
---	-----------

2	0.6953821
---	-----------

3	0.8430313
---	-----------

4	0.9443704
---	-----------

5	0.9999999
---	-----------

Cross-Validation Metrics Summary:

	mean	sd	cv_1_valid	cv_2_valid	cv_3_valid	cv_4_valid	cv
accuracy	0.4476204	0.0022904	0.4506656	0.4488384	0.4477222	0.4460253	0.4488384
aic	nan	0.0	nan	nan	nan	nan	nan
auc	nan	0.0	nan	nan	nan	nan	nan
err	0.5523796	0.0022904	0.5493344	0.5511616	0.5522777	0.5539747	0.5511616
err_count	4232.0	17.291616	4209.0	4223.0	4231.0	4244.0	4223.0
loglikelihood	nan	0.0	nan	nan	nan	nan	nan
logloss	1.2781849	0.0069784	1.2709156	1.2716556	1.2815711	1.2792895	1.2716556
max_per_class_error	0.7509846	0.0139953	0.7424639	0.7698361	0.7567213	0.7527869	0.7567213
mean_per_class_accuracy	0.4474611	0.0022894	0.4505320	0.4486489	0.4475531	0.4458568	0.4486489
mean_per_class_error	0.5525389	0.0022894	0.549468	0.5513511	0.5524469	0.5541432	0.5513511
mse	0.4799699	0.0023366	0.4781609	0.4781010	0.4808136	0.4791127	0.4781010
pr_auc	nan	0.0	nan	nan	nan	nan	nan
r2	0.7596443	0.0011746	0.760591	0.7605583	0.759184	0.7600829	0.7605583
rmse	0.692797	0.0016849	0.6914918	0.6914484	0.6934073	0.6921797	0.6914484

Scoring History:

timestamp	duration	number_of_trees	training_rmse	training_logloss	training_classification
2024-06-28 23:22:48	7 min 11.819 sec	0.0	0.8000000	1.6094379	Cross-Validation
2024-06-28 23:22:48	7 min 11.874 sec	5.0	0.7633879	1.4649494	Cross-Validation
2024-06-28 23:22:48	7 min 11.930 sec	10.0	0.7426499	1.4009799	Cross-Validation
2024-06-28 23:22:48	7 min 11.994 sec	15.0	0.7289350	1.3613157	Cross-Validation
2024-06-28 23:22:48	7 min 12.054 sec	20.0	0.7205001	1.3379106	Cross-Validation
2024-06-28 23:22:48	7 min 12.111 sec	25.0	0.7136524	1.3184776	Cross-Validation
2024-06-28 23:22:48	7 min 12.164 sec	30.0	0.7080581	1.3022030	Cross-Validation

timestamp	duration	number_of_trees	training_rmse	training_logloss	training_classification
2024-06-28 23:22:48	7 min 12.216 sec	35.0	0.7035127	1.2892344	C
2024-06-28 23:22:48	7 min 12.271 sec	40.0	0.6995442	1.2774777	C
2024-06-28 23:22:48	7 min 12.324 sec	45.0	0.6962857	1.2676348	C
---	---	---	---	---	---
2024-06-28 23:22:49	7 min 12.496 sec	60.0	0.6883589	1.2430378	C
2024-06-28 23:22:49	7 min 12.553 sec	65.0	0.6856821	1.2348036	C
2024-06-28 23:22:49	7 min 12.605 sec	70.0	0.6836955	1.2287287	C
2024-06-28 23:22:49	7 min 12.659 sec	75.0	0.6814822	1.2216597	C
2024-06-28 23:22:49	7 min 12.714 sec	80.0	0.6797271	1.2163080	C
2024-06-28 23:22:49	7 min 12.768 sec	85.0	0.6780260	1.2107911	C
2024-06-28 23:22:49	7 min 12.824 sec	90.0	0.6762315	1.2050725	C
2024-06-28 23:22:49	7 min 12.877 sec	95.0	0.6743988	1.1993096	C
2024-06-28 23:22:49	7 min 12.933 sec	100.0	0.6726608	1.1937864	C
2024-06-28 23:22:49	7 min 12.981 sec	104.0	0.6714561	1.1900052	C

[22 rows x 9 columns]

Variable Importances:

variable	relative_importance	scaled_importance	percentage
vessel_class	7004.7416992	1.0	0.2328711
damage_status	4742.7651367	0.6770792	0.1576722

variable	relative_importance	scaled_importance	percentage
longitude	4144.0849609	0.5916114	0.1377692
wind_speed	2844.1596680	0.4060335	0.0945535
region	2835.3781738	0.4047798	0.0942615
vessel_length	1845.1317139	0.2634118	0.0613410
activity_id	1731.7703857	0.2472283	0.0575723
hour	1510.7999268	0.2156825	0.0502262
air_temp	1228.7844238	0.1754218	0.0408507
watertype	860.9852905	0.1229146	0.0286233
age	704.7161255	0.1006056	0.0234281
gross_ton	626.5904541	0.0894523	0.0208309

[tips]

Use `model.explain()` to inspect the model.

--

Use `h2o.display.toggle_user_tips()` to switch on/off this section.

```
In [91]: # Explanation inform
h2o_model.explain(h2o_test, exclude_explanations='pdp')

# ALL behind is generated by this method
```

Confusion Matrix

Confusion matrix shows a predicted class vs an actual class.

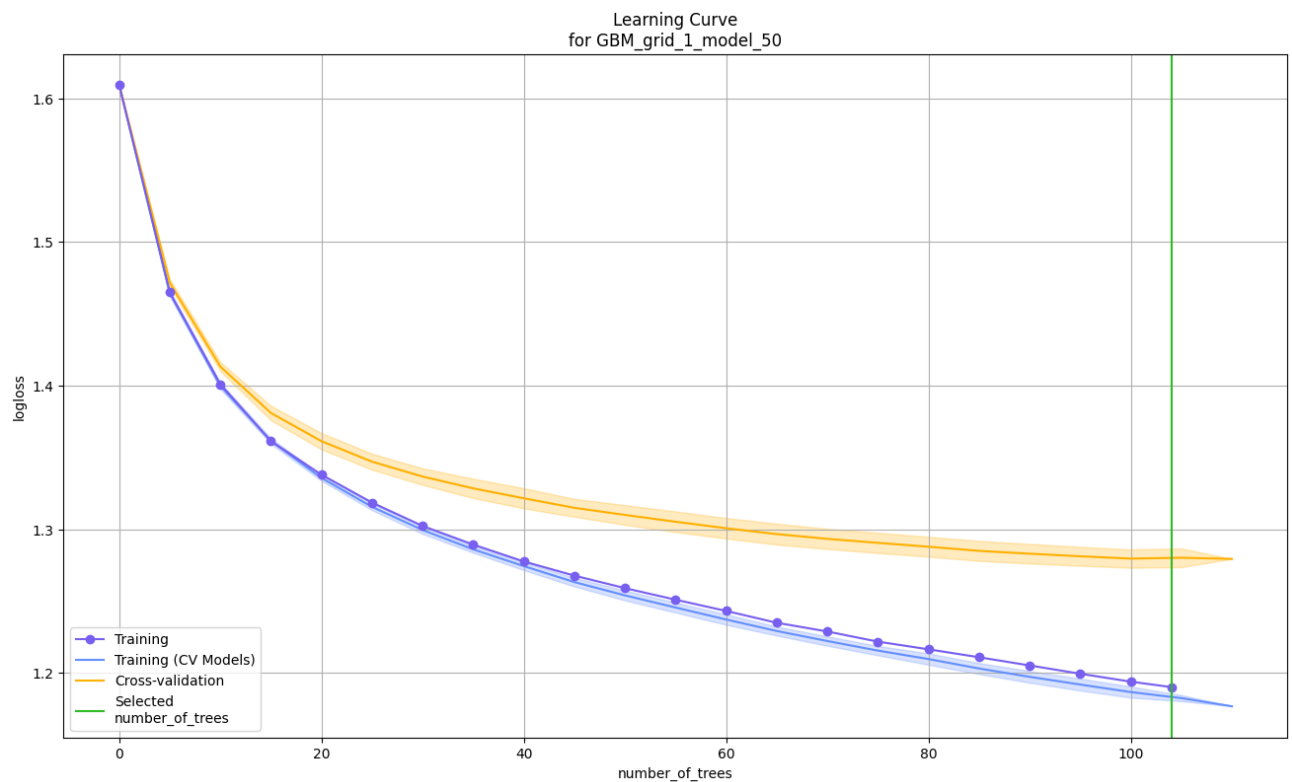
GBM_grid_1_AutoML_1_20240628_231328_model_50

Confusion Matrix: Row labels: Actual class; Column labels: Predicted class

Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages	Error	Rate
449.0	199.0	438.0	151.0	125.0	0.6703377	913 / 1.362
121.0	731.0	198.0	117.0	135.0	0.4385561	571 / 1.302
249.0	195.0	675.0	118.0	106.0	0.4973939	668 / 1.343
197.0	125.0	232.0	678.0	176.0	0.5184659	730 / 1.408
136.0	116.0	112.0	157.0	799.0	0.3946970	521 / 1.320
1152.0	1366.0	1655.0	1221.0	1341.0	0.5052710	3.403 / 6.735

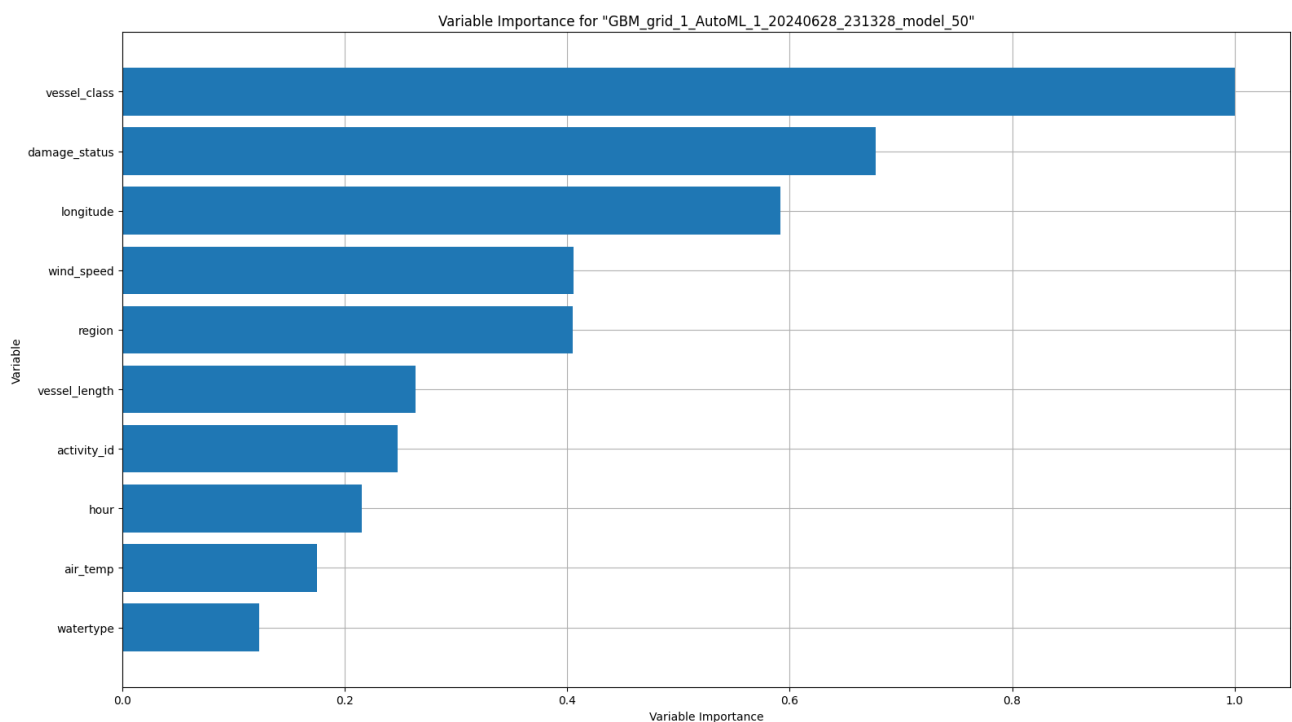
Learning Curve Plot

Learning curve plot shows the loss function/metric dependent on number of iterations or trees for tree-based algorithms. This plot can be useful for determining whether the model overfits.



Variable Importance

The variable importance plot shows the relative importance of the most important variables in the model.



Confusion Matrix

Confusion matrix shows a predicted class vs an actual class.

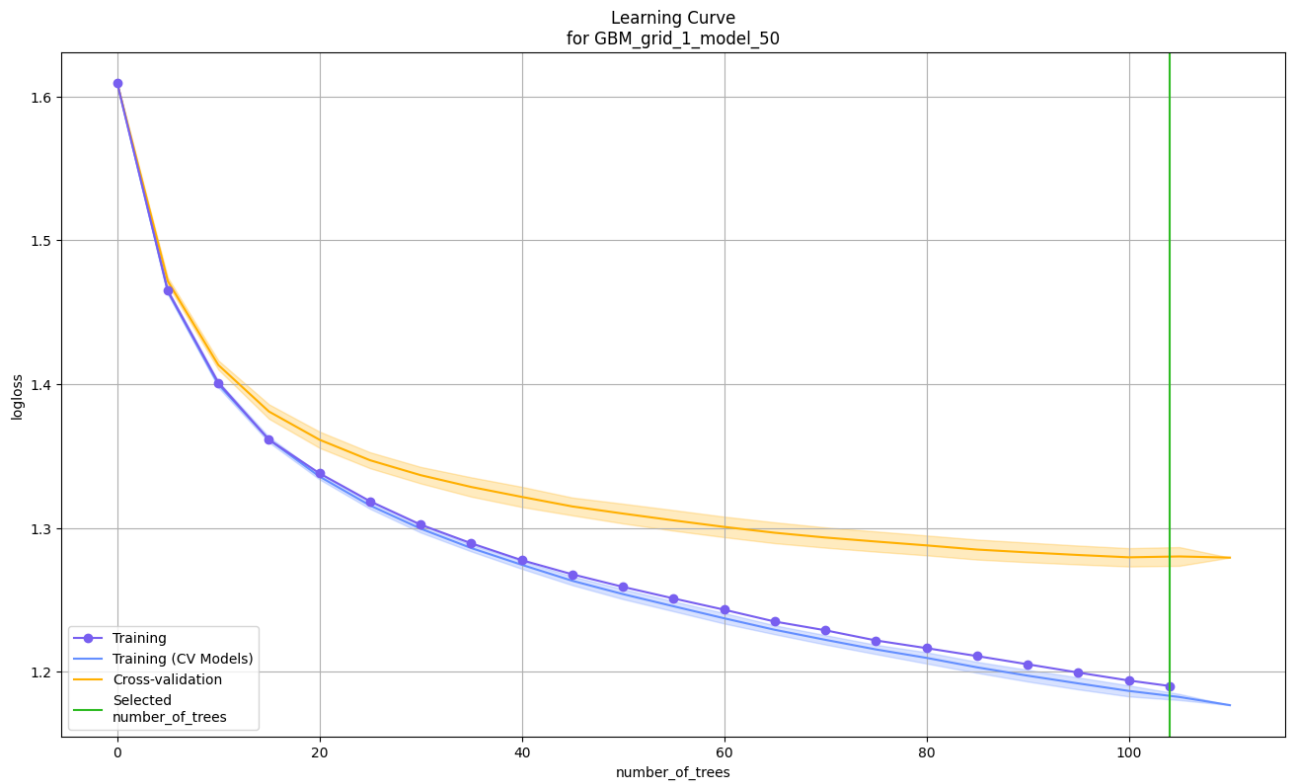
GBM_grid_1_AutoML_1_20240628_231328_model_50

Confusion Matrix: Row labels: Actual class; Column labels: Predicted class

Critical Events	Maritime Accidents	Material Issues	Onboard Emergencies	Third-party Damages	Error	Rate
449.0	199.0	438.0	151.0	125.0	0.6703377	913 / 1.362
121.0	731.0	198.0	117.0	135.0	0.4385561	571 / 1.302
249.0	195.0	675.0	118.0	106.0	0.4973939	668 / 1.343
197.0	125.0	232.0	678.0	176.0	0.5184659	730 / 1.408
136.0	116.0	112.0	157.0	799.0	0.3946970	521 / 1.320
1152.0	1366.0	1655.0	1221.0	1341.0	0.5052710	3.403 / 6.735

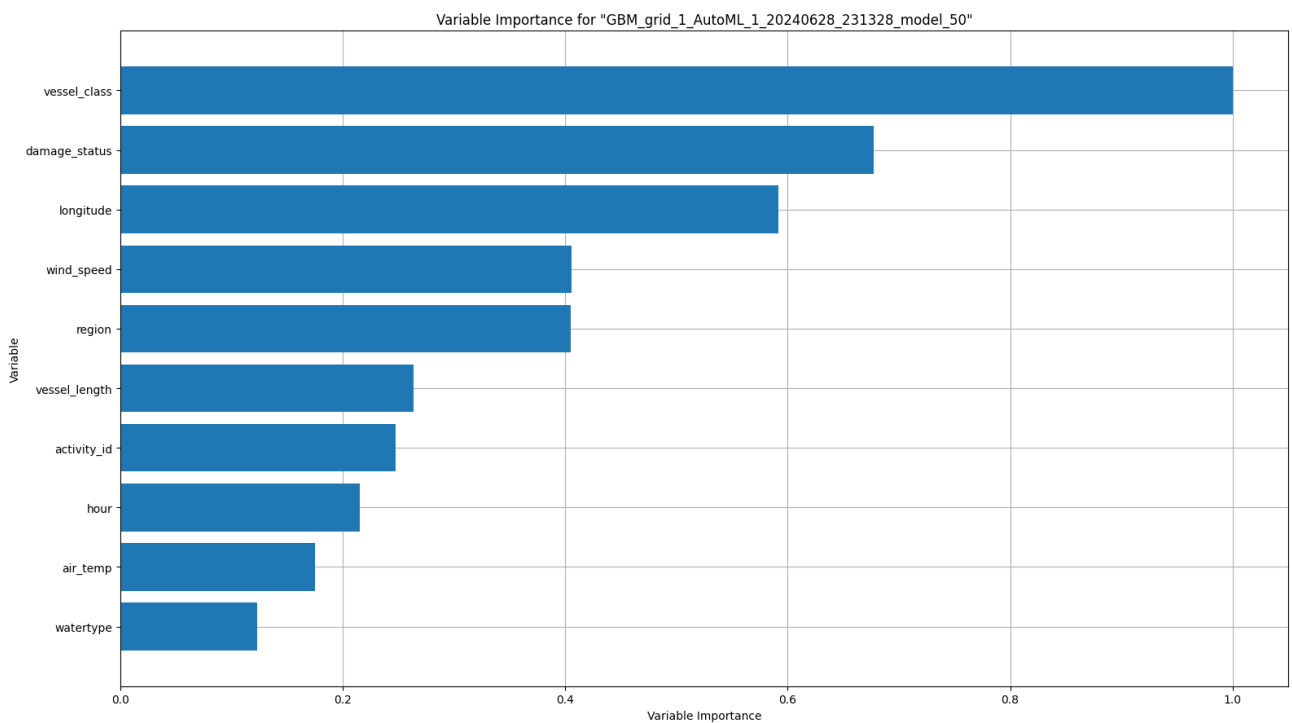
Learning Curve Plot

Learning curve plot shows the loss function/metric dependent on number of iterations or trees for tree-based algorithms. This plot can be useful for determining whether the model overfits.



Variable Importance

The variable importance plot shows the relative importance of the most important variables in the model.



3. Model Comparison

```
In [92]: # Midpoint milestone: Load all trained models from here

# Model Lists by training framework
sklearn_models = ['nb_MA_train', 'GBM_MA_train', 'rf_MA_train', 'nnet_MA_train', 'cart_M
keras_models = ['keras_model_1', 'keras_model_2']
h2o_models = ['mod_aml']
```

```
# Load models by training framework
for model in sklearn_models:
    globals()[model] = joblib.load(models_folder + '/' + model + '.pkl')
    print(f'{model} loaded')
for model in keras_models:
    globals()[model] = load_model(models_folder + '/' + model + '.keras')
    print(f'{model} loaded')
for model in h2o_models:
    globals()[model] = h2o.load_model(models_folder + '/' + model + '.h2o')
    print(f'{model} loaded')
```

```
nb_MA_train loaded
GBM_MA_train loaded
rf_MA_train loaded
nnet_MA_train loaded
cart_MA_train loaded
keras_model_1 loaded
keras_model_2 loaded
mod_aml loaded
```

3.1. Features importance

Heatmap of ALL Feature importances for trained models

```
In [93]: # Concatenate variable importance values by variable_name as index
all_models_importances = pd.concat([
    permutation_importances(nb_MA_train, X_train, y_train).set_index('variable_name'),
    sklearn_importances(GBM_MA_train).set_index('variable_name'),
    sklearn_importances(rf_MA_train).set_index('variable_name'),
    permutation_importances(nnet_MA_train, X_train, y_train).set_index('variable_name'),
    sklearn_importances(cart_MA_train).set_index('variable_name'),

    keras_sec_permutation_importances(keras_model_1, X_train, y_train_ohe).set_index('va
    keras_func_permutation_importances(keras_model_2, X_train_var_splited, y_train_ohe).

    h2o_feature_importances(h2o_model).set_index('variable_name')
], axis=1).sort_index()

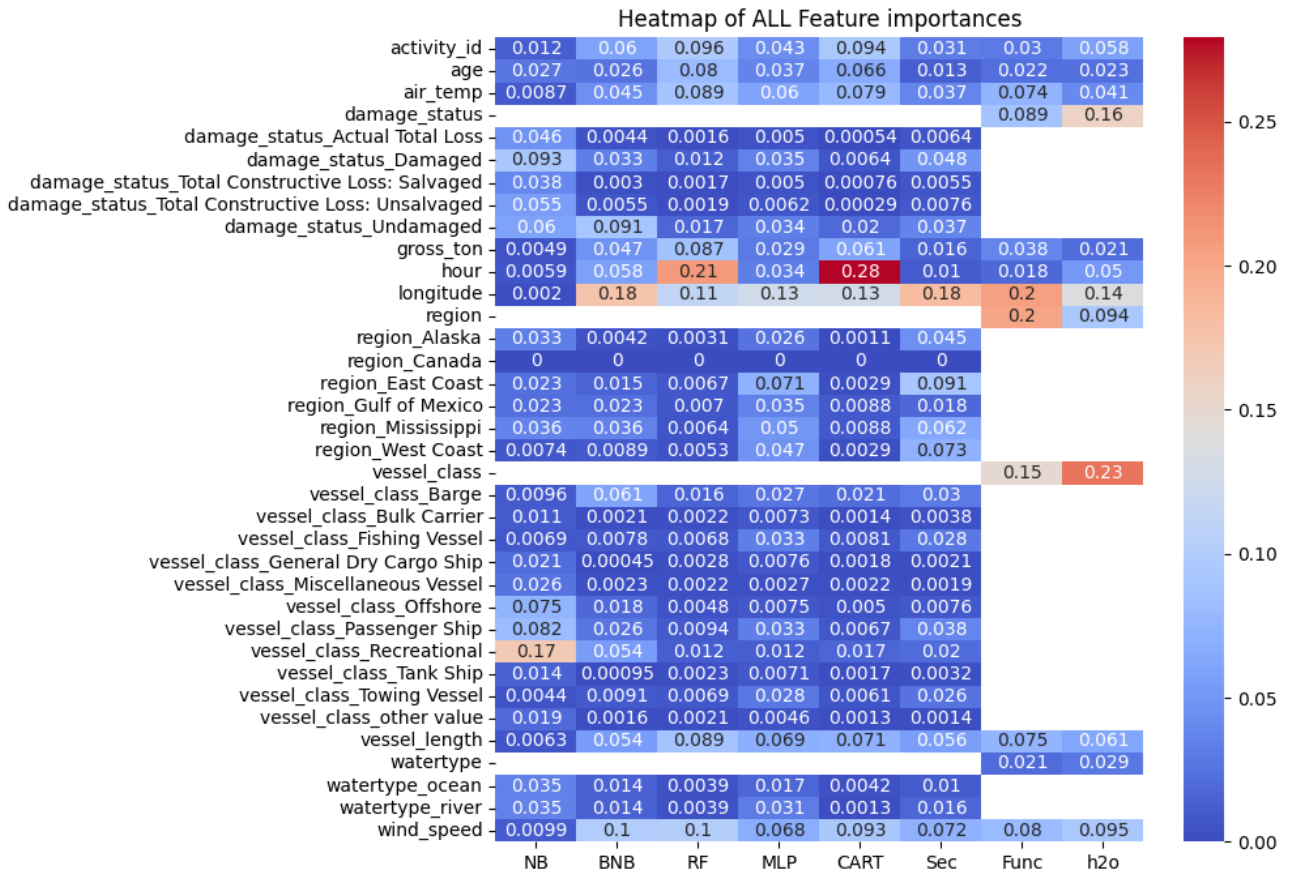
# Model names for column naming
all_models_importances.columns = ('NB', 'BNB', 'RF', 'MLP', 'CART', 'Sec', 'Func', 'h2o'

# Heatmap creation
plt.figure(figsize=(8, 8))
sns.heatmap(all_models_importances, annot=True, cmap='coolwarm')
plt.title('Heatmap of ALL Feature importances')
plt.ylabel('')
plt.show()
```

1125/1125	0s	399us/step
1125/1125	0s	399us/step
1125/1125	0s	403us/step
1125/1125	0s	403us/step
1125/1125	0s	407us/step
1125/1125	0s	413us/step
1125/1125	0s	410us/step
1125/1125	0s	406us/step
1125/1125	0s	410us/step
1125/1125	0s	403us/step
1125/1125	0s	397us/step
1125/1125	0s	433us/step
1125/1125	0s	401us/step
1125/1125	0s	389us/step
1125/1125	0s	384us/step
1125/1125	0s	388us/step
1125/1125	0s	399us/step
1125/1125	0s	385us/step
1125/1125	0s	390us/step
1125/1125	0s	385us/step
1125/1125	0s	388us/step
1125/1125	0s	389us/step
1125/1125	0s	384us/step
1125/1125	0s	384us/step
1125/1125	0s	384us/step
1125/1125	0s	384us/step
1125/1125	0s	384us/step
1125/1125	0s	384us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	384us/step
1125/1125	0s	390us/step
1125/1125	0s	384us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	385us/step
1125/1125	0s	384us/step
1125/1125	0s	385us/step
1125/1125	0s	390us/step
1125/1125	0s	384us/step
1125/1125	0s	395us/step
1125/1125	0s	395us/step
1125/1125	0s	390us/step
1125/1125	0s	384us/step
1125/1125	0s	385us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	385us/step
1125/1125	0s	390us/step
1125/1125	0s	386us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	379us/step
1125/1125	0s	390us/step
1125/1125	0s	385us/step
1125/1125	0s	390us/step
1125/1125	0s	390us/step
1125/1125	0s	384us/step
1125/1125	0s	384us/step
1125/1125	0s	390us/step
1125/1125	0s	383us/step

1125/1125	0s	391us/step
1125/1125	0s	377us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	377us/step
1125/1125	0s	390us/step
1125/1125	0s	405us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	396us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	390us/step
1125/1125	0s	391us/step
1125/1125	0s	390us/step
1125/1125	0s	397us/step
1125/1125	0s	390us/step
1125/1125	0s	392us/step
1125/1125	0s	390us/step
1125/1125	0s	391us/step
1125/1125	0s	405us/step
1125/1125	0s	405us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	391us/step
1125/1125	0s	405us/step
1125/1125	0s	418us/step
1125/1125	0s	411us/step
1125/1125	0s	433us/step
1125/1125	0s	418us/step
1125/1125	1s	935us/step
1125/1125	1s	932us/step
1125/1125	1s	913us/step
1125/1125	1s	876us/step
1125/1125	1s	879us/step
1125/1125	1s	890us/step
1125/1125	1s	891us/step
1125/1125	1s	876us/step
1125/1125	1s	887us/step
1125/1125	1s	876us/step
1125/1125	1s	883us/step
1125/1125	1s	880us/step
1125/1125	1s	891us/step
1125/1125	1s	882us/step
1125/1125	1s	869us/step
1125/1125	1s	879us/step
1125/1125	1s	876us/step
1125/1125	1s	876us/step
1125/1125	1s	883us/step
1125/1125	1s	887us/step
1125/1125	1s	887us/step
1125/1125	1s	891us/step
1125/1125	1s	902us/step
1125/1125	1s	890us/step
1125/1125	1s	894us/step
1125/1125	1s	894us/step
1125/1125	1s	911us/step
1125/1125	1s	883us/step
1125/1125	1s	907us/step

1125/1125 — 1s 893us/step
 1125/1125 — 1s 879us/step
 1125/1125 — 1s 894us/step
 1125/1125 — 1s 893us/step
 1125/1125 — 1s 893us/step
 1125/1125 — 1s 900us/step
 1125/1125 — 1s 897us/step
 1125/1125 — 1s 948us/step



Heatmap of GROUPED Feature importances for trained models

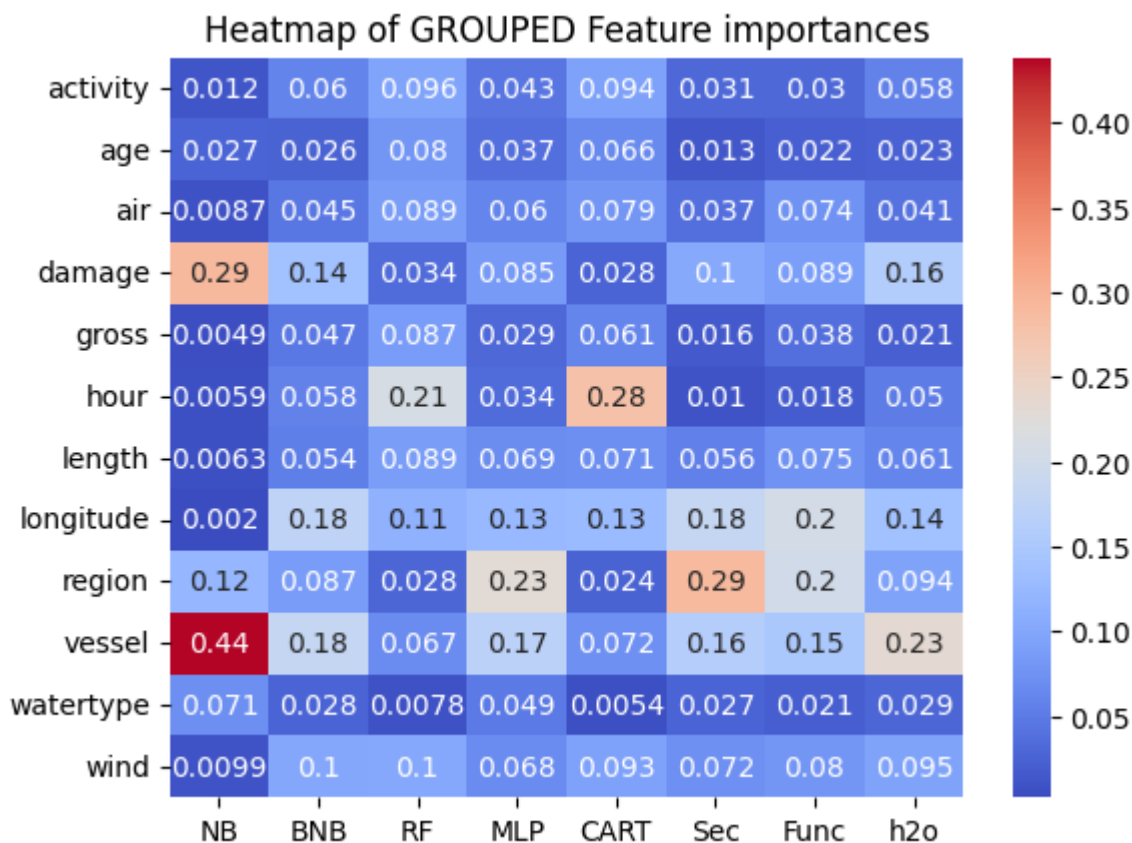
Same as previous but grouping by variable

```
In [94]: # Rename vessel_length for better distinction
all_models_importances.index = all_models_importances.index.str.replace('vessel_length',

# Create a new variable for the group
all_models_importances['group'] = all_models_importances.index.to_series().str.split('_')

# Sum calculated vertically by group
all_models_importances_grouped = all_models_importances.groupby('group').sum()

# Heatmap creation
plt.figure()
sns.heatmap(all_models_importances_grouped , annot=True, cmap='coolwarm')
plt.title('Heatmap of GROUPED Feature importances')
plt.ylabel('')
plt.show()
```



3.2. Performance of the models

Main metrics comparison chart

```
In [95]: # Function: join model metrics in one comparison table
def comparison_table(X, X_var_splited, y, y_ohe, h2o_data, title = 'Model performance c
# Build dataframe for main metric
metrics_df = pd.DataFrame({
    'NB': ma_model_metrics(nb_MA_train, X, y)['value'],
    'GBM': ma_model_metrics(GBM_MA_train, X, y)['value'],
    'RF': ma_model_metrics(rf_MA_train, X, y)['value'],
    'NNET': ma_model_metrics(nnet_MA_train, X, y)['value'],
    'CART': ma_model_metrics(cart_MA_train, X, y)['value'],

    'keras Sec': keras_model_metrics(keras_model_1, X, y_ohe)['value'],
    'keras Func': keras_model_metrics(keras_model_2, list(X_var_splited.values()), y

    'h2o': h2o_model_metrics(h2o_model, h2o_data)['value']
}).set_index(ma_model_metrics(eval(sklearn_models[0]), X, y)['metric'])

# Build a dataframe for recall and precision
recprec_df = pd.DataFrame({
    'NB': sens_spec(nb_MA_train, X, y).index.get_level_values(1),
    'GBM': sens_spec(GBM_MA_train, X, y).index.get_level_values(1),
    'RF': sens_spec(rf_MA_train, X, y).index.get_level_values(1),
    'NNET': sens_spec(nnet_MA_train, X, y).index.get_level_values(1),
    'CART': sens_spec(cart_MA_train, X, y).index.get_level_values(1),

    'keras Sec': keras_sens_spec(keras_model_1, X, y_ohe).index.get_level_values(1),
    'keras Func': keras_sens_spec(keras_model_2, list(X_var_splited.values()), y_ohe

    'h2o': h2o_sens_spec(h2o_model, h2o_data).index.get_level_values(1)
}).set_index(sens_spec(eval(sklearn_models[0]), X, y).index.get_level_values(0))

# Join both dataframes vertically, transpose and sort by ROC AUC
```

```

df = pd.concat([metrics_df, recprec_df]).T.sort_values(['ROC AUC:'], ascending=[False])

# Add title
df.columns = pd.MultiIndex.from_tuples([(title, col) for col in df.columns])

return df

```

```

In [96]: # Call previous function for training samples
train_comparison_table = comparison_table(X_train, X_train_var_splited, y_train, y_train_ohe,
                                          title='Training Samples')

# Show table
train_comparison_table

```

```

1125/1125 ██████████ 0s 433us/step
1125/1125 ██████████ 1s 961us/step
gbm prediction progress: |████████████████████████████████████████████████████████████████████████████████| (done)
100%
gbm prediction progress: |████████████████████████████████████████████████████████████████████████████████| (done)
100%
1125/1125 ██████████ 0s 387us/step
1125/1125 ██████████ 1s 908us/step
gbm prediction progress: |████████████████████████████████████████████████████████████████████████████████| (done)
100%

```

Out[96]: **Model performance comparison for training samples**

	ROC AUC:	Accuracy:	Log loss:	Kappa:	MCC:	F1:	Recall:	Precision:
RF	0.9588	0.8668	0.6305	0.8334	0.8338	0.8645	0.8665	0.8639
CART	0.8989	0.8152	5.7058	0.7690	0.7697	0.8130	0.8150	0.8140
GBM	0.8227	0.5238	1.1612	0.4047	0.4053	0.5221	0.5233	0.5238
NNET	0.8204	0.5128	1.1424	0.3908	0.3938	0.5055	0.5121	0.5129
keras Func	0.8118	0.5004	1.1674	0.3754	0.3768	0.5033	0.4999	0.5145
h2o	0.8100	0.5033	1.1998	0.3790	0.3800	0.5011	0.5030	0.5034
keras Sec	0.7724	0.4512	1.2733	0.3140	0.3188	0.4470	0.4508	0.4687
NB	0.6657	0.3375	4.2296	0.1712	0.1888	0.2970	0.3366	0.3952

```

In [97]: # Call previous function for test samples
test_comparison_table = comparison_table(X_test, X_test_var_splited, y_test, y_test_ohe,
                                         title='Test Samples')

# Show table
test_comparison_table

```

```

282/282 ██████████ 0s 466us/step
282/282 ██████████ 0s 1ms/step
gbm prediction progress: |████████████████████████████████████████████████████████████████████████████████| (done)
100%
gbm prediction progress: |████████████████████████████████████████████████████████████████████████████████| (done)
100%
282/282 ██████████ 0s 449us/step
282/282 ██████████ 0s 917us/step
gbm prediction progress: |████████████████████████████████████████████████████████████████████████████████| (done)
100%

```

Out[97]:

Model performance comparison for test samples

	ROC AUC:	Accuracy:	Log loss:	Kappa:	MCC:	F1:	Recall:	Precision:
RF	0.8587	0.6427	1.1947	0.5532	0.5561	0.6376	0.6438	0.6458
h2o	0.8077	0.4947	1.2072	0.3686	0.3696	0.4944	0.4961	0.4968
GBM	0.7914	0.4832	1.2390	0.3541	0.3549	0.4823	0.4849	0.4835
NNET	0.7754	0.4518	1.2952	0.3155	0.3187	0.4455	0.4548	0.4528
keras Func	0.7735	0.4491	1.2679	0.3115	0.3129	0.4535	0.4512	0.4645
keras Sec	0.7566	0.4326	1.3044	0.2909	0.2953	0.4295	0.4343	0.4503
CART	0.7504	0.5904	14.2234	0.4879	0.4910	0.5832	0.5914	0.5904
NB	0.6579	0.3252	4.3607	0.1592	0.1760	0.2853	0.3287	0.3792

In [98]:

```
# Function: lollipop for previous comparision tables
def plot_comparison_table(df, title='Metrics comparison'):
    # Drop title from table
    df.columns = df.columns.get_level_values(-1)
    # Create subplots with a 4x2 grid
    fig, axes = plt.subplots(nrows=4, ncols=2, figsize=(8, 8))

    # Flatten the axes array for easy iteration
    axes = axes.flatten()

    # Function to create a horizontal lollipop chart on a specific subplot
    def plot_lollipop(ax, df, metric):
        # Sort models by the metric
        df_sorted = df.sort_values(by=metric)

        # Create the lollipop chart
        ax.hlines(y=df_sorted.index, xmin=0, xmax=df_sorted[metric], color='#00bfc4')
        ax.plot(df_sorted[metric], df_sorted.index, 'o', color='#00bfc4')
        ax.set_title(f'{metric}')

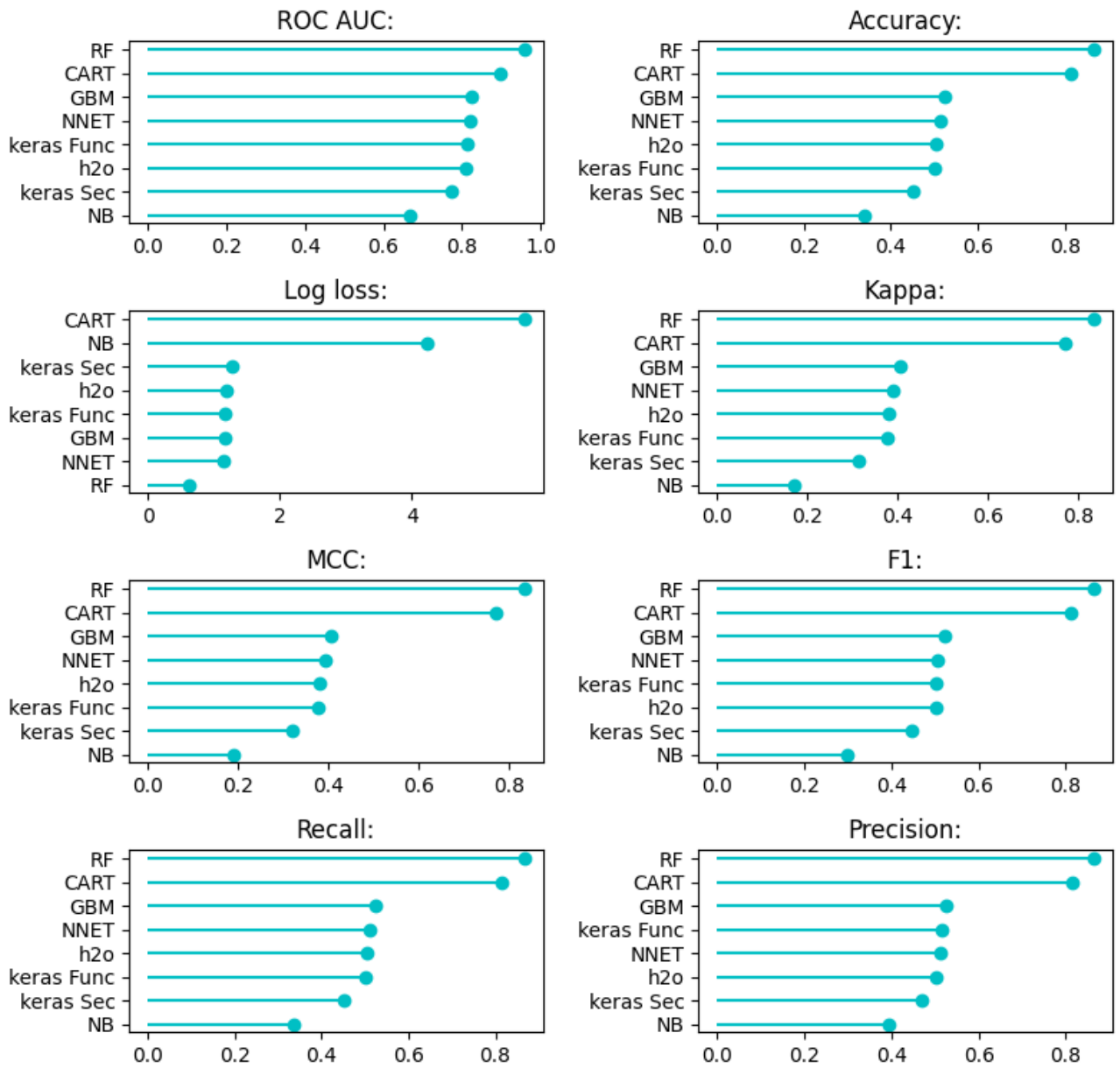
    # Create lollipop charts for each metric
    for ax, metric in zip(axes, df.columns):
        plot_lollipop(ax, df, metric)

    # Adjust the space between subplots
    plt.suptitle(title)
    plt.tight_layout()
    plt.show()
```

In [99]:

```
# Call plot function for train samples
plot_comparison_table(train_comparison_table, title = 'Metrics comparison for train samp
```

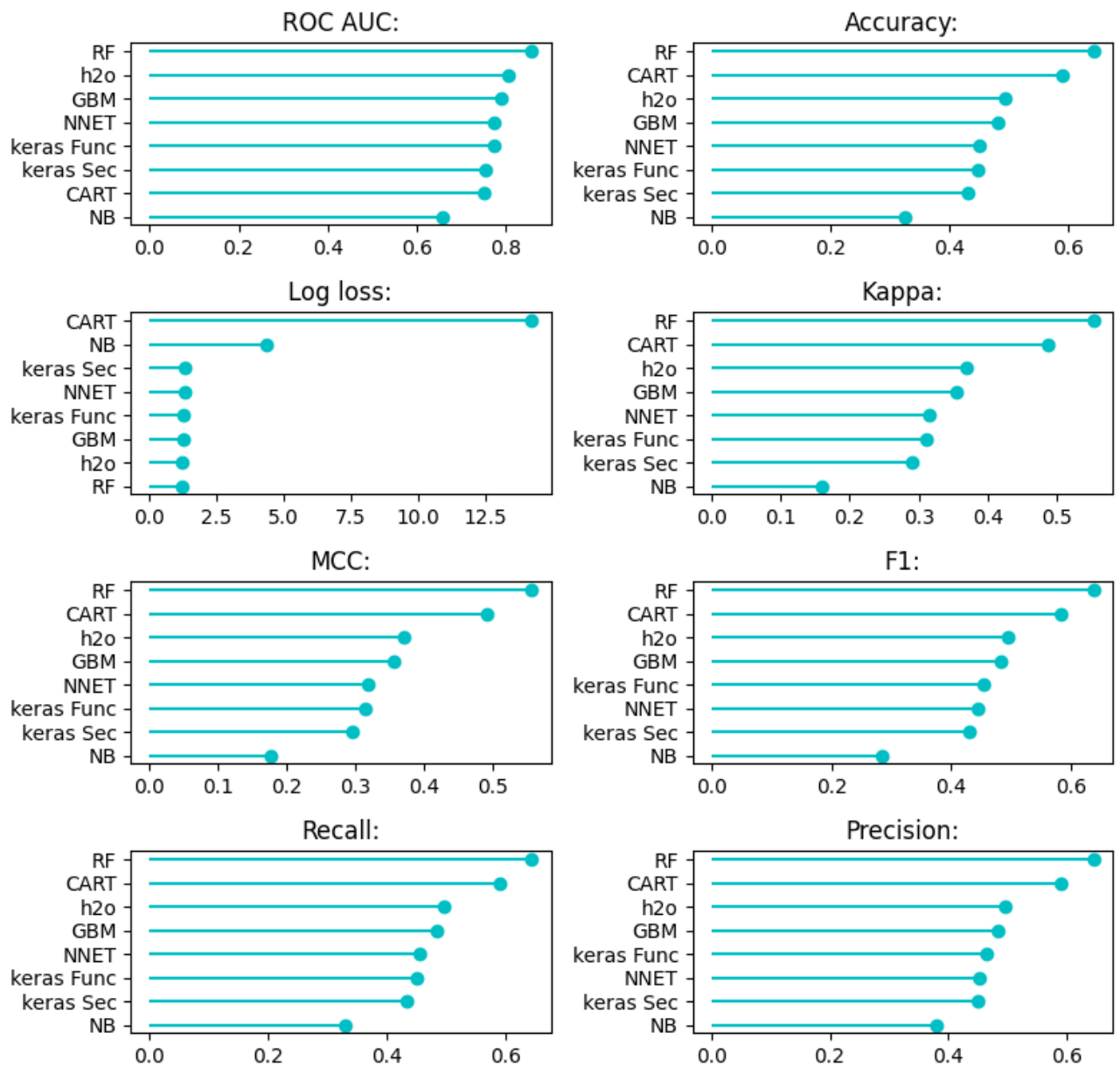
Metrics comparison for train samples



In [100...

```
# Call plot function for test samples
plot_comparison_table(test_comparison_table, title = 'Metrics comparison for test sample
```

Metrics comparison for test samples



Recall & Precision comparison chart (Sensitivity & Specificity)

```
In [101... # Built dataframe with all sens_spec functions along
recprec_comparison = pd.DataFrame({
    'NB': sens_spec(nb_MA_train, X_test, y_test).index.get_level_values(1),
    'GBM': sens_spec(GBM_MA_train, X_test, y_test).index.get_level_values(1),
    'RF': sens_spec(rf_MA_train, X_test, y_test).index.get_level_values(1),
    'NNET': sens_spec(nnet_MA_train, X_test, y_test).index.get_level_values(1),
    'CART': sens_spec(cart_MA_train, X_test, y_test).index.get_level_values(1),

    'keras Sec': keras_sens_spec(keras_model_1, X_test, y_test_ohe).index.get_level_valu
    'keras Func': keras_sens_spec(keras_model_2, list(X_test_var_splited.values()), y_te

    'h2o': h2o_sens_spec(h2o_model, h2o_test).index.get_level_values(1)
}).set_index(sens_spec(nb_MA_train, X_test, y_test).index.get_level_values(0))

# Show table
recprec_comparison
```

282/282 ————— 0s 399us/step

282/282 ————— 0s 947us/step

```
gbm prediction progress: |████████████████████████████████████████| (done)
100%
```

Out[101...

	NB	GBM	RF	NNET	CART	keras Sec	keras Func	h2o
Recall:	0.3287	0.4849	0.6438	0.4548	0.5914	0.4343	0.4512	0.4961
Precision:	0.3792	0.4835	0.6458	0.4528	0.5904	0.4503	0.4645	0.4968

Accuracy for Cross Validation (10 splits) - Sklearn models

In [102...

```
# Data will be exported due to the computing time needed
if file_export_enabled :
    # Calculate scores for accuracy for sklearn models
    kfold = KFold(n_splits=10, shuffle=True, random_state=seed)
    accuracy_scores = []
    for model in sklearn_models:
        accuracy_scores.append(cross_val_score(eval(model), X_train, y_train, scoring='a

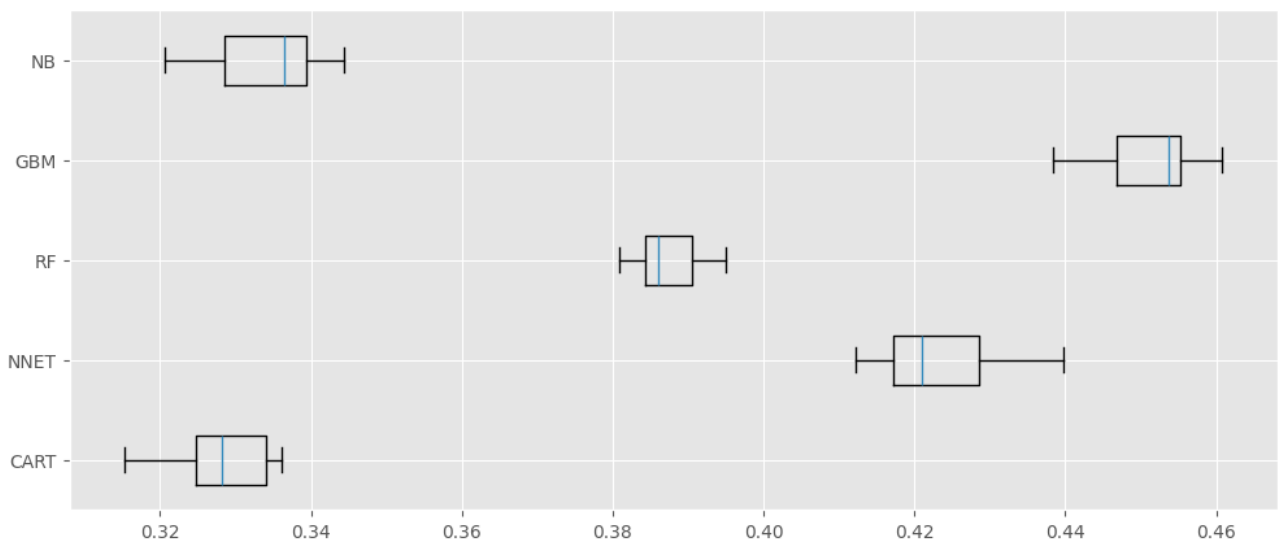
    # Store scores in a pandas dataframe and export to external file
    accuracy_scores = pd.DataFrame(accuracy_scores)
    accuracy_scores.columns = accuracy_scores.columns.astype(str)
    accuracy_scores.reset_index().to_feather(datasets_folder + '/' + 'accuracy_scores_MA

else:
    # Load this dataframe otherwise
    accuracy_scores = pd.read_feather(datasets_folder + '/' + 'accuracy_scores_MA.feather
```

In [103...

```
# Boxplots of accuracy values in cross validation
plt.style.use('ggplot')
fig = plt.figure(figsize=(12,5))
fig.suptitle('Cross Validation Accuracy')
ax = fig.add_subplot(111)
plt.boxplot(accuracy_scores[:, -1].T.iloc[1:].reset_index(drop=True), vert=False)
ax.set_yticklabels([model.split('_')[0].upper() for model in sklearn_models][:-1])
plt.show()
```

Cross Validation Accuracy



In [104...

```
from scipy.stats import ttest_rel

# Convert score data to list
score_list = accuracy_scores.values.tolist()

# Initiate table
table=[]
for i in range(len(score_list)):
```

```

table.append([])
for i in range(len(score_list)):
    for j in range(len(score_list)):
        table[i].append(0)

# Populate values
for i in range(len(score_list)):
    for j in range(i+1, len(score_list)):
        stat, p = ttest_rel(score_list[i], score_list[j])

        alpha = 0.05

        table[i][j]=np.round(p,3) # upper diagonal
        table[j][i]=np.round(stat,3) # Lower diagonal
        table[i][i]=''

print('The following table shows the comparison of the statistics and the p-values of the models')
print('The value of the statistic on the lower diagonal and the p-value on the upper diagonal')

# Show table, including model names
table = pd.DataFrame(table).set_index(pd.Index([model.split('_')[0].upper() for model in sklearn_models]))
table.columns = [model.split('_')[0].upper() for model in sklearn_models]
table

```

The following table shows the comparison of the statistics and the p-values of the models
The value of the statistic on the lower diagonal and the p-value on the upper diagonal

Out[104]...

	NB	GBM	RF	NNET	CART
NB		0.034	0.224	0.212	0.349
GBM	-2.449		0.742	0.416	0.583
RF	-1.297	-0.339		0.189	0.508
NNET	-1.334	-0.848	-1.41		0.962
CART	-0.983	-0.568	-0.687	-0.049	0.000

4. Interpretability

In [105]...

```

selected_models = ['rf_MA_train', 'GBM_MA_train', 'keras_model_1']

# Create daLEX explainers dict for selected models
explainers = {}
for model in selected_models:
    explainers[model] = dx.Explainer(eval(model), X_train, y_train, label=model)

```

Preparation of a new explainer is initiated

```
-> data                : 36000 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 36000 values
-> model_class         : sklearn.ensemble._forest.RandomForestClassifier (default)
-> label               : rf_MA_train
-> predict function    : <function yhat_proba_default at 0x00000243182EA700> will be used
(default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 0.0, mean = 0.197, max = 1.0
-> model type          : classification will be used (default)
-> residual function   : difference between y and yhat (default)
-> residuals           : min = -0.942, mean = 1.8, max = 4.0
-> model_info          : package sklearn
```

A new explainer has been created!

Preparation of a new explainer is initiated

```
-> data                : 36000 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 36000 values
-> model_class         : sklearn.ensemble._gb.GradientBoostingClassifier (default)
-> label               : GBM_MA_train
-> predict function    : <function yhat_proba_default at 0x00000243182EA700> will be used
(default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 0.000268, mean = 0.202, max = 0.968
-> model type          : classification will be used (default)
-> residual function   : difference between y and yhat (default)
-> residuals           : min = -0.941, mean = 1.8, max = 4.0
-> model_info          : package sklearn
```

A new explainer has been created!

Preparation of a new explainer is initiated

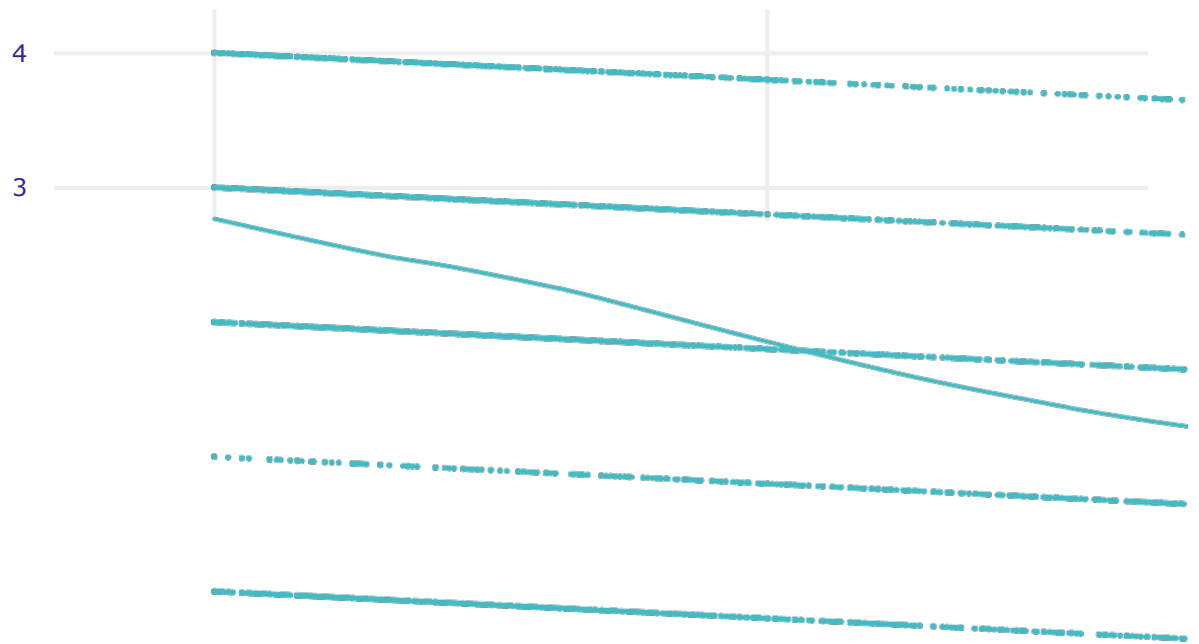
```
-> data                : 36000 rows 32 cols
-> target variable     : Parameter 'y' was a pandas.Series. Converted to a numpy.ndarray.
-> target variable     : 36000 values
-> model_class         : keras.src.models.sequential.Sequential (default)
-> label               : keras_model_1
-> predict function    : <function yhat_tf_classification at 0x00000243182EA980> will be
used (default)
-> predict function    : Accepts pandas.DataFrame and numpy.ndarray.
-> predicted values    : min = 3.8e-23, mean = 0.204, max = 0.951
-> model type          : classification will be used (default)
-> residual function   : difference between y and yhat (default)
-> residuals           : min = -0.918, mean = 1.8, max = 4.0
-> model_info          : package keras
```

A new explainer has been created!

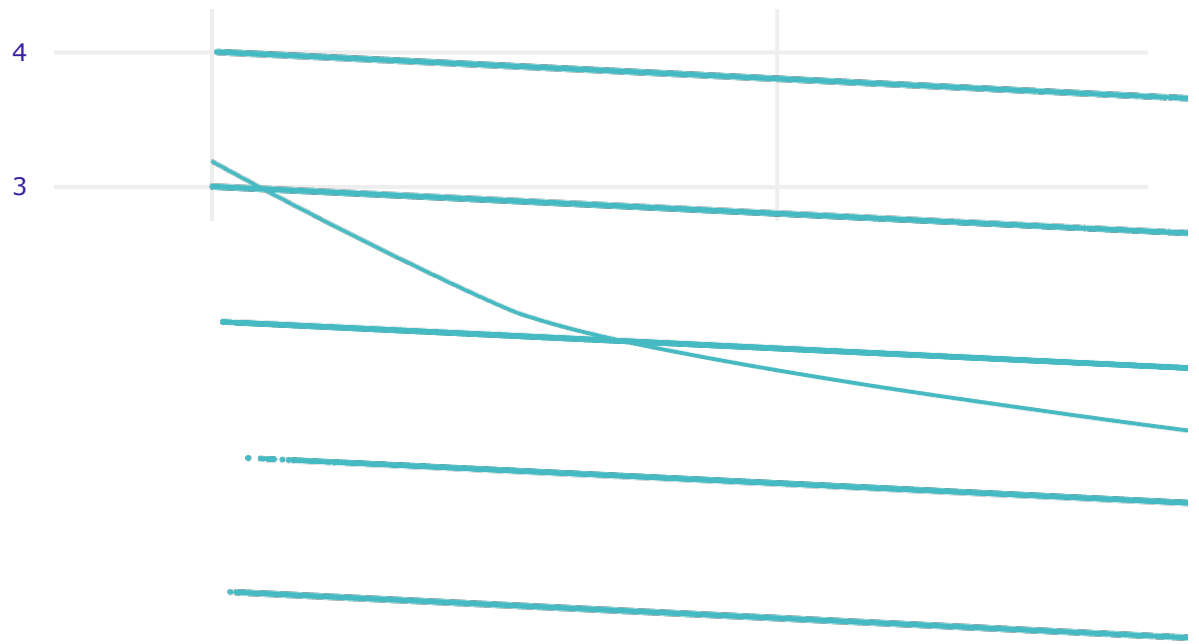
4.1. Residual Diagnostics

```
In [106... # Plot Residual Diagnostics
for explainer in explainers.values():
    explainer.model_diagnostics().plot()
```

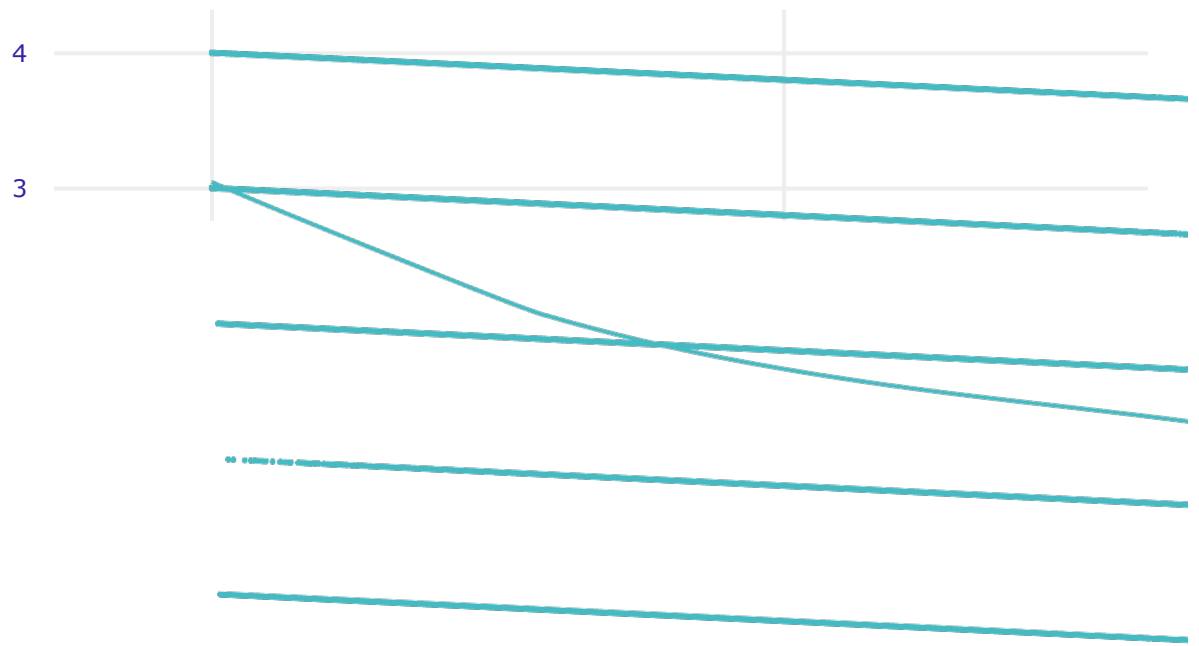
Residual Diagnostics



Residual Diagnostics



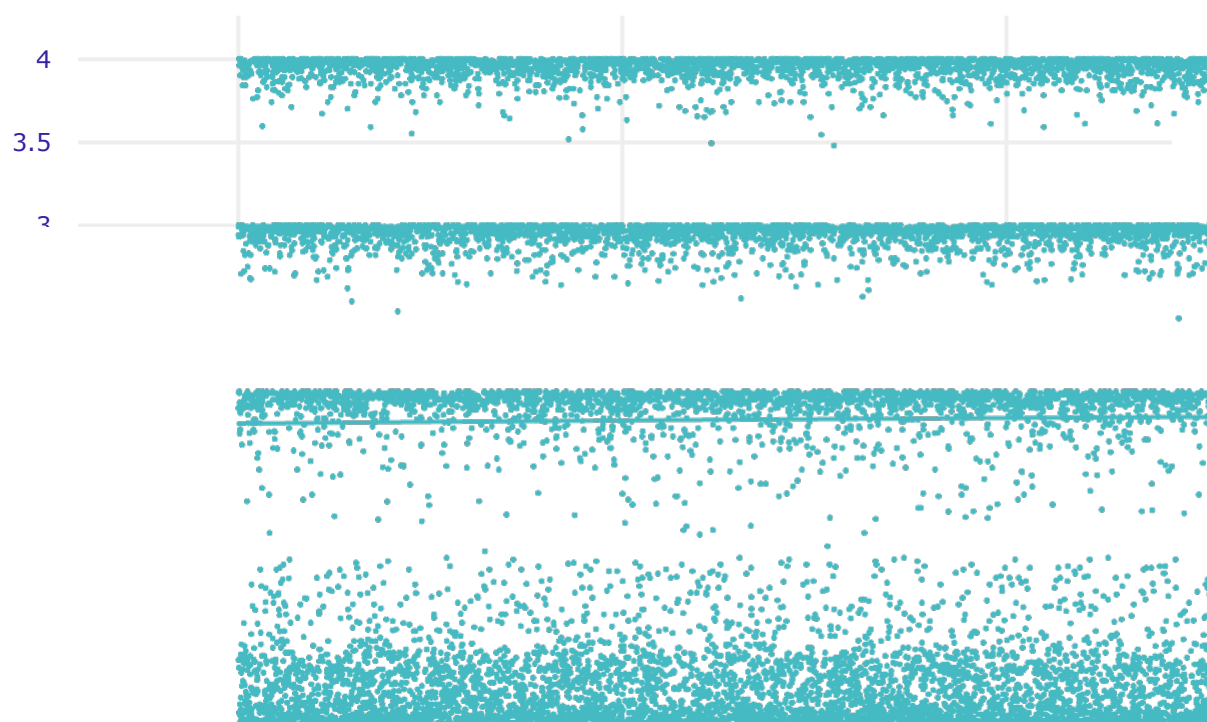
Residual Diagnostics



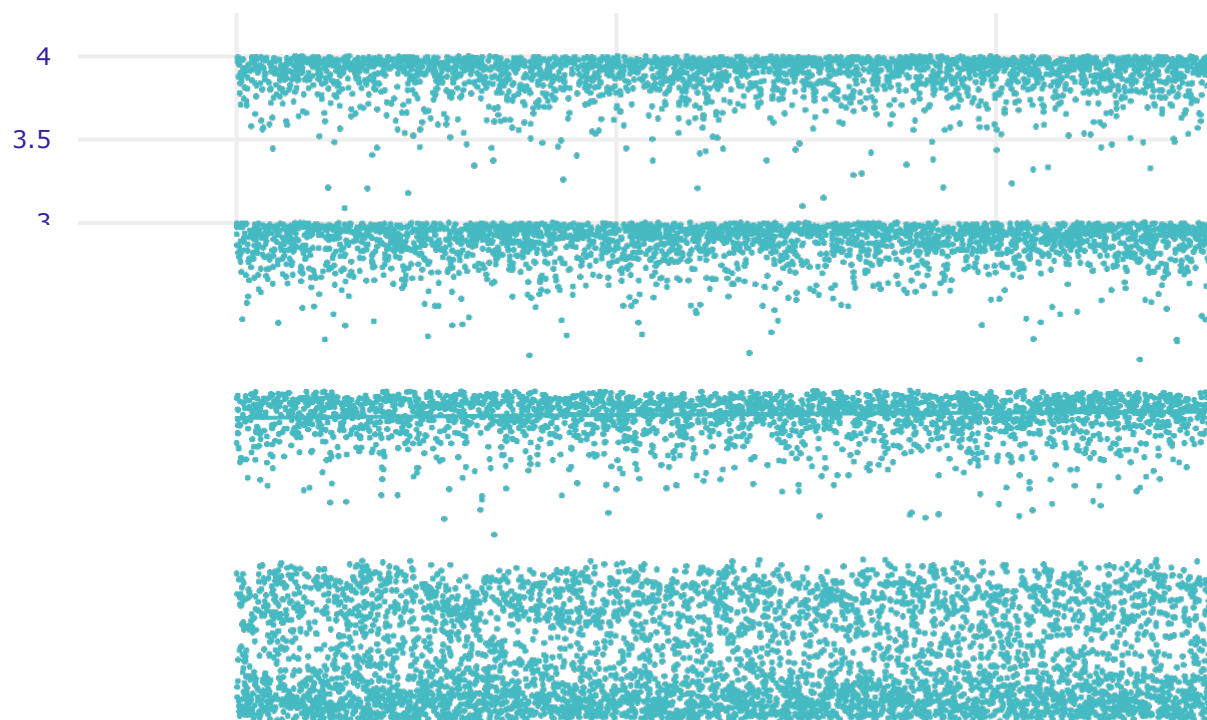
In [107...

```
# Plot Residual Diagnostics (Absolute)
for explainer in explainers.values():
    explainer.model_diagnostics().plot(variable = "ids", yvariable = "abs_residuals")
```

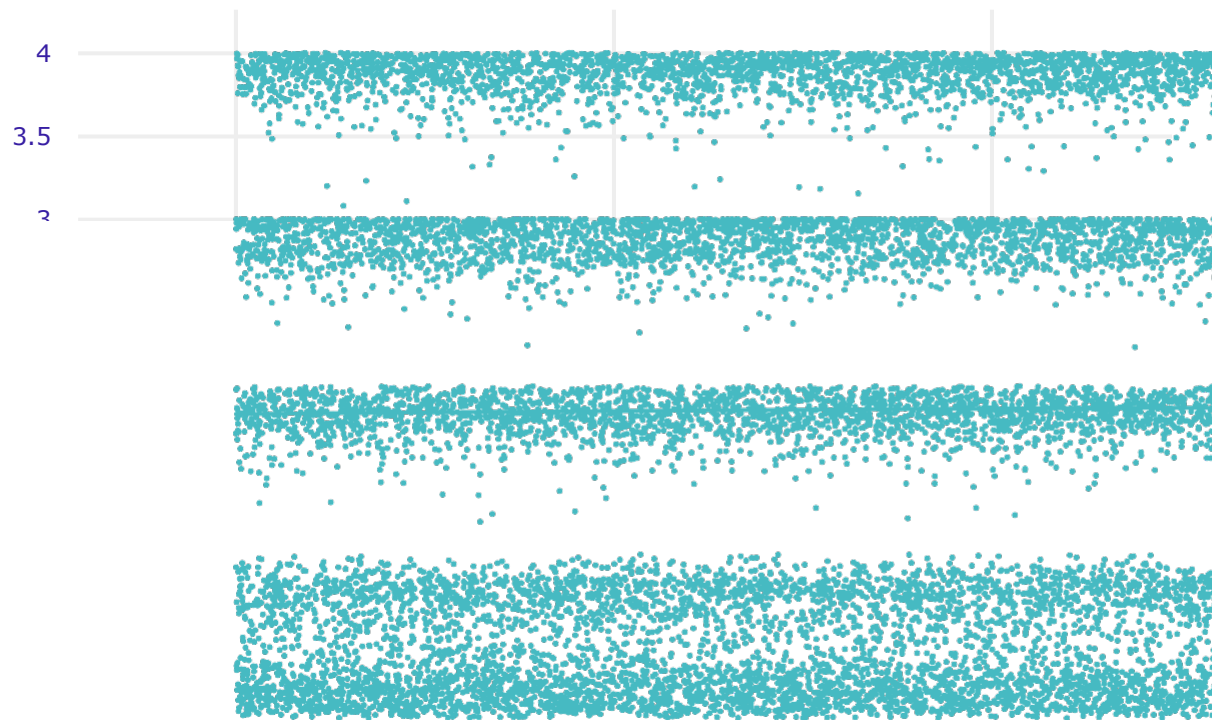
Residual Diagnostics



Residual Diagnostics



Residual Diagnostics



4.2. Global Explainability - Variable importances

```
In [108... # Plot variable importances for selected models
for explainer in explainers.values():
    explainer.model_parts().plot()
```

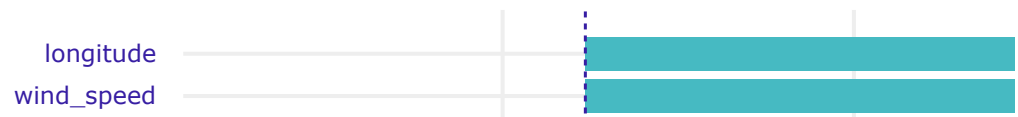
Variable Importance



Variable Importance



Variable Importance



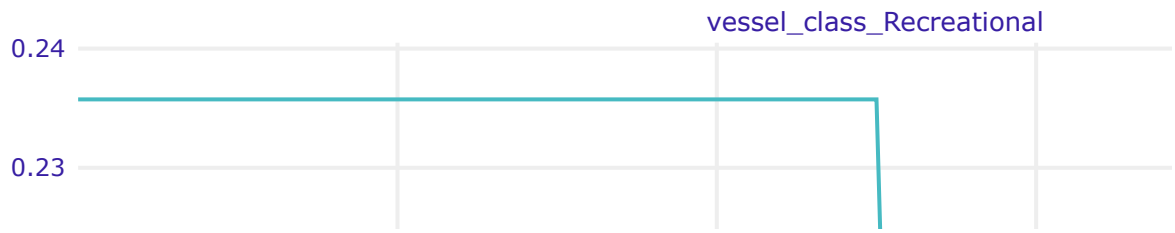
4.3. Partial Dependence Profile (PDP)

In [109...

```
# Plot PDP for selected models
for explainer in explainers.values():
    explainer.model_profile(variables = ["vessel_class_Recreational", "watertype_river"])
```

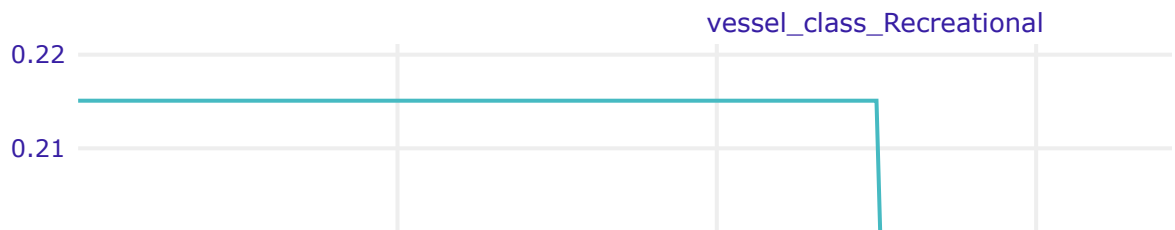
Calculating ceteris paribus: 100% |██████████| 2/2 [00:00<00:00, 6.02it/s]

Aggregated Profiles



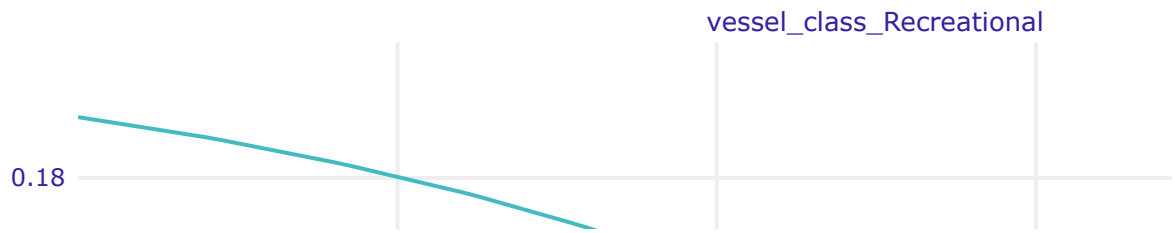
Calculating ceteris paribus: 100% |  | 2/2 [00:00<00:00, 4.67it/s]

Aggregated Profiles



Calculating ceteris paribus: 100% |  | 2/2 [00:01<00:00, 1.90it/s]

Aggregated Profiles



4.4. Break Down profiles

```
In [110... # Select a sample observation
vessel_sample = X_train.sample(n=1, random_state=seed)

# Show this observation
vessel_sample
```

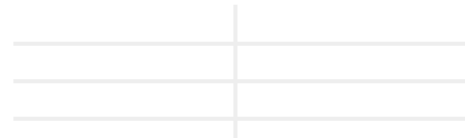
```
Out[110...
      region_Alaska  region_Canada  region_East Coast  region_Gulf of Mexico  region_Mississippi  region_West Coast
35673              0              0              0              0              1              0
```

1 rows × 32 columns

```
In [111... # Plot shapley values for selected observation
for explainer in explainers.values():
    explainer.predict_parts(new_observation = vessel_sample, type = "break_down").plot()
```

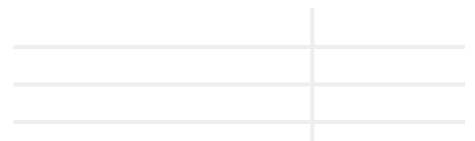
Break Down

intercept
wind_speed = -0.3601
damage_status_Damaged = 1.0



Break Down

intercept
wind_speed = -0.3601
damage_status_Damaged = 1.0



Break Down

intercept	
wind_speed = -0.3601	
activity_id = -0.7739	

4.5. SHapley Additive exPlanations (SHAP)

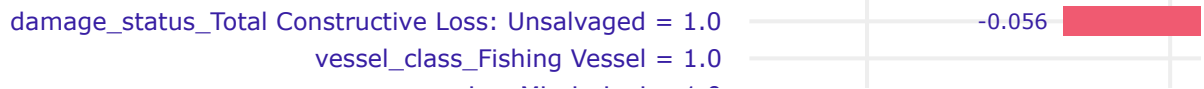
```
In [112... # Calculate predictions (time-consuming process)
if train_model_enabled :
    shaps = {}
    for model in explainers:
        shaps[model] = explainers[model].predict_parts(new_observation = vessel_sample,
        joblib.dump(shaps, models_folder + '/' + 'MA_shaps.pkl')
else:
    shaps = joblib.load(models_folder + '/' + 'MA_shaps.pkl')

# Plot shapley values
for model in explainers:
    shaps[model].plot()
```

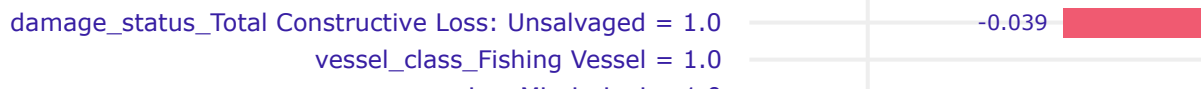
Shapley Values

damage_status_Undamaged = 0.0	-0.042	
vessel_class_Fishing Vessel = 1.0		

Shapley Values



Shapley Values



4.6. Ceteris Paribus profiles

```
In [113... # Plot shapley values for selected observation
for explainer in explainers.values():
    explainer.predict_profile(new_observation = vessel_sample).plot(variables = ["vessel",
```

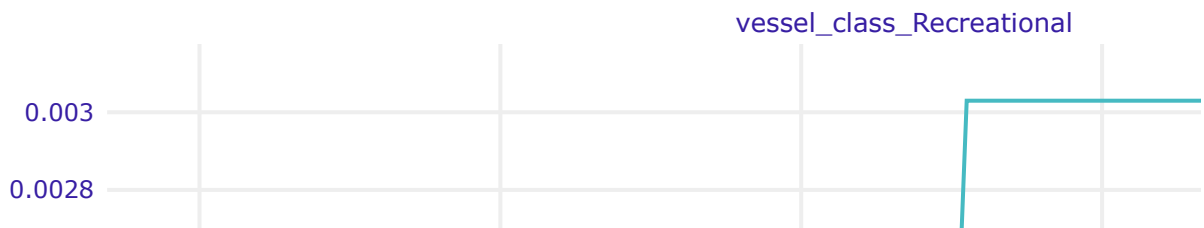
Calculating ceteris paribus: 100%|██████████| 32/32 [00:00<00:00, 290.44it/s]

Ceteris Paribus Profiles



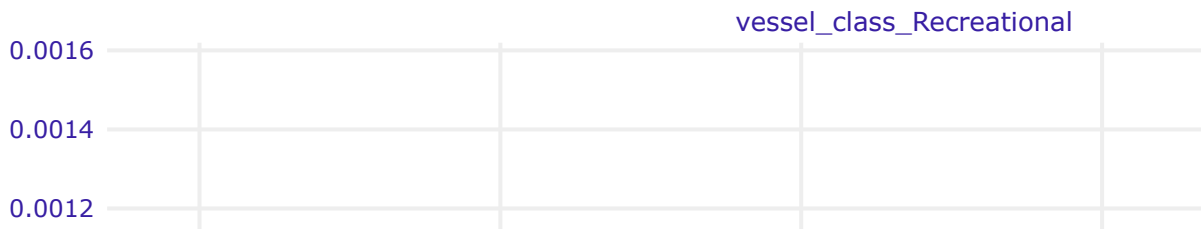
Calculating ceteris paribus: 100% | ██████████ | 32/32 [00:00<00:00, 405.64it/s]

Ceteris Paribus Profiles



Calculating ceteris paribus: 100% | ██████████ | 32/32 [00:00<00:00, 34.84it/s]

Ceteris Paribus Profiles



Session info

```
In [114... # Python version
import sys
print(f'Python version: {sys.version}')

# Initiate variables for package versions
pkgs_dict = {}
version = []

# Loop importing __version__ for each package (because no whole packages are imported)
for pkg in ['pandas', 'numpy', 'sklearn', 'keras', 'tensorflow', 'h2o', 'dalex']:
    exec(f"from {pkg} import __version__ as version")
    pkgs_dict[pkg] = version

# Show as table
pd.DataFrame(pkgs_dict, index=["Package version"])
```

Python version: 3.11.5 | packaged by Anaconda, Inc. | (main, Sep 11 2023, 13:26:23) [MSC v.1916 64 bit (AMD64)]

```
Out[114...      pandas  numpy  sklearn  keras  tensorflow  h2o  dalex
Package version  2.2.2  1.25.2   1.4.2   3.2.0      2.16.1  3.46.0.1  1.7.0
```